



République Algérienne Démocratique et
Populaire

Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique



Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Mémoire de Master

Filière : Informatique

Spécialité

Ingénierie des Logiciels

Thème

CONCEPTION ET RÉALISATION D'UNE PLATEFORME PAAS
POUR LE DÉPLOIEMENT AUTOMATIQUE D'APPLICATIONS WEB
AVEC INTÉGRATION GIT ET GESTION DE RESSOURCES CLOUD

Sujet encadré par :

Mlle Rihane Selma

Soutenu le : ../06/2026

Présenté par :

BOUCHAREB Rami

Devant le jury composé de :

.....

.....

Projet N° 000/2026

Remerciement

Tout d'abord je remercie bien évidemment Dieu le plus miséricordieux de m'avoir accordé la force et la patience pour réaliser ce projet de fin d'études. En second lieu, je tiens à exprimer toute ma gratitude envers mon encadrante dans ce mémoire, Mlle Rihane Selma, pour sa patience, sa disponibilité et surtout ses conseils avisés, qui ont guidé mon travail vers une conclusion réussie. Je tiens également à exprimer ma profonde gratitude envers le corps enseignant de mon université, car sans leur accompagnement, je n'aurais pas acquis toutes les compétences nécessaires à la réalisation de ce projet. Je souhaite également remercier les membres du jury d'avoir accepté d'évaluer ce travail modeste. Enfin, j'adresse mes plus sincères remerciements à tous mes proches, amis et famille, qui ont contribué de près ou de loin à l'achèvement de ce travail. Merci à tous et à toutes.

Résumé

Ce projet vise à concevoir et implémenter une plateforme PaaS innovante pour le déploiement en un clic d'applications web, en utilisant un lien GitHub public ou privé via une authentification OAuth sécurisée. Le déploiement s'effectue dans un environnement cloud orchestré par Kubernetes, qui gère l'orchestration des conteneurs Docker pour assurer scalabilité, haute disponibilité (HA) avec auto-scaling horizontal basé sur les métriques de charge, et une stratégie multi-tenancy adoptant l'approche silo pour isoler complètement les ressources des utilisateurs, garantissant ainsi une sécurité renforcée et une confidentialité des données. L'expérience utilisateur est centrée sur une interface web intuitive et responsive, permettant non seulement de soumettre le dépôt Git pour un build automatique, mais aussi de monitorer en temps réel les logs d'exécution, les métriques de performance (CPU, mémoire, trafic réseau), et de générer dynamiquement des sous-domaines avec activation automatique de certificats SSL via Let's Encrypt pour une accessibilité sécurisée. De plus, la plateforme intègre une gestion avancée des environnements, incluant la configuration de variables d'environnement (env) et de secrets sensibles (comme les clés API ou mots de passe) stockés de manière chiffrée dans Kubernetes Secrets, évitant toute exposition accidentelle. Un historique des déploiements est maintenu, offrant des fonctionnalités de rollback vers des versions précédentes en cas d'erreur, avec un journal détaillé des builds, tests et mises en production pour une traçabilité complète. La persistance des volumes est assurée via des PersistentVolumes dans Kubernetes, permettant aux applications stateful (comme celles avec bases de données) de conserver les données malgré les redéploiements ou scaling. Inspiré de solutions comme Railway, Render ou Vercel, ce système supporte divers langages et frameworks (Node.js, Python, etc.), intègre des pipelines CI/CD automatisés pour des mises à jour continues via webhooks GitHub, et favorise une adoption agile dans des contextes DevOps, réduisant les temps de mise en production tout en optimisant les coûts cloud via une allocation dynamique des ressources, rendant la plateforme adaptée à des environnements hybrides ou multi-cloud.

Mots Clée : PaaS, Déploiement automatique, GitHub, Git, Cloud computing, Kubernetes, Haute disponibilité, SSL.

Table des matières

Table des figures	v
Liste des tableaux	viii
Introduction Générale	1
Structure Du Mémoire	2
1 État de l’Art et Analyse de l’Existant	4
1.1 Le Cloud Computing et les modèles de service	4
1.2 L’orchestration de conteneurs — Docker et Kubernetes	8
1.2.1 La conteneurisation avec Docker et l’OCI	8
1.2.2 Kubernetes comme standard d’orchestration	9
1.2.3 Concepts clés de Kubernetes	10
1.3 Analyse des plateformes PaaS internationales	12
1.4 Analyse des initiatives PaaS algériennes	19
1.4.1 Étude critique des acteurs locaux émergents	19
1.5 Le contexte réglementaire algérien	23
1.6 Le contexte réglementaire algérien	23
1.7 Analyse du marché et identification du besoin	27
1.8 Synthèse et positionnement de Hostifer	31
2 Conception et Architecture du Système	35
2.1 Recueil et spécification des besoins	35
2.1.1 Identification des acteurs	35
2.1.2 Besoins fonctionnels	39
2.1.3 Besoins non fonctionnels	42
2.2 Modélisation UML des cas d’utilisation	45
2.2.1 Diagramme de cas d’utilisation global	45
2.2.2 Cas d’utilisation détaillés	46

2.3	Modélisation des classes et données	49
2.3.1	Diagramme de classes	49
2.3.2	Modèle Entité-Association	50
2.3.3	Description des entités principales	51
2.4	Modélisation comportementale	52
2.4.1	Diagrammes de séquence	52
2.4.2	Diagramme d'états	56
2.5	Architecture technique globale	58
2.5.1	Vue d'ensemble de l'architecture	58
2.5.2	Architecture multi-tenant	58
2.5.3	Architecture Du Réseaux Cloud	61
2.5.4	Architecture de gestion des secrets	62
2.6	Choix technologiques	63
2.6.1	Justification des technologies retenues	63
3	Réalisation et Implémentation	69
3.1	Environnement de développement et de déploiement	69
3.1.1	Infrastructure matérielle	69
3.1.2	Outils de développement	72
3.1.3	Organisation des dépôts GitHub	74
3.2	Implémentation du backend NestJS	77
3.2.1	Structure modulaire de l'application	77
3.2.2	Module d'authentification	78
3.2.3	Module de déploiement et orchestration Argo Workflows	81
3.2.4	Module de streaming des logs	83
3.2.5	Module de gestion des variables d'environnement avec Vault	86
3.2.6	Versioning et rollback des déploiements	86
3.3	Implémentation du builder Go	86
3.3.1	Structure du binaire	86
3.3.2	Détection de framework	88
3.3.3	Génération du plan Railpack	89
3.3.4	Construction et push de l'image via Buildkit	93
3.4	Implémentation de l'infrastructure Kubernetes	95
3.4.1	Chart Helm multi-tenant	95
3.4.2	Pipeline Argo Workflows	102
3.4.3	Configuration de l'observabilité (Loki + Promtail)	104
3.4.4	Gestion des certificats TLS avec cert-manager	107

3.4.5	Collecte des métriques avec Prometheus	109
3.5	Présentation des interfaces utilisateur	112
3.5.1	Page d'accueil (Landing Page)	112
3.5.2	Tableau de bord utilisateur	113
3.5.3	Assistant de déploiement (Deploy Wizard)	113
3.5.4	Gestion des variables d'environnement	113
3.5.5	Interface d'administration Argo Workflows	113
3.6	Tests et validation	113
3.6.1	Tests fonctionnels	113
3.6.2	Tests d'isolation multi-tenant	113
3.6.3	Tests de charge et performance	113
3.6.4	Tests de conformité réglementaire	113
3.7	Bilan et perspectives	119
3.7.1	Fonctionnalités réalisées	119
3.7.2	Difficultés rencontrées	125
3.7.3	Perspectives d'évolution	125
	Conclusion Générale	126
	Bibliographie	127

Table des figures

1.1	Pyramide des modèles de service cloud (IaaS / PaaS / SaaS) avec exemples de plateformes pour chaque niveau	7
1.2	Architecture d'un cluster Kubernetes (Master node, Worker nodes, etcd, API Server)	9
1.3	Cycle de vie d'un Pod Kubernetes (Pending → Running → Succeeded/Failed)	12
1.4	Diagramme TAM/SAM/SOM du marché algérien du cloud pour développeurs	30
1.5	Matrice de positionnement compétitif (axes : qualité technique / conformité algérienne)	34
2.1	Diagramme de cas d'utilisation global montrant tous les acteurs et leurs interactions avec le système (authentification, déploiement, gestion des projets, administration, CI/CD, logs)	46
2.2	Diagramme de cas d'utilisation montrant les interactions spécifiques de l'utilisateur avec le système pour créer un projet, configurer les paramètres, déclencher un déploiement et consulter les journaux en temps réel	47
2.3	Diagramme de cas d'utilisation montrant les interactions spécifiques de l'utilisateur avec le système pour gérer les variables d'environnement	48
2.4	Diagramme de cas d'utilisation montrant les interactions spécifiques de l'utilisateur avec le système pour gérer les volumes de stockage persistants associés à son projet	49
2.5	Diagramme de classes UML complet (User, Project, Deployment, DeploymentStep, EnvVar, Plan, GitHubInstallation, GitHubRepository, ActivityLog, Notification, Domain)	50
2.6	Diagramme Entité-Association de la base de données PostgreSQL avec cardinalités	51
2.7	Diagramme de séquence : Authentification (NextJS → NestJS → PostgreSQL → JWT)	52

2.8	Diagramme de séquence : Déclenchement d'un déploiement (Frontend → NestJS → Argo Workflows → Build Job → Registry → Helm → DNS) . . .	53
2.9	Diagramme de séquence : Streaming des logs en temps réel (Builder Pod → Promtail → Loki → NestJS SSE → Browser)	54
2.10	Diagramme de séquence : Webhook CI/CD GitHub Push (GitHub → NestJS → DeploymentsService → Argo Workflows)	55
2.11	Diagramme d'états d'un Deployment (QUEUED → BUILDING → PROVISIONING → CONFIGURING_DNS → LIVE / FAILED / CANCELLED)	57
2.12	Schéma d'architecture générale de Hostifer : couche client (NextJS), couche API (NestJS), couche orchestration (Argo Workflows), couche infrastructure (Kubernetes, Buildkit, Registry, Loki, Vault), couche réseau (Traefik, Cloudflare, cert-manager)	58
2.13	Schéma d'isolation multi-tenant : Namespace par projet, NetworkPolicy bloquant le trafic inter-tenant, ressources quotas par plan	59
2.14	Architecture réseau cloud : VPC, sous-réseaux publics/privés, Load Balancer, Worker Nodes	62
2.15	Schéma de gestion des secrets avec HashiCorp Vault et External Secrets Operator	63
3.1	Schéma d'organisation des dépôts GitHub et registres OCI	77
3.2	Arborescence du projet NestJS avec description de chaque module (Auth, Users, Projects, Deployments, Logs, EnvVars, GitHub, Plans, Infrastructure)	78
3.3	Capture d'écran de l'interface Argo Workflows UI montrant un workflow de déploiement complet avec les trois étapes	104
3.4	Capture d'écran de la section Hero de la landing page	112
3.5	Capture d'écran de la section Pricing	112
3.6	Capture d'écran de la section comparaison concurrentielle	112
3.7	Capture d'écran du dashboard avec liste des projets et statuts	113
3.8	Capture d'écran de la page de détail d'un projet	113
3.9	Capture d'écran de l'étape 1 : validation de l'URL GitHub et détection du framework	113
3.10	Capture d'écran de l'étape 2 : configuration (région, plan, variables d'environnement)	113
3.11	Capture d'écran de l'étape 3 : streaming des logs de build en temps réel . .	113
3.12	Capture d'écran de l'étape 4 : confirmation du déploiement réussi avec URL live	114

3.13 Capture d'écran de l'interface de gestion des variables d'environnement avec masquage des secrets	114
3.14 Capture d'écran de l'interface Argo Workflows montrant les étapes Build, Provision, DNS d'un déploiement réel	114
3.15 Graphique du temps de déploiement moyen par framework (de la soumis- sion du workflow à l'état LIVE)	114

Liste des tableaux

1.1	Comparaison des responsabilités client/fournisseur dans les trois modèles .	7
1.2	Comparaison des plateformes PaaS internationales	17
1.3	Comparaison des plateformes PaaS algériennes	22
1.4	Synthèse des obligations légales imposées par les lois 18-07 et 25-11 et leur impact sur l'hébergement cloud	26
1.5	Synthèse des besoins non satisfaits et réponse apportée par Hostifer	30
2.1	Synthèse des acteurs du système Hostifer	38
2.2	Tableau des besoins fonctionnels	40
2.3	Tableau des besoins non fonctionnels	43
2.4	Ressources allouées par profil d'allocation et comportements d'autoscaling .	59
2.5	Tableau des choix technologiques	64
3.1	Caractéristiques de l'environnement de déploiement	71
3.2	Environnement de développement	73
3.3	Organisation des dépôts GitHub et artefacts GHCR	76
3.4	Correspondance entre les phases Argo Workflows et les statuts internes Hostifer	83
3.5	Structure du projet builder Go	88
3.6	Ressources Kubernetes provisionnées par le chart tenant-chart	98
3.7	Labels Promtail attachés aux flux de journaux Hostifer	106
3.8	Résultats des tests de déploiement par framework (Express, NestJS, Next.js, Flask, Laravel) : statut, durée de build, mémoire consommée	113
3.9	Résultats des tests d'isolation réseau (tentative de connexion inter-tenant, résultat attendu vs observé)	113
3.10	Tableau de performance : temps de build moyen, latence SSE, consommation mémoire buildkitd pour 1, 3 et 5 builds simultanés, plus métriques d'autoscaling (temps de montée HPA, recommandations VPA, latence d'ajout de nœuds)	114

3.11 Checklist de conformité réglementaire et technique	114
3.12 Tableau de couverture fonctionnelle	119

Introduction Générale

À l'ère de la transformation numérique accélérée, l'infrastructure cloud est devenue le socle indispensable de tout projet logiciel moderne. Les développeurs, startups et entreprises du monde entier s'appuient désormais sur des plateformes de déploiement automatisé pour mettre leurs applications en production rapidement, de manière fiable et sans maîtrise approfondie des mécanismes d'infrastructure sous-jacents. Des solutions de déploiement cloud ont démocratisé ce paradigme à l'échelle internationale, offrant une expérience de déploiement fluide résumée en quelques clics.

Cependant, l'écosystème numérique algérien se heurte à des obstacles structurels qui rendent ces plateformes internationales difficilement accessibles. Les cartes bancaires algériennes sont fréquemment rejetées sur les plateformes étrangères, les frais de change imposent une surcharge financière significative, et la latence introduite par des serveurs situés en Europe ou aux États-Unis dégrade l'expérience des utilisateurs finaux locaux. Plus grave encore, les législations algériennes en vigueur — notamment la loi 18-07 relative à la protection des personnes physiques dans le traitement des données à caractère personnel, et la loi 25-11 portant sur la souveraineté numérique — imposent des contraintes réglementaires croissantes sur l'hébergement des données à l'étranger, exposant les entreprises locales à des risques de conformité non négligeables.

C'est dans ce contexte que s'inscrit **Hostifer**, la première plateforme algérienne de type *Platform-as-a-Service* (PaaS) reposant sur une architecture Kubernetes native. Hostifer a pour vocation de combler le fossé technologique entre les développeurs algériens et les standards internationaux de déploiement cloud, en proposant une solution entièrement hébergée en Algérie, facturée en dinars algériens (DZD), et conçue pour être conforme aux exigences réglementaires nationales.

La plateforme permet à tout développeur, qu'il soit étudiant, freelance ou membre d'une startup, de déployer une application web en moins de cinq minutes à partir d'une simple URL GitHub, sans aucune connaissance préalable en administration système ou en orchestration de conteneurs. Le processus est entièrement automatisé : détection du framework applicatif, construction de l'image Docker via Railpack et Buildkit, provision-

nement d'un environnement Kubernetes isolé, configuration du certificat SSL, et création d'un sous-domaine dédié — le tout orchestré par Argo Workflows et supervisé en temps réel par un système de streaming de logs basé sur Loki et Promtail.

Notre projet vise ainsi à répondre à plusieurs enjeux simultanément :

- **Un enjeu technique** : concevoir une architecture multi-tenant robuste, isolée et scalable, exploitant les primitives natives de Kubernetes telles que les *Namespaces*, les *NetworkPolicies*, les *ResourceQuotas* et les *LimitRanges*, pour garantir une isolation stricte entre les projets des différents utilisateurs.
- **Un enjeu économique** : proposer une tarification transparente en dinars algériens, compatible avec les moyens de paiement locaux (carte CIB, Edahabia, BaridiMob, virement bancaire), et offrir un palier gratuit pour favoriser l'adoption auprès des développeurs étudiants et indépendants.
- **Un enjeu réglementaire** : assurer la conformité avec les lois algériennes sur la protection des données et la souveraineté numérique, en hébergeant l'intégralité de l'infrastructure sur le cloud algérien DjazzyCloud, opéré en partenariat avec Alibaba Cloud.
- **Un enjeu de souveraineté numérique** : contribuer à l'émergence d'une infrastructure cloud locale de qualité professionnelle, réduisant la dépendance des acteurs numériques algériens vis-à-vis des hébergeurs étrangers.

Structure Du Mémoire

Ce mémoire détaille la conception, le développement et la mise en œuvre de la plateforme Hostifer. Il s'articule autour d'une introduction générale, de trois chapitres principaux, d'une conclusion générale et d'une bibliographie.

— CHAPITRE 1

Nous dressons un état de l'art des plateformes PaaS existantes, en analysant les solutions internationales leaders du marché ainsi que les initiatives locales algériennes émergentes. Cette analyse comparative nous permet d'identifier les lacunes du marché algérien, de justifier la pertinence du projet Hostifer, et de définir les exigences fonctionnelles et non fonctionnelles auxquelles la plateforme doit répondre.

— **CHAPITRE 2**

Nous présentons la phase de conception de la plateforme, en décrivant les choix architecturaux retenus : l'architecture multi-tenant sur Kubernetes, le pipeline de déploiement orchestré par Argo Workflows, la gestion des secrets via HashiCorp Vault, et la stratégie de collecte et de streaming des logs applicatifs. Des diagrammes UML et des schémas d'architecture accompagnent cette description pour en illustrer les interactions et les flux de données.

— **CHAPITRE 3**

Nous abordons la phase de réalisation, en détaillant les technologies utilisées pour le développement du backend (NestJS), du frontend (Next.js), et de l'infrastructure (k3s, Traefik, cert-manager, Buildkit, Railpack, Loki, Promtail). Nous présentons les interfaces de la plateforme, le processus de déploiement de bout en bout, ainsi que les résultats des tests fonctionnels réalisés sur des applications représentatives de différents frameworks supportés.

Chapitre 1

État de l’Art et Analyse de l’Existant

Ce chapitre positionne Hostifer dans le contexte technologique mondial et algérien. Il présente les modèles de service cloud fondamentaux, analyse les solutions existantes sur le marché international et local, et identifie les lacunes que Hostifer entend combler. Cette analyse constitue la justification technique et économique du projet.

1.1 Le Cloud Computing et les modèles de service

Le cloud computing représente l’une des mutations les plus profondes de l’informatique moderne. Le NIST (*National Institute of Standards and Technology*) en donne la définition de référence : le cloud computing est un modèle permettant un accès réseau ubiquitaire, pratique et à la demande à un pool partagé de ressources informatiques configurables (réseaux, serveurs, stockage, applications et services) pouvant être rapidement provisionnées et libérées avec un effort de gestion minimal ou une interaction réduite avec le fournisseur de services. Ce modèle repose sur cinq caractéristiques essentielles : le libre-service à la demande, l’accès réseau large bande, la mutualisation des ressources, l’élasticité rapide, et la mesure du service [1].

Sur la base de cette définition, le NIST distingue trois modèles de service fondamentaux — IaaS, PaaS et SaaS — qui structurent l’ensemble de l’offre cloud commerciale et académique, et qui se différencient par le niveau d’abstraction fourni au consommateur et par la répartition des responsabilités entre le fournisseur et l’utilisateur [2].

Infrastructure as a Service (IaaS)

L’IaaS est une forme de cloud computing qui fournit un accès à la demande à des ressources informatiques hébergées dans le cloud — calcul, stockage et réseau — soit l’infrastructure IT sous-jacente nécessaire à l’exécution des applications et des charges

de travail dans le cloud. Dans ce modèle, le fournisseur cloud gère les couches physiques (matériel, datacenter, virtualisation), tandis que le consommateur conserve le contrôle total sur les systèmes d'exploitation, le stockage, les applications déployées et, partiellement, les composants réseau [1]. L'IaaS offre la plus grande flexibilité des trois modèles, au prix d'une responsabilité opérationnelle importante : le consommateur doit administrer lui-même les machines virtuelles, les patches de sécurité, la haute disponibilité et la mise à l'échelle. Des exemples représentatifs de ce modèle incluent Amazon EC2, Google Compute Engine, Microsoft Azure Virtual Machines et Alibaba Cloud ECS.

Platform as a Service (PaaS)

Le NIST définit le PaaS comme un modèle dans lequel le consommateur déploie sur l'infrastructure cloud des applications qu'il a créées ou acquises, en utilisant les langages de programmation, bibliothèques, services et outils supportés par le fournisseur. Le consommateur ne gère ni ne contrôle l'infrastructure cloud sous-jacente, incluant le réseau, les serveurs, les systèmes d'exploitation ou le stockage, mais dispose d'un contrôle sur les applications déployées et éventuellement sur les paramètres de configuration de l'environnement d'hébergement applicatif.

Le PaaS constitue ainsi une couche d'abstraction intermédiaire entre l'IaaS et le SaaS : il masque la complexité de l'infrastructure tout en laissant au développeur le contrôle complet de son code applicatif. L'idée fondatrice du PaaS est d'abstraire autant que possible l'infrastructure, afin que les développeurs puissent se concentrer sur la construction et le déploiement de leurs applications, sans gérer les serveurs sous-jacents. Ce paradigme a été popularisé par Heroku, la première plateforme à avoir rendu le déploiement d'applications web accessible sans compétences en administration système, introduisant ainsi le concept de PaaS à la communauté des développeurs au sens large. Une nouvelle génération de plateformes PaaS — Vercel, Railway, Render, Fly.io — a depuis émergé, offrant une expérience développeur améliorée, une tarification plus transparente et un support natif des conteneurs [3].

Software as a Service (SaaS)

Le NIST définit le SaaS comme un modèle de service dans lequel le consommateur ne gère ni ne contrôle l'infrastructure cloud sous-jacente, incluant le réseau, les serveurs, les systèmes d'exploitation, le stockage, ni même les capacités individuelles de l'application, à l'exception possible de paramètres de configuration applicatifs limités propres à l'utilisateur. Le SaaS représente le niveau d'abstraction maximal : l'utilisateur consomme une

application complète livrée via un navigateur web ou une API, sans aucune responsabilité technique sur la pile logicielle ou l'infrastructure. Des exemples emblématiques incluent Google Workspace, Microsoft 365, Salesforce, et Notion.

Le modèle de responsabilité partagée

La distinction fondamentale entre ces trois modèles réside dans la répartition des responsabilités entre le fournisseur et le consommateur. La Figure 1.1 illustre cette pyramide de responsabilités : plus on monte vers le SaaS, plus le fournisseur prend en charge de couches, et moins le consommateur a de contrôle mais aussi de charge opérationnelle. Le Tableau 1.1 détaille cette répartition couche par couche pour les trois modèles.

Le SaaS offre un confort maximal avec une gestion minimale, le PaaS apporte une flexibilité de développement, et l'IaaS fournit un contrôle de l'infrastructure avec une responsabilité opérationnelle complète. De nombreuses organisations utilisent simultanément plusieurs modèles de service pour optimiser différentes fonctions métiers.

Positionnement de Hostifer dans le modèle PaaS

Hostifer se positionne clairement dans la catégorie **PaaS**, en adoptant les principes fondamentaux de ce modèle tout en les instanciant sur une infrastructure Kubernetes native. Comme les plateformes PaaS de référence, Hostifer abstrait intégralement la gestion de l'infrastructure sous-jacente — provisionnement des conteneurs, configuration réseau, émission des certificats TLS, gestion des secrets — et n'expose à l'utilisateur qu'une interface simple orientée vers le code source : fournir une URL de dépôt GitHub suffit pour déclencher le déploiement complet d'une application. Le développeur conserve le contrôle total de son code applicatif, de ses variables d'environnement et de la configuration de son pipeline CI/CD, sans jamais interagir directement avec les primitives Kubernetes.

Ce positionnement PaaS se distingue cependant des offres internationales existantes sur deux axes déterminants. D'une part, Hostifer repose sur une infrastructure Kubernetes *multi-tenant native*, où chaque application est isolée dans son propre namespace Kubernetes avec des politiques réseau, des quotas de ressources et une gestion des secrets dédiés — une granularité d'isolation que les PaaS traditionnels basés sur des conteneurs partagés ne proposent pas systématiquement. D'autre part, Hostifer est conçu pour et hébergé en Algérie, avec une tarification en dinars algériens (DZD) et une conformité aux lois 18-07 et 25-11 sur la protection des données personnelles — des caractéristiques structurellement impossibles à offrir pour les plateformes étrangères et absentes des initiatives locales existantes, comme analysé dans les sections suivantes.

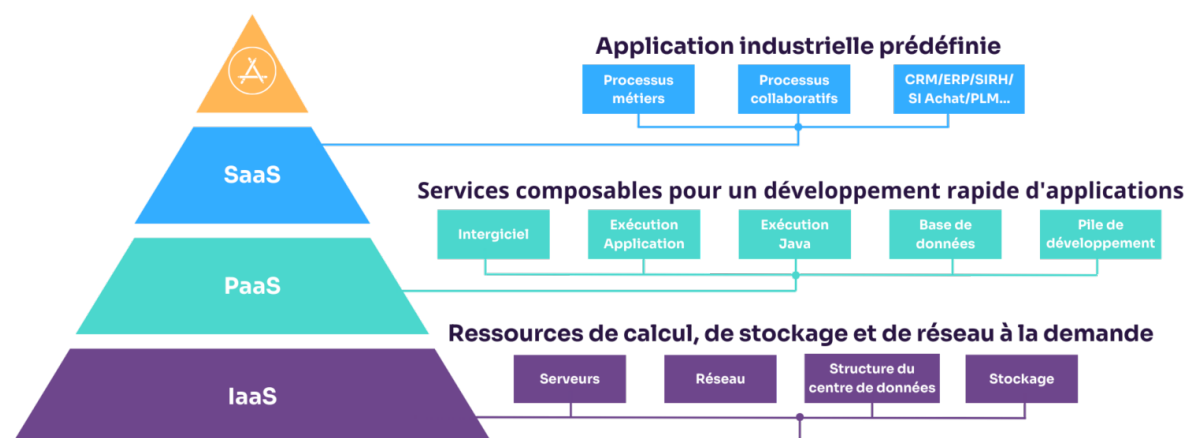


FIGURE 1.1 – Pyramide des modèles de service cloud (IaaS / PaaS / SaaS) avec exemples de plateformes pour chaque niveau

TABLEAU 1.1 – Comparaison des responsabilités client/fournisseur dans les trois modèles

Critère	IaaS	PaaS	SaaS
Infrastructure physique	Fournisseur	Fournisseur	Fournisseur
Système d'exploitation	Client	Fournisseur	Fournisseur
Plateforme d'exécution / middle-ware	Client	Fournisseur	Fournisseur
Application	Client	Client	Fournisseur
Données et configuration métier	Client	Client	Client / partiellement fournisseur selon le service

1.2 L'orchestration de conteneurs — Docker et Kubernetes

1.2.1 La conteneurisation avec Docker et l'OCI

La conteneurisation est une technologie de virtualisation légère qui permet d'encapsuler une application et l'ensemble de ses dépendances — bibliothèques, fichiers de configuration, variables d'environnement — dans une unité portable et autonome appelée conteneur. Contrairement à la virtualisation traditionnelle basée sur des machines virtuelles, un conteneur partage le noyau du système d'exploitation hôte et n'embarque que les couches logicielles strictement nécessaires à l'application. Cette architecture réduit considérablement l'empreinte mémoire, accélère le démarrage des instances et garantit la reproductibilité de l'environnement d'exécution entre les phases de développement, de test et de production [4].

Docker, lancé en 2013 par la société dotCloud (rebaptisée Docker Inc.), a popularisé la conteneurisation auprès de la communauté des développeurs en proposant un outillage accessible pour la construction, la distribution et l'exécution de conteneurs. Son succès a conduit à la création, en juin 2015, de l'*Open Container Initiative* (OCI) sous l'égide de la Linux Foundation. L'OCI a été lancée par Docker, CoreOS et d'autres acteurs de l'industrie des conteneurs dans le but de créer des standards ouverts pour les formats de conteneurs et les runtimes. Elle comprend aujourd'hui trois spécifications : la spécification de runtime (*runtime-spec*), la spécification d'image (*image-spec*) et la spécification de distribution (*distribution-spec*). Ces standards garantissent l'interopérabilité entre les outils de l'écosystème : une image construite avec Docker peut être exécutée par containerd, Podman ou tout autre runtime conforme OCI, et stockée dans n'importe quel registre OCI-compatible.

Une image OCI est structurée en couches superposées (*layers*) immuables, chaque couche représentant un ensemble de modifications du système de fichiers par rapport à la couche précédente. Ce mécanisme permet la réutilisation et la mise en cache des couches communes entre différentes images, réduisant les temps de téléchargement et l'espace de stockage consommé. Dans le contexte de Hostifer, le daemon Buildkitd exploite précisément cette structure en couches pour optimiser les builds répétés d'une même application via un cache de couches persistant [5].

1.2.2 Kubernetes comme standard d'orchestration

La conteneurisation seule ne résout pas les problèmes opérationnels liés au déploiement à grande échelle : comment assurer la haute disponibilité, la mise à l'échelle automatique, le rééquilibrage de charge, la gestion des pannes et la mise à jour sans interruption d'un ensemble de conteneurs distribués sur une flotte de machines ? Ces problématiques sont adressées par les orchestrateurs de conteneurs, dont Kubernetes (**k8s**) s'est imposé comme le standard industriel de référence [6].

Kubernetes est un projet open source initialement développé par Google, publié en 2014 et confié à la Cloud Native Computing Foundation (CNCF) en 2016. Il automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées sur un cluster de machines. Son architecture repose sur un plan de contrôle (*control plane*) composé de l'API Server, du Scheduler, du ControllerManager et d'etcd (le magasin de données distribué de l'état du cluster), et d'un plan de données (*data plane*) constitué des nœuds de travail (*worker nodes*) exécutant les conteneurs applicatifs via un *kubelet* et un runtime conforme CRI (*Container Runtime Interface*). La Figure 1.2 illustre cette architecture.

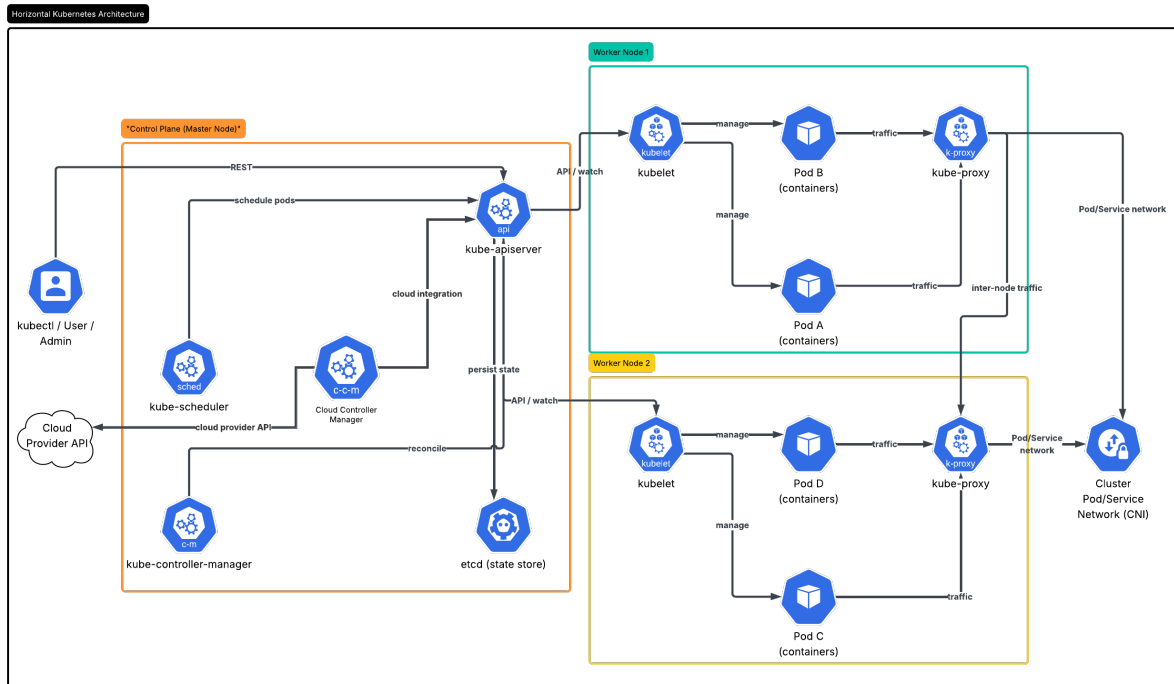


FIGURE 1.2 – Architecture d'un cluster Kubernetes (Master node, Worker nodes, etcd, API Server)

1.2.3 Concepts clés de Kubernetes

La compréhension de l'implémentation de Hostifer nécessite la maîtrise de sept ressources Kubernetes fondamentales, décrites ci-après.

Pod. Le Pod est l'unité d'exécution atomique de Kubernetes. Il encapsule un ou plusieurs conteneurs partageant le même espace réseau (adresse IP unique, ports) et le même espace de stockage (*volumes*). Kubernetes garantit que les conteneurs au sein d'un Pod partagent le même namespace réseau et peuvent communiquer entre eux via `localhost`. Dans la grande majorité des cas, un Pod contient un seul conteneur applicatif. Les Pods sont éphémères : s'ils tombent en panne, Kubernetes les recrée automatiquement, potentiellement sur un nœud différent, d'où l'importance des ressources d'abstraction supérieures telles que les Deployments et les Services.

Namespace. Un Namespace est une partition logique du cluster Kubernetes, permettant d'isoler des groupes de ressources au sein d'un même cluster physique. Les Namespaces constituent un excellent moyen de séparer des environnements (développement, staging, production) ou d'isoler les workloads de différentes équipes. Ils servent de périmètre d'application pour les politiques de sécurité réseau, les quotas de ressources et les règles RBAC. Dans Hostifer, chaque application déployée par un utilisateur est provisionnée dans un Namespace Kubernetes dédié, réalisant ainsi le modèle d'isolation silo décrit dans la section 2.5.2 [6].

Deployment. Un Deployment est un contrôleur Kubernetes qui gère le cycle de vie d'un ensemble de Pods identiques (*ReplicaSet*). Il garantit qu'un nombre spécifié de réplicas du Pod est en cours d'exécution à tout moment, gère les mises à jour progressives (*rolling updates*) sans interruption de service, et assure le retour arrière vers une version précédente en cas d'échec. Le Deployment est la ressource de déploiement standard pour les applications stateless dans Kubernetes.

Service. Un Service est une abstraction réseau qui expose un ensemble de Pods sous une adresse IP virtuelle stable et un nom DNS fixe, indépendamment des adresses IP éphémères des Pods individuels. Les Pods étant dynamiques et leurs adresses IP pouvant être réassignées lors des mises à l'échelle ou des redémarrages, un Service agit comme un endpoint stable qui abstrait les IPs des Pods et offre un équilibrage de charge, une découverte de service et une abstraction réseau. Kubernetes propose plusieurs types de Services : **ClusterIP** (accessible uniquement en interne au cluster), **NodePort** (exposé sur un port de chaque nœud), et **LoadBalancer** (intégrant un équilibreur de charge cloud externe).

Ingress. L'Ingress est une ressource Kubernetes qui gère l'accès HTTP et HTTPS depuis l'extérieur du cluster vers les Services internes. L'Ingress permet de définir des

règles de routage basées sur l'URL pour distribuer les routes HTTP/HTTPS vers différents Services backend ; les requêtes sont ensuite transmises aux Pods correspondants de chaque Service. L'Ingress nécessite un contrôleur d'entrée (*Ingress Controller*) déployé dans le cluster pour implémenter ces règles ; Hostifer utilise Traefik comme contrôleur d'entrée, qui expose des ressources CRD étendues telles que l'**IngressRoute** décrite dans la section 3.4.1.

ResourceQuota. Un ResourceQuota définit des contraintes agrégées sur les ressources consommables dans un Namespace. Les ResourceQuotas plafonnent la consommation totale de ressources d'un namespace en définissant des limites dures sur les `requests.cpu`, `requests.memory`, les `limits.cpu` et `limits.memory`, ainsi que sur le nombre total d'objets Kubernetes de différents types. Lorsqu'un utilisateur tente de créer une ressource qui dépasse le quota défini, la création est rejetée par l'API server. Dans Hostifer, les ResourceQuotas sont utilisés pour garantir que chaque tenant ne peut consommer que les ressources correspondant à son plan tarifaire souscrit, préservant l'isolation et l'équité entre tenants.

NetworkPolicy. Une NetworkPolicy est une ressource Kubernetes qui définit les règles de contrôle du trafic réseau entre les Pods ou entre les Pods et des endpoints externes. Une NetworkPolicy agit comme un pare-feu, appliquant des règles d'entrée (ingress) et de sortie (egress) pour sécuriser le trafic dans le cluster. Par défaut, sans aucune NetworkPolicy appliquée, tous les Pods d'un cluster Kubernetes peuvent communiquer librement entre eux. L'application d'une NetworkPolicy de type *default-deny* transforme ce modèle permissif en un modèle *allowlist* strict, où seul le trafic explicitement autorisé peut transiter. Il convient de noter qu'une NetworkPolicy n'a d'effet que si un plugin réseau capable d'appliquer ces politiques est installé dans le cluster — parmi les plus courants : Calico, Cilium, Kube-router et Weave Net. L'implémentation Hostifer s'appuie sur ce mécanisme pour garantir l'isolation réseau inter-tenant décrite dans la section 2.5.2.

Le cycle de vie d'un Pod Kubernetes, depuis son état **Pending** jusqu'aux états terminaux **Succeeded** ou **Failed**, est illustré dans la Figure 1.3.

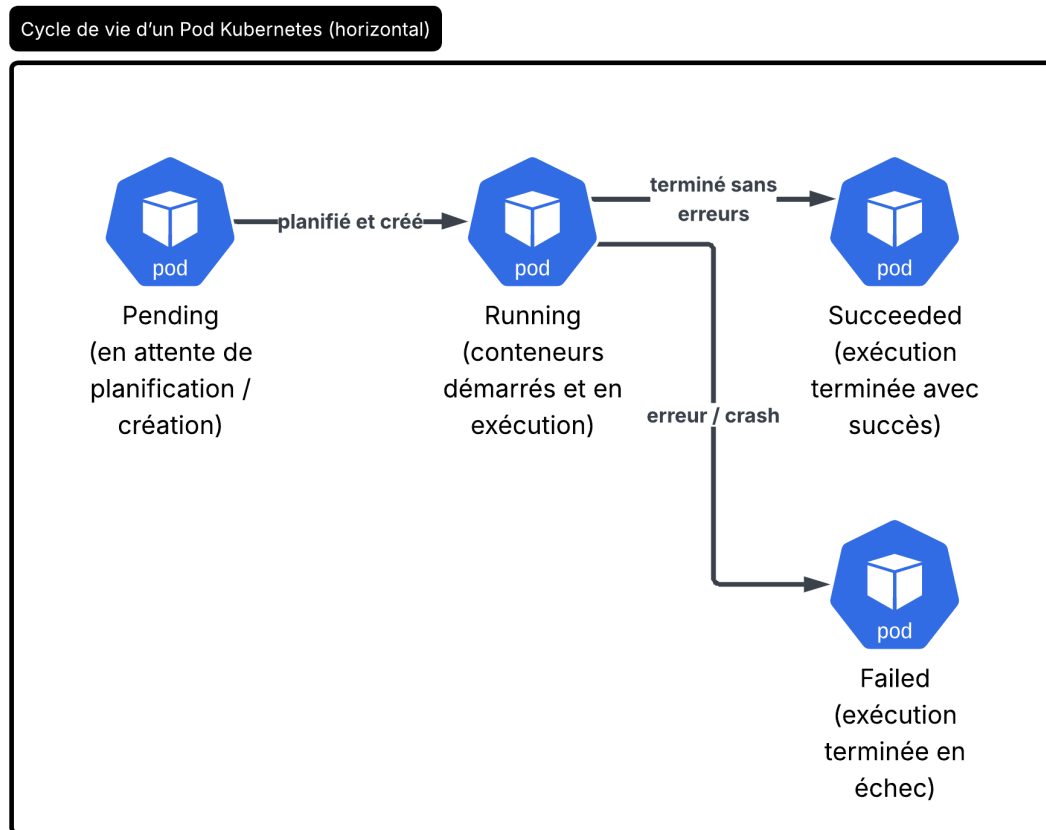


FIGURE 1.3 – Cycle de vie d'un Pod Kubernetes (Pending → Running → Succeeded/Failed)

1.3 Analyse des plateformes PaaS internationales

Le marché des plateformes PaaS pour développeurs a connu une croissance significative depuis 2010, portée d'abord par Heroku qui a défini le paradigme du déploiement depuis un dépôt Git, puis par une nouvelle génération de plateformes nées de la fin du tier gratuit Heroku en 2022. Cette section analyse les six plateformes les plus représentatives du marché international, en évaluant pour chacune son modèle de déploiement, ses technologies sous-jacentes, sa tarification et ses limitations structurelles pour le contexte algérien [3].

Vercel

Vercel est une plateforme PaaS fondée en 2015, initialement sous le nom ZEIT, et créatrice du framework Next.js. Vercel se concentre sur les développeurs frontend et l'architecture Jamstack. Elle excelle dans le déploiement de sites statiques et de fonctions

serverless, et offre une excellente expérience développeur avec une interface de déploiement soignée. Son modèle de déploiement est hybride : les assets statiques sont distribués directement sur un CDN mondial, tandis que les fonctions backend sont exécutées en tant que fonctions serverless dans des conteneurs légers de type Lambda sur l'infrastructure AWS sous-jacente.

La force de Vercel réside dans son intégration native avec Next.js et son réseau edge mondial qui garantit des temps de réponse très faibles pour les applications frontend. Cependant, Vercel n'est pas adapté aux charges de travail longue durée, incluant le traitement de données ETL, le transcodage média, la génération de rapports, les tâches DevOps/infrastructure, ou toute charge nécessitant une connexion persistante. Sa tarification, exprimée en USD avec un plan pro à 20 \$/mois, est inaccessible via les moyens de paiement algériens standards, et ses serveurs sont localisés exclusivement aux États-Unis, en Europe et en Asie, ce qui exclut toute conformité à la loi 18-07 sur la localisation des données personnelles des ressortissants algériens.

Railway

Railway est une plateforme PaaS lancée en 2020 qui cible les développeurs souhaitant déployer rapidement des applications backend complètes, incluant des bases de données. Railway est la meilleure alternative à Heroku pour les applications de petite et moyenne taille. Elle propose une configuration plus facile, une meilleure tarification et une expérience développeur plus moderne. Railway supporte le déploiement depuis un Dockerfile ou via la détection automatique de stack via Nixpacks ou Railpacks.

Sur le plan technique, Railway est précisément l'un des créateurs de Railpack — l'outil de détection de framework et de génération de plans de build utilisé dans Hostifer — ce qui témoigne de son investissement dans l'outillage de build automatisé. Railway propose une configuration en un clic pour PostgreSQL, MySQL, Redis et MongoDB, et permet de déployer toute base de données fonctionnant dans un conteneur Docker. Sa tarification est basée sur la consommation (*usage-based*), avec une facturation à la seconde d'utilisation des ressources. Comme pour l'ensemble des plateformes internationales analysées, la tarification est exclusivement en USD, les régions de déploiement disponibles sont limitées à l'Amérique du Nord, l'Europe et l'Asie, et aucun mécanisme de conformité réglementaire algérien n'est prévu ou envisageable dans leur modèle économique.

Render

Render est une plateforme PaaS lancée en 2019 qui se positionne comme une alternative à Heroku avec une tarification mensuelle prévisible par instance. Render propose un ensemble de fonctionnalités similaire à Heroku avec quelques améliorations, mais maintient une tarification traditionnelle basée sur les instances, ce qui la rend plus prévisible que les modèles usage-based de Railway ou Fly.io. Elle supporte le déploiement d'applications web, de workers en arrière-plan, de tâches cron et de bases de données PostgreSQL managées.

Render se distingue par son support natif du déploiement multi-région et par une interface utilisateur claire orientée vers les développeurs débutants. Elle offre également une gestion des *Infrastructure as Code* via des fichiers `render.yaml` (*blueprints*), permettant de versionner la configuration d'infrastructure dans le dépôt Git. Ses limitations pour le contexte algérien sont identiques aux autres plateformes internationales : facturation en USD, absence de région Afrique du Nord, et aucune conformité possible avec les obligations de résidence des données imposées par la loi 25-11.

Heroku

Heroku, acquis par Salesforce en 2010, est la plateforme qui a fondé et popularisé le paradigme PaaS tel qu'il est connu aujourd'hui. Heroku est une plateforme PaaS mature qui abstrait la complexité de l'infrastructure, permettant aux développeurs de se concentrer sur le code applicatif. Elle supporte plusieurs langages de programmation et fournit un écosystème robuste d'add-ons pour étendre les applications. Son modèle de déploiement repose sur le concept de *dynos* — des conteneurs légers exécutant les processus déclarés dans un fichier `Procfile`.

La fin du tier gratuit Heroku en novembre 2022 a provoqué une migration massive de ses utilisateurs vers Railway, Render et Fly.io. Sa tarification, significativement plus élevée que ses concurrents modernes pour des fonctionnalités équivalentes, combinée à une dette technologique accumulée depuis plus d'une décennie, a considérablement érodé sa position sur le marché des développeurs indépendants. Pour le contexte algérien, Heroku cumule toutes les limitations des plateformes internationales : tarification USD, infrastructure hébergée sur AWS (régions US/EU/AP), et absence totale de mécanismes de conformité réglementaire locale.

Fly.io

Fly.io adopte une approche architecturale distinctive parmi les plateformes PaaS : plutôt que d'exécuter les applications dans des conteneurs Docker traditionnels, elle s'appuie sur Firecracker, la technologie de microVM développée par AWS. Fly.io déploie les applications conteneurisées sous forme de Firecracker microVMs dans plus de 18 régions mondiales, exécutant le calcul au plus près des utilisateurs pour réduire la latence. Chaque application bénéficie d'un réseau maillé WireGuard privé par défaut, permettant une communication sécurisée inter-services entre régions sans configuration.

Firecracker est une technologie de microVMs légères qui combine les propriétés de sécurité et d'isolation des VMs traditionnelles avec la rapidité et l'efficacité en ressources des conteneurs. Elle permet d'exécuter en toute sécurité des charges de travail de clients différents sur le même matériel physique. Ce modèle confère à Fly.io une isolation de sécurité supérieure aux plateformes basées sur des conteneurs partagés, mais impose que l'application soit packagée sous forme d'image Docker. La plateforme est résolument orientée CLI via l'outil `flyctl`, avec une interface web limitée, ce qui représente une courbe d'apprentissage non négligeable pour les développeurs peu familiers avec les outils d'infrastructure. Sa tarification basée sur la consommation de temps de calcul et de transfert de données est difficile à prévoir. Aucune région de déploiement n'est disponible en Afrique.

Koyeb

Koyeb est la plateforme la plus récente parmi les acteurs analysés, positionnée comme une plateforme serverless à déploiement global. Koyeb est une plateforme serverless conçue pour permettre aux développeurs de déployer des applications fiables et évolutives à l'échelle mondiale. Elle fournit un moyen unifié d'exécuter des inférences, du code dans des bacs à sable isolés, et une variété de services incluant des applications web complètes, des APIs et des workers. La plateforme abstrait le calcul afin que les charges de travail s'exécutent sur tout matériel, dans n'importe quelle région, sur différents clouds.

Koyeb offre un réseau edge mondial avec plus de 250 emplacements edge fournissant un chiffrement TLS natif, un cache CDN, et un load balancing automatique pour toutes les applications déployées. Sa tarification est basée sur la consommation à la seconde, avec un tier starter et des plans Pro à partir de 29 \$/mois. Koyeb se distingue par son support natif des GPUs pour les charges d'inférence IA, le faisant sortir du périmètre des PaaS généralistes. Pour le marché algérien, Koyeb présente les mêmes limitations structurelles que ses concurrents : tarification USD, régions limitées à l'Europe, aux États-Unis et à l'Asie, et absence de tout mécanisme de conformité réglementaire locale.

Limitations communes pour le contexte algérien

L'analyse de ces six plateformes révèle un ensemble de limitations structurelles communes qui les rendent inadaptées, ou difficilement accessibles, pour les développeurs et entreprises algériens.

Inaccessibilité des moyens de paiement. L'ensemble de ces plateformes facture exclusivement en USD via carte bancaire internationale (Visa/Mastercard) ou PayPal. Or, les cartes bancaires algériennes sont majoritairement limitées aux transactions en dinars algériens (DZD), et les cartes de crédit en devises étrangères restent soumises à des réglementations strictes et à des plafonds annuels de change qui excluent de facto l'accès à ces services pour une large fraction des développeurs algériens.

Absence de résidence des données. Aucune des plateformes analysées ne propose de région de déploiement en Algérie ou dans les pays voisins d'Afrique du Nord. Les données applicatives et personnelles transitent et sont stockées exclusivement dans des datacenters situés en dehors du territoire algérien, en contradiction directe avec les obligations de localisation des données imposées par les lois 18-07 et 25-11, analysées dans la section 1.5.

Latence réseau. L'absence de région de déploiement proche du Maghreb implique des latences réseau structurellement élevées pour les utilisateurs finaux algériens, dégradant l'expérience utilisateur des applications déployées sur ces plateformes.

Ces limitations justifient l'existence d'une offre PaaS locale adaptée au contexte réglementaire et économique algérien, dont l'analyse des initiatives existantes fait l'objet de la section suivante.

TABLEAU 1.2 – Comparaison des plateformes PaaS internationales

Plate-forme	Modèle de déploiement	Frameworks / Langages	Tarification	Free tier	Limitations contexte algérien
Vercel	Serverless + CDN edge ; déploiement depuis GitHub /GitLab /Bitbucket	Next.js (natif), React, Vue, Svelte, Nuxt ; fonctions serverless Node.js / Edge Runtime	Gratuit limité ; Pro 20 \$/mois ; Enterprise sur devis — facturation à la requête au-delà des quotas	Oui — limites sur build minutes, bande passante et requêtes serverless	Paieement USD uniquement ; régions US/EU/AP ; aucune conformité loi 18-07 ; latence élevée depuis l'Algérie [3]
Railway	Conteneurs Docker ou détection automatique via Nixpacks /Railpack ; déploiement depuis GitHub	Node.js, Python, Go, Ruby, PHP, Java, Rust ; bases de données PostgreSQL, MySQL, Redis, MongoDB	Usage-based : 5 \$ de crédits/mois offerts après ajout d'un moyen de paiement ; facturation à la seconde	Oui — 5 \$ de crédits mensuels (carte requise)	Paieement USD, carte bancaire internationale obligatoire ; régions US/EU ; aucune résidence des données en Algérie [7]

Suite à la page suivante

Plate-forme	Modèle de déploiement	Frameworks / Langages	Tarification	Free tier	Limitations contexte algérien
Render	Conteneurs managés ; déploiement depuis GitHub / GitLab ; IaC via <code>render.yaml</code>	Node.js, Python, Ruby, Go, Rust, PHP, Elixir, Docker ; bases de données PostgreSQL managées	Plans mensuels fixes par instance ; Web Service à partir de 7 \$/mois ; base de données à partir de 7 \$/mois	Oui — services statiques gratuits ; services web avec mise en veille après inactivité	Paieement USD ; régions US/EU uniquement ; données hors territoire algérien [7]
Heroku	Dynos (conteneurs légers) ; déploiement via <code>git push heroku</code> ou GitHub ; Procfile	Node.js, Python, Ruby, Java, PHP, Go, Scala, Clojure ; écosystème d'add-ons riche	Eco dynos à partir de 5 \$/mois ; Basic à 7 \$/mois ; Standard à partir de 25 \$/mois	Non — tier gratuit supprimé en novembre 2022	Paieement USD ; infrastructure AWS (US/EU/AP) ; tarification élevée pour fonctionnalités équivalentes aux concurrents modernes [7]
Fly.io	microVMs Firecracker ; déploiement via <code>flyctl</code> CLI ; image Docker requise	Tout langage packagé en image Docker ; réseau maillé WireGuard multi-région natif	Usage-based ; 3 VMs partagées incluses ; facturation à la seconde de calcul et au Go de transfert	Oui — 3 microVMs shared CPU (carte requise pour déblocage complet)	Paieement USD ; CLI-first (courbe d'apprentissage élevée) ; aucune région Afrique ; carte bancaire internationale requise [8]

Suite à la page suivante

Plate-forme	Modèle de déploiement	Frameworks / Langages	Tarification	Free tier	Limitations contexte algérien
Koyeb	Serverless edge global ; déploiement depuis GitHub ou image Docker ; 25+ régions edge	Node.js, Python, Go, Ruby, PHP, Rust, Java ; support natif GPU pour inférence IA	Starter gratuit (2 nano services) ; plans Pro à partir de 79 \$/mois — facturation à la seconde	Oui — 2 nano services sans carte bancaire requise	Paieement USD ; aucune région Afrique du Nord ; plans Pro très onéreux pour le marché algérien [9]

1.4 Analyse des initiatives PaaS algériennes

1.4.1 Étude critique des acteurs locaux émergents

L'écosystème numérique algérien connaît depuis 2020 une dynamique entrepreneuriale croissante dans le domaine du cloud et de l'hébergement. Plusieurs initiatives locales ont émergé avec l'ambition de proposer des alternatives aux plateformes internationales, répondant à la fois à une demande de souveraineté numérique et à l'inaccessibilité des moyens de paiement en devises étrangères pour une large fraction des développeurs algériens. Cette section propose une analyse critique de cinq acteurs locaux identifiés, en évaluant la réalité de l'offre proposée, ses limitations techniques constatées, et son positionnement par rapport à Hostifer.

Hawiyat

Hawiyat est l'un des acteurs les plus visibles de l'hébergement web algérien. Sa page produit propose une gamme d'offres VPS tarifées en dinars algériens, avec des configurations allant de 2 400 DZD/mois pour 1 vCore / 2 Go RAM / 20 Go SSD à des offres supérieures incluant une option cPanel. La présence d'une tarification en DZD et d'une infrastructure partiellement localisée en Algérie constitue un avantage certain pour l'accessibilité locale.

Cependant, l'offre réelle de Hawiyat s'apparente davantage à du VPS managé et à des services d'automatisation no-code — notamment n8n pour les flux de travail automatisés — qu'à une véritable plateforme PaaS orientée développeur. Il n'existe aucune abstraction du type « déployer depuis un dépôt Git », aucun pipeline de build automatisé, aucune gestion des certificats TLS, et aucune orchestration de conteneurs. Le modèle est celui d'un hébergeur traditionnel proposant des ressources prêtes à l'emploi que l'utilisateur configure et administre lui-même. L'écart avec la proposition de valeur d'une plateforme PaaS au sens défini dans la section 1.1 est donc structurel et non pas seulement fonctionnel.

Deployi

Deployi se positionne comme un prestataire de services d'hébergement et d'infrastructure à l'ancienne, proposant principalement des offres VPS, des services de messagerie professionnelle, et des prestations d'hébergement web traditionnel. Son catalogue reprend les catégories classiques de l'hébergement mutualisé : domaines, certificats SSL, serveurs de mail, et machines virtuelles administrées par l'utilisateur.

Cette approche, bien que légitime pour des besoins d'hébergement web standard, ne constitue pas une offre PaaS. Aucun mécanisme d'automatisation du déploiement depuis un dépôt de code, de détection de framework, de gestion du cycle de vie des conteneurs, ou de provisionnement à la demande n'est proposé. L'utilisateur reste entièrement responsable de l'installation, de la configuration et de la maintenance de sa pile logicielle sur les ressources louées. Deployi s'adresse à une clientèle ayant des besoins en hébergement classique plutôt qu'à des développeurs souhaitant une expérience de déploiement automatisée.

PivoCloud

PivoCloud propose une offre qui se rapproche davantage d'une plateforme de déploiement, en permettant aux utilisateurs de déployer des applications conteneurisées. Cependant, l'analyse de l'infrastructure sous-jacente révèle que le modèle de déploiement repose sur Docker Compose plutôt que sur Kubernetes. Cette distinction est fondamentale d'un point de vue technique : Docker Compose est un outil de définition et d'exécution d'applications multi-conteneurs sur un hôte unique, sans orchestration distribuée, sans gestion automatique des défaillances de nœuds, et sans capacité d'autoscaling horizontal native [4].

L'absence de Kubernetes implique l'absence de l'ensemble des primitives d'isolation et de gestion des ressources qui fondent la proposition de valeur d'un PaaS multi-tenant robuste : pas de *Namespace* d'isolation par tenant, pas de *NetworkPolicy* pour le contrôle

du trafic inter-service, pas de *ResourceQuota* pour la garantie des ressources par plan tarifaire, et pas de *HorizontalPodAutoscaler* pour l'élasticité automatique. La question de la scalabilité de l'infrastructure sous-jacente reste également ouverte : sans orchestrateur de cluster, la capacité à ajouter dynamiquement des nœuds de calcul en réponse à une augmentation de la charge n'est pas garantie par l'architecture déclarée.

Sagittario

Sagittario est une initiative locale qui s'est présentée comme une plateforme PaaS algérienne en cours de développement. Au moment de la rédaction de ce mémoire, la plateforme n'est pas fonctionnelle et aucune offre de service opérationnelle n'est accessible publiquement. En l'absence d'une infrastructure déployée et utilisable, il n'est pas possible de procéder à une évaluation technique de l'offre réelle. Sagittario ne peut donc être considérée ni comme un concurrent direct de Hostifer ni comme une solution alternative pour les développeurs algériens dans son état actuel.

OpenScaler

OpenScaler est, parmi les initiatives algériennes analysées, le seul acteur présentant une ambition et une architecture technique comparables à celles d'Hostifer. La plateforme se définit elle-même comme le premier *HyperScaler* Cloud d'Afrique et de la région MENA, construite depuis zéro pour les applications cloud-natives et les charges de travail d'intelligence artificielle [10]. Son catalogue de services, actuellement en phase Alpha, couvre un spectre significativement plus large que les autres acteurs locaux.

OpenScaler propose des machines virtuelles Linux configurables à la demande, des clusters Kubernetes managés provisionnés en quelques secondes sans maintenance d'infrastructure, du stockage bloc flexible attachable entre machines, un load balancer avec surveillance automatique de la santé des instances, et un système d'Elastic Server permettant de créer des groupes de machines virtuelles en autoscaling basés sur des métriques telles que l'utilisation CPU ou le trafic réseau. Cette offre IaaS/PaaS constitue une base technique sérieuse et témoigne d'une compréhension réelle des besoins des développeurs cloud-natifs.

La principale limitation d'OpenScaler au moment de cette analyse est son stade de maturité : la plateforme est en phase Alpha publique, ce qui implique une instabilité potentielle, une couverture fonctionnelle incomplète et l'absence de SLA de production. La plateforme communique transparentement sur cette réalité en affichant explicitement sa roadmap par jalons (POC → MVP → Alpha → Beta → Release), ce qui est un signe de

maturité organisationnelle appréciable mais confirme que la plateforme n'est pas encore prête pour des déploiements de production critiques.

Sur le plan du positionnement, OpenScaler et Hostifer ciblent des segments complémentaires plutôt que strictement concurrents. OpenScaler est positionné sur la couche **IaaS/CaaS** (*Container as a Service*), en fournissant des briques d'infrastructure — VMs, clusters Kubernetes managés, stockage — que le développeur assemble et configure lui-même. Hostifer, en revanche, opère exclusivement sur la couche **PaaS** : le développeur fournit une URL de dépôt GitHub et la plateforme gère intégralement le pipeline de build, le provisionnement du tenant, la configuration DNS et la gestion des secrets, sans exposer aucune primitive d'infrastructure. Ces deux approches sont complémentaires et coexistent sur le marché international : AWS (IaaS) et Vercel (PaaS) occupent des positions analogues sans se concurrencer directement.

TABLEAU 1.3 – Comparaison des plateformes PaaS algériennes

Plateforme	Offre réelle	Infrastructure	Maturité	Positionnement vs Hostifer
Hawiyat	VPS + n8n (no-code)	Hébergeurs partenaires (ADEX, ISSAL, Ayrade)	Production	Hébergeur traditionnel, pas de PaaS
Deployi	VPS, mailing, hébergement web	Infrastructure propre non documentée	Production	Hébergeur classique, pas de déploiement automatisé
PivoCloud	Déploiement conteneurs	Docker Compose, sans Kubernetes	Production (limitée)	Pas d'orchestration, pas d'autoscaling, pas d'isolation multi-tenant
Sagittario	Non disponible	Inconnue	Non fonctionnel	Pas évaluable techniquement

Suite à la page suivante

Plateforme	Offre réelle	Infrastructure	Maturité	Positionnement vs Hostifer
OpenScaler	VMs, K8s managé, sto- ckage, LB, autoscaling	Infrastructure propriétaire, Al- gérie + MENA	Alpha pu- blique	Concurrent IaaS/CaaS sérieux ; com- plémentaire à Hostifer sur la couche PaaS

1.5 Le contexte réglementaire algérien

1.6 Le contexte réglementaire algérien

La conformité réglementaire constitue l'un des piliers fondateurs de la conception de Hostifer. Au-delà de la simple opportunité commerciale, c'est la contrainte légale croissante pesant sur les entreprises et développeurs algériens utilisant des infrastructures cloud étrangères qui justifie structurellement l'existence d'une plateforme PaaS hébergée localement. Cette section présente le cadre réglementaire algérien applicable à l'hébergement et au traitement des données, et en analyse les implications concrètes pour les acteurs numériques locaux.

La loi 18-07 du 10 juin 2018 : cadre fondateur

La loi n° 18-07 du 10 juin 2018, relative à la protection des personnes physiques dans le traitement des données à caractère personnelle, publiée au Journal officiel n° 34, est l'équivalent algérien du Règlement Général sur la Protection des Données (RGPD) européen. Elle encadre la collecte, le stockage, l'utilisation et la transmission des données personnelles (nom, prénom, adresse, email, téléphone, données bancaires, données de santé, etc.).

La loi 18-07 fixe les règles de protection des données personnelles qui se traduisent par l'ancrage de nouveaux principes, la consécration de nouveaux droits et la mise en place de nombreuses obligations. Dans cette dynamique, l'année 2020 fut l'année de la constitutionnalisation du principe de protection des données à caractère personnel à travers la nouvelle disposition introduite par la Constitution algérienne.

Les obligations qu'elle impose aux organisations traitant des données personnelles de ressortissants algériens peuvent être regroupées en quatre catégories. Premièrement, l'obligation de **consentement éclairé** : toute collecte de données personnelles doit être

précédée du consentement explicite de la personne concernée, qui doit être informée de la finalité du traitement. Deuxièmement, l'obligation de **sécurité** : l'organisation doit mettre en place des mesures techniques et organisationnelles appropriées pour protéger les données contre tout accès non autorisé, toute divulgation ou toute altération. Troisièmement, l'obligation de **notification** : toute violation de sécurité affectant des données personnelles doit être notifiée aux autorités compétentes et aux personnes concernées. Quatrièmement, et c'est la disposition ayant l'impact le plus direct sur l'hébergement cloud : les transferts de données vers l'étranger requièrent une autorisation préalable de l'autorité de contrôle. Cette obligation concerne notamment l'hébergement de données chez des prestataires étrangers, et les services cloud internationaux entrent dans cette catégorie.

L'autorité chargée de veiller à l'application de la loi 18-07 est l'**Autorité Nationale de Protection des Données Personnelles** (ANPDP), dont les pouvoirs incluent le contrôle des traitements de données, l'instruction des plaintes, et l'imposition de sanctions en cas de non-conformité [11].

La loi 25-11 du 24 juillet 2025 : renforcement et souveraineté numérique

La loi 25-11, désormais en vigueur depuis le 24 juillet 2025, modifie et complète la loi 18-07 de 2018 relative à la protection des personnes physiques dans le traitement des données à caractère personnel. Cette évolution législative majeure redéfinit les obligations des organisations algériennes et modernise le cadre de protection des données personnelles dans le pays.

La loi 25-11 apporte une nouvelle facette stratégique à la législation nationale : la souveraineté des données. Ce concept implique que les données personnelles des citoyens algériens doivent être stockées, traitées et protégées localement, sous le contrôle direct des autorités nationales, afin de garantir une meilleure maîtrise des risques numériques et géopolitiques.

Les nouvelles obligations introduites par la loi 25-11 par rapport au cadre de la loi 18-07 sont substantielles. La loi 25-11 rend obligatoire la désignation d'un Délégué à la Protection des Données (DPD) pour chaque organisation traitant des données personnelles, impose la tenue d'un registre automatisé des opérations de traitement traçant toutes les opérations effectuées sur les données (collecte, consultation, suppression), et réduit à cinq jours le délai maximal de notification des violations de données, avec des amendes administratives renforcées. Ces exigences s'appliquent également aux organisations non établies en Algérie dès lors qu'elles ciblent des résidents algériens ou traitent

leurs données — ce qui inclut de facto tout service cloud étranger utilisé par une entreprise algérienne pour héberger des données de clients locaux.

Implications pour les startups et freelances algériens

L'analyse combinée des lois 18-07 et 25-11 révèle un paradoxe structurel qui affecte directement les développeurs et startups algériens. D'un côté, les plateformes PaaS internationales les plus performantes et accessibles (Vercel, Railway, Render, Fly.io) sont hors de portée pour des raisons de paiement en devises. De l'autre, même pour ceux disposant de moyens de paiement internationaux, l'utilisation de ces plateformes expose leurs clients et leur organisation à des risques légaux croissants au regard des lois 18-07 et 25-11.

En pratique, une startup algérienne déployant son application sur Vercel (infrastructure AWS, régions US/EU) et collectant des données de clients algériens — adresses email, numéros de téléphone, informations de paiement — sans autorisation préalable de l'ANPDP pour le transfert international de données se trouve en situation de non-conformité au regard de la loi 18-07. Avec l'entrée en vigueur de la loi 25-11, cette non-conformité est susceptible d'exposer l'organisation à des sanctions administratives renforcées, en plus du risque réputationnel associé. La nécessité de stocker et d'héberger les données en Algérie n'est donc pas une option mais une obligation légale pour les entreprises traitant des données personnelles de ressortissants algériens.

Cette réalité crée une demande non satisfaite pour une infrastructure PaaS locale offrant les mêmes garanties de qualité technique et d'automatisation que les leaders internationaux, tout en garantissant la résidence des données sur le territoire algérien. C'est précisément dans ce vide que Hostifer se positionne, en proposant une plateforme de déploiement automatisée, hébergée en Algérie, dont l'architecture garantit que l'ensemble des données applicatives — code source cloné, images OCI construites, variables d'environnement, journaux — ne quittent jamais l'infrastructure nationale.

TABLEAU 1.4 – Synthèse des obligations légales imposées par les lois 18-07 et 25-11 et leur impact sur l'hébergement cloud

Obligation	Disposition légale	Impact sur l'hébergement cloud
Consentement éclairé	Loi 18-07, Art. principes généraux	L'hébergeur doit être en mesure de garantir que les données collectées avec consentement sont traitées exclusivement selon la finalité déclarée
Interdiction de transfert international sans autorisation	Loi 18-07, dispositions sur les transferts	Tout hébergeur étranger (Vercel, Railway, AWS, etc.) nécessite une autorisation préalable de l'ANPDP; en l'absence d'autorisation, l'hébergement est illégal
Sécurité des données	Loi 18-07, obligations de sécurité	L'infrastructure d'hébergement doit garantir chiffrement en transit et au repos, contrôle d'accès, et traçabilité des accès
Notification des violations	Loi 18-07 / Loi 25-11 (délai réduit à 5 jours)	L'hébergeur doit permettre la détection et la notification rapide des incidents de sécurité dans le délai légal
Désignation d'un DPD	Loi 25-11 (nouvelle obligation)	Les organisations utilisant des services cloud étrangers doivent désigner un DPD capable de superviser la conformité des flux de données transfrontaliers
Registre des traitements	Loi 25-11 (nouvelle obligation)	L'hébergeur doit fournir des journaux d'audit exploitables permettant de tracer toutes les opérations effectuées sur les données

Suite à la page suivante

Obligation	Disposition légale	Impact sur l'hébergement cloud
Souveraineté des données	Loi 25-11, principe directeur	Les données des ressortissants algériens doivent être stockées et traitées localement ; les services cloud sans infrastructure en Algérie ne peuvent garantir cette conformité
Résidence des données	Loi 18-07 / Loi 25-11 combinées	Obligation de recourir à un hébergeur disposant d'une infrastructure physique en Algérie pour les données soumises à la loi

1.7 Analyse du marché et identification du besoin

Quantification du marché cible

L'analyse du marché adressable par Hostifer s'appuie sur des indicateurs chiffrés de l'écosystème numérique algérien, dont la trajectoire de croissance est documentée par plusieurs sources indépendantes.

La base d'utilisateurs numériques. En début d'année 2025, l'Algérie comptait 36,2 millions d'utilisateurs d'internet, représentant 76,9% de la population, contre 33,49 millions en 2024. Cette base constitue le substrat sur lequel se développe l'ensemble de l'activité numérique productive, incluant les développeurs freelances, les agences digitales et les startups. L'Algérie comptait 4,8 millions d'abonnés LinkedIn en 2025, une métrique pertinente pour estimer la population de professionnels du numérique actifs, dont une fraction significative est constituée de développeurs et d'ingénieurs logiciels.

L'écosystème startup. Les startups algériennes ont levé 650 millions de dollars en 2024, marquant une croissance de 60% par rapport à l'année précédente. Le portail `startup.dz` recense plus de 7 800 entités enregistrées, dont environ 2 300 labellisées [12]. 800 nouvelles startups ont été créées en 2023 uniquement, reflétant l'influence croissante du pays dans l'écosystème startup. Ces organisations constituent le cœur de la cible Hostifer : des équipes de développement de petite taille ayant besoin de déployer et maintenir des applications web sans disposer de compétences ou de budgets en administration système.

La dynamique réglementaire comme accélérateur de marché. Comme analysé

dans la section 1.5, l'entrée en vigueur de la loi 25-11 en juillet 2025 crée une pression réglementaire croissante sur les organisations algériennes utilisant des hébergeurs étrangers. La loi sur l'activité audiovisuelle entrée en vigueur en décembre 2024 impose désormais aux services audiovisuels en ligne d'être hébergés exclusivement sur des serveurs physiquement situés en Algérie et d'utiliser le domaine national `.dz`. Cette obligation sectorielle préfigure une extension progressive des exigences de résidence des données à d'autres catégories d'acteurs numériques, élargissant mécaniquement le marché adressable d'une plateforme cloud locale conforme.

Le marché TAM/SAM/SOM. Le marché total adressable (TAM) de Hostifer peut être estimé à l'ensemble des acteurs numériques algériens ayant besoin d'héberger des applications web : développeurs freelances, agences digitales, startups, et PME digitalisées. Le marché adressable serviceable (SAM) se restreint aux acteurs ayant adopté ou souhaitant adopter un workflow de déploiement basé sur Git et des conteneurs — segment en forte croissance corrélé à la montée en compétences de la communauté technique algérienne. Le marché obtenu serviceable (SOM) à court terme se concentre sur les startups early-stage et les freelances développant des applications web basées sur les frameworks supportés par Hostifer (Node.js, Python, PHP). La Figure 1.4 illustre cette segmentation du marché.

Identification des pain points non adressés

L'analyse croisée des solutions existantes — plateformes internationales (section 1.3), initiatives algériennes (section 1.4) et contraintes réglementaires (section 1.5) — permet d'identifier six pain points structurels non adressés par l'offre disponible à la date de conception de Hostifer.

Pain point 1 : Inaccessibilité financière des plateformes internationales. Les plateformes PaaS de référence (Vercel, Railway, Render) facturent exclusivement en USD via carte bancaire internationale. La grande majorité des développeurs et startups algériens ne disposent pas de cartes en devises étrangères, ou font face à des plafonds annuels de change insuffisants pour souscrire à des abonnements mensuels récurrents. Il n'existe, à la date de ce travail, aucune plateforme PaaS de qualité comparable proposant une facturation en dinars algériens.

Pain point 2 : Non-conformité réglementaire systémique. Tout développeur algérien utilisant un hébergeur étranger pour une application traitant des données personnelles de ressortissants algériens est potentiellement en situation de non-conformité au regard de la loi 18-07, sans mécanisme simple pour s'en extraire. Les hébergeurs locaux existants ne proposent pas de plateforme PaaS permettant de migrer depuis un hébergeur

étranger sans effort technique significatif.

Pain point 3 : Absence de pipeline de déploiement automatisé local. Les développeurs algériens souhaitant déployer une application web depuis un dépôt GitHub n'ont pas d'équivalent local à Vercel ou Railway. Les hébergeurs existants (Hawiyat, Deployi) requièrent une configuration manuelle du serveur, l'installation des dépendances, et la gestion des processus — des tâches chronophages qui représentent une friction majeure pour les équipes de petite taille et les développeurs solo.

Pain point 4 : Absence d'isolation multi-tenant robuste. L'initiative la plus proche d'un PaaS local (PivoCloud) repose sur Docker Compose, sans isolation réseau entre tenants, sans quotas de ressources, et sans mécanisme d'autoscaling. Dans un environnement mutualisé, cette absence d'isolation représente un risque de sécurité et de disponibilité pour les utilisateurs, et rend la conformité à la loi 18-07 difficile à garantir contractuellement.

Pain point 5 : Latence réseau des hébergeurs étrangers. Pour les applications ciblant une base d'utilisateurs algérienne, héberger sur des serveurs situés en Europe ou aux États-Unis implique des latences réseau de l'ordre de 80 à 150 ms aller-retour — une dégradation perceptible de l'expérience utilisateur pour les applications web interactives. Aucune plateforme internationale n'offre de région de déploiement dans la région Afrique du Nord.

Pain point 6 : Absence d'observabilité intégrée et accessible. Les développeurs déployant sur des VPS algériens n'ont pas accès à des outils d'observabilité intégrés (journaux centralisés, métriques d'utilisation, alertes) sans les configurer et les maintenir eux-mêmes. Ce coût opérationnel est prohibitif pour les équipes de petite taille, qui renoncent souvent à l'observabilité au détriment de la fiabilité de leurs applications.

Conclusion : justification de Hostifer

L'ensemble de ces analyses convergent vers un constat univoque : il existe en Algérie une demande réelle et croissante pour une plateforme PaaS de qualité professionnelle, accessible financièrement, hébergée localement et conforme au cadre réglementaire algérien. Cette demande n'est adressée par aucune solution existante — ni internationale pour des raisons d'accessibilité et de conformité, ni locale pour des raisons de maturité technique.

Hostifer est conçu pour combler précisément cet espace. En combinant une architecture Kubernetes multi-tenant native, un pipeline de déploiement automatisé depuis GitHub, une tarification en DZD, et une infrastructure hébergée en Algérie, la plateforme répond simultanément aux six pain points identifiés. Le tableau 1.5 synthétise cette correspondance entre les besoins non satisfaits et les réponses apportées par Hostifer.

FIGURE 1.4 – Diagramme TAM/SAM/SOM du marché algérien du cloud pour développeurs

Marché algérien du cloud pour développeurs — TAM / SAM / SOM (2025)

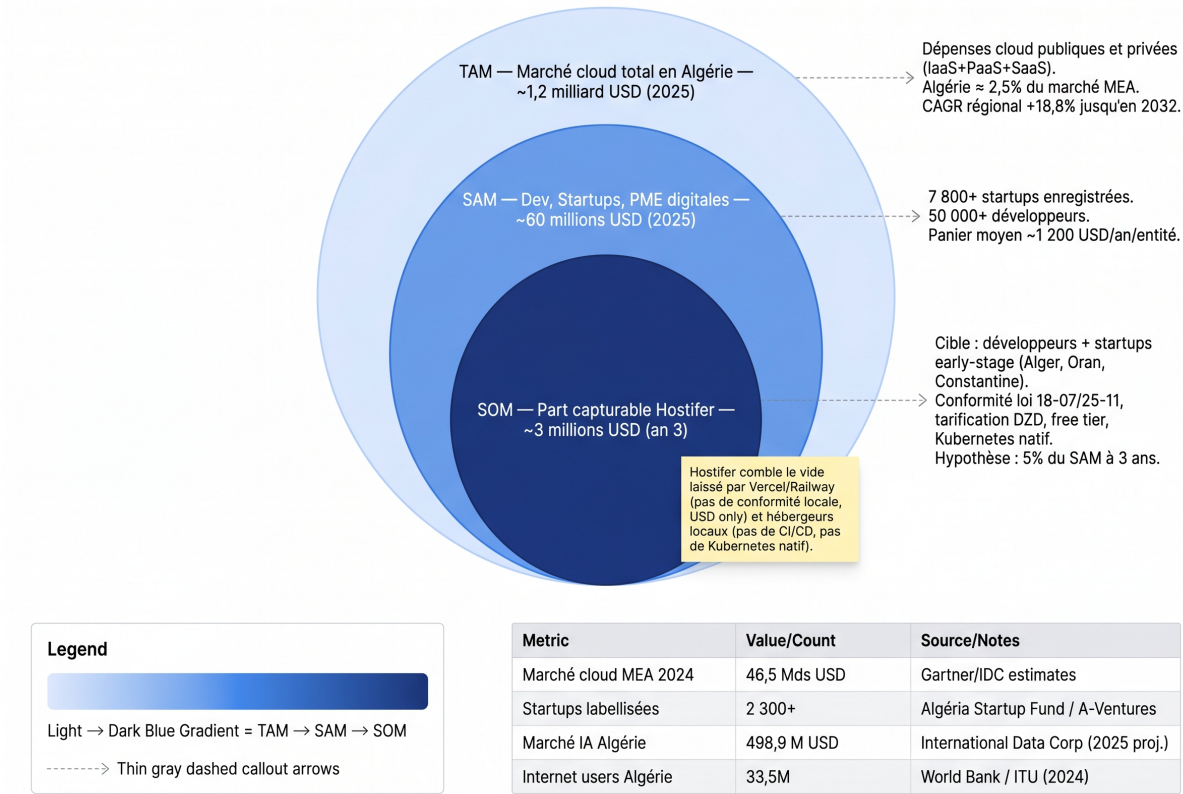


TABLEAU 1.5 – Synthèse des besoins non satisfaits et réponse apportée par Hostifer

Besoin non satisfait	Solution existante	Réponse Hostifer
Tarification accessible en DZD	Aucune (plateformes PaaS en USD uniquement)	Facturation native en dinars algériens, sans carte en devises étrangères
Conformité lois 18-07 / 25-11	Aucune (hébergeurs étrangers hors juridiction algérienne)	Infrastructure hébergée en Algérie ; données ne quittant jamais le territoire national

Suite à la page suivante

Besoin non satisfait	Solution existante	Réponse Hostifer
Déploiement automatisé depuis GitHub	Aucune solution locale (hébergeurs VPS uniquement)	Pipeline complet Build → Provision → DNS déclenché par un push GitHub
Isolation multi-tenant robuste	PivoCloud (Docker Compose sans isolation K8s)	Namespace Kubernetes dédié par tenant, NetworkPolicy, ResourceQuota par plan
Faible latence pour utilisateurs algériens	Aucune région locale chez les PaaS internationaux	Infrastructure hébergée en Algérie sur cluster ACK ; latence réduite pour les utilisateurs locaux
Observabilité intégrée (logs, métriques)	Configuration manuelle sur VPS	Journaux centralisés, métriques et alertes intégrés

1.8 Synthèse et positionnement de Hostifer

L'analyse conduite tout au long de ce chapitre a permis de dresser un tableau complet de l'environnement dans lequel s'inscrit Hostifer : un marché international dominé par des plateformes techniquement excellentes mais structurellement inaccessibles au contexte algérien, un marché local en émergence mais encore trop immature pour répondre aux besoins réels des développeurs, et un cadre réglementaire en renforcement rapide qui transforme la conformité en obligation non négociable. Cette section synthétise les lacunes identifiées et positionne Hostifer comme la réponse systémique à cet ensemble de contraintes.

Lacunes identifiées

L'analyse comparative des plateformes PaaS internationales (section 1.3) et algériennes (section 1.4), combinée à l'étude du contexte réglementaire (section 1.5) et à la quantification du marché (section 1.6), permet d'identifier quatre lacunes structurelles qui définissent l'espace de marché non adressé.

Lacune 1 : Absence d'une plateforme PaaS accessible et conforme. Aucune

solution disponible ne combine simultanément les trois attributs fondamentaux requis par un développeur algérien : une qualité d'automatisation comparable aux leaders internationaux (déploiement depuis GitHub, détection automatique de framework, gestion des certificats TLS), une tarification en dinars algériens accessible sans carte en devises étrangères, et une infrastructure hébergée en Algérie garantissant la conformité aux lois 18-07 et 25-11. Ces trois attributs sont chacun présents partiellement dans l'offre existante — les plateformes internationales maîtrisent le premier, aucune ne satisfait les deux autres — mais leur combinaison est absente du marché.

Lacune 2 : Déficit d'isolation technique dans les solutions locales. Les initiatives locales qui proposent une forme de déploiement conteneurisé (PivoCloud) reposent sur des technologies inadaptées au multi-tenant de production : Docker Compose n'offre ni isolation réseau entre tenants, ni garanties de ressources par client, ni mécanismes d'autoscaling. L'absence de Kubernetes comme substrat d'orchestration interdit toute implémentation sérieuse des principes de séparation des domaines de sécurité, de gestion des quotas et de résilience aux défaillances de composants individuels [6].

Lacune 3 : Vide réglementaire croissant créant une demande forcée. La succession législative 18-07 (2018) → constitutionnalisation (2020) → 25-11 (2025) → obligations sectorielles (presse, audiovisuel, 2024) dessine une trajectoire réglementaire univoque vers l'obligation de résidence des données pour les acteurs numériques algériens. Cette trajectoire crée mécaniquement une demande croissante pour des alternatives locales conformes, qui ne peut être absorbée par les hébergeurs traditionnels (VPS, mutualisé) sans qu'une couche d'automatisation PaaS ne soit interposée [13].

Lacune 4 : Absence d'observabilité et de sécurité des secrets intégrées. L'ensemble des solutions locales analysées ne propose ni journalisation centralisée avec streaming en temps réel, ni gestion des secrets applicatifs dans un coffre-fort chiffré. Ces fonctionnalités, considérées comme basiques par les développeurs familiers avec les plateformes internationales, sont absentes de l'offre locale, contraignant les équipes à les implémenter manuellement à chaque nouveau projet ou à y renoncer au détriment de la sécurité et de l'opérabilité de leurs applications.

Positionnement unique de Hostifer

Face à ces lacunes, Hostifer se positionne comme la première plateforme PaaS algérienne à répondre simultanément aux exigences techniques, réglementaires et économiques du marché local. Ce positionnement repose sur quatre piliers distinctifs dont la combinaison est inédite dans l'écosystème numérique algérien.

Infrastructure Kubernetes native. Contrairement aux approches basées sur Do-

cker Compose ou sur la virtualisation traditionnelle, Hostifer repose sur Kubernetes comme substrat d'orchestration fondamental. Chaque application déployée est provisionnée dans un Namespace Kubernetes dédié, isolé par des NetworkPolicies granulaires, borné par des ResourceQuotas alignés sur le plan tarifaire, et exposé via Traefik avec terminaison TLS automatique. Cette architecture garantit une isolation multi-tenant de niveau production, une élasticité automatique via HPA et Cluster Autoscaler, et une gestion déclarative de l'infrastructure qui rend chaque provisionnement reproductible et auditable [6]. Aucune initiative locale n'atteint ce niveau de sophistication technique.

Hébergement local et conformité réglementaire by design. L'infrastructure de Hostifer est déployée sur le service ACK d'Alibaba Cloud en Algérie. L'ensemble du cycle de vie des données applicatives — clonage du dépôt source, construction de l'image OCI, stockage dans le registre interne, exécution des pods applicatifs, journalisation dans Loki, persistance des secrets dans Vault — se déroule exclusivement dans des datacenters situés sur le territoire algérien. Cette architecture garantit par construction la conformité aux obligations de résidence des données imposées par les lois 18-07 et 25-11, sans qu'aucune configuration supplémentaire ne soit nécessaire de la part de l'utilisateur [14] [15].

Tarification en dinars algériens et accessibilité économique. Hostifer propose une tarification nativement exprimée en DZD, avec un tier gratuit permettant à tout développeur de déployer sa première application sans abonnement. Cette approche lève la barrière d'accès principale identifiée dans la section 1.6, en rendant la plateforme accessible à l'ensemble de la communauté des développeurs algériens, y compris ceux ne disposant pas de moyens de paiement en devises étrangères. Le modèle économique par paliers (Free → Standard → Enterprise) permet une progression naturelle depuis le déploiement de projets personnels jusqu'aux applications de production à forte charge.

Expérience développeur comparable aux standards internationaux. La proposition de valeur de Hostifer sur l'axe de l'expérience utilisateur est d'apporter la fluidité des plateformes PaaS de référence — déploiement en quelques clics depuis une URL GitHub, streaming des journaux de build en temps réel, variables d'environnement chiffrées, CI/CD automatique sur push — dans un contexte local. Le pipeline de build basé sur Railpack et Buildkit assure la détection automatique du framework et la construction de l'image OCI sans Dockerfile, réduisant la friction d'onboarding au minimum absolu pour les développeurs non familiers avec les outils de conteneurisation [16].

La Figure 1.5 illustre ce positionnement sur une matrice à deux axes — qualité technique de l'infrastructure et conformité au contexte algérien — sur laquelle Hostifer occupe une position unique parmi l'ensemble des solutions analysées dans ce chapitre. C'est sur la base de ce positionnement et de ces lacunes identifiées que les chapitres suivants décrivent la conception et l'implémentation de la plateforme.

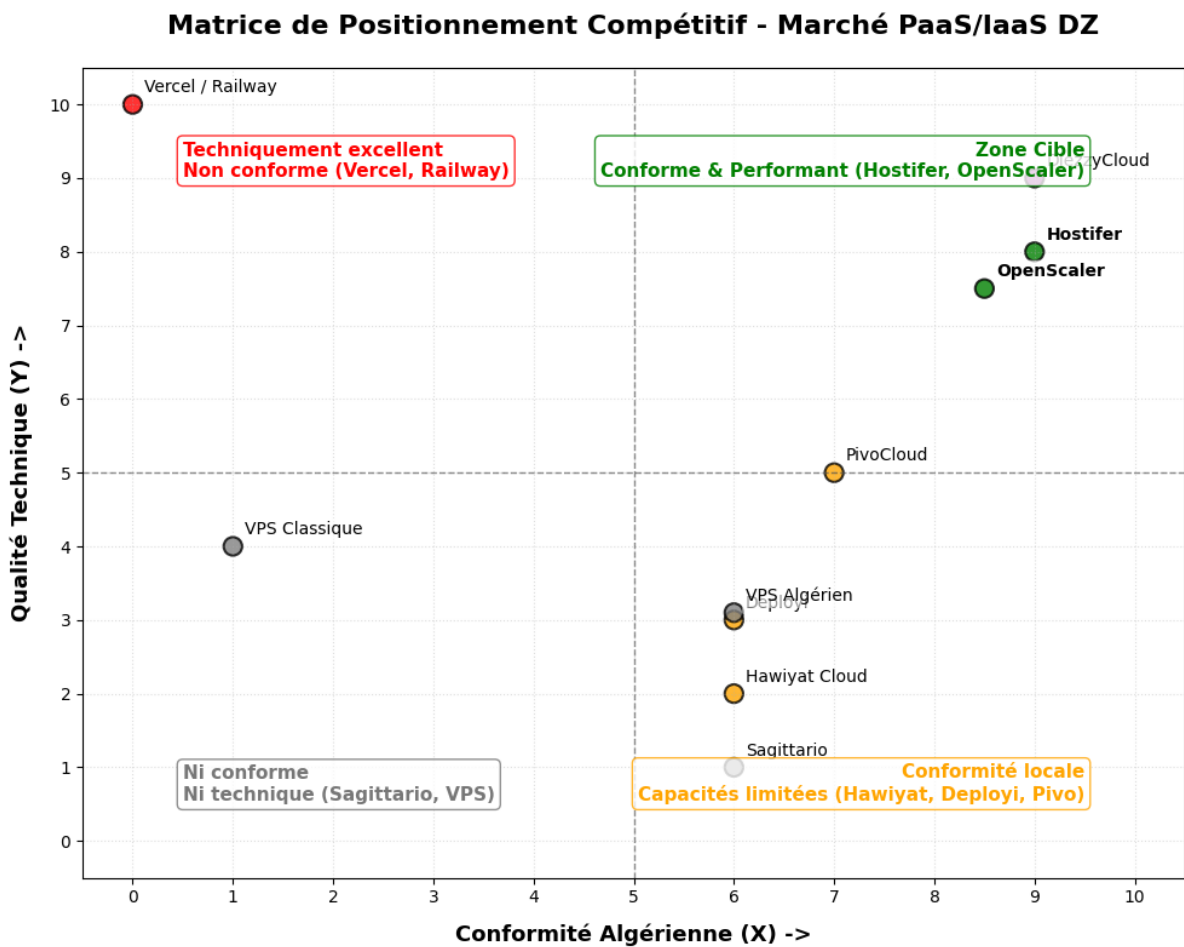


FIGURE 1.5 – Matrice de positionnement compétitif (axes : qualité technique / conformité algérienne)

Chapitre 2

Conception et Architecture du Système

2.1 Recueil et spécification des besoins

La phase de recueil et de spécification des besoins constitue le fondement de toute démarche d'ingénierie logicielle rigoureuse. Elle permet d'établir un contrat clair entre les parties prenantes et l'équipe de développement, en formalisant les attentes fonctionnelles et les contraintes non fonctionnelles du système. Dans le cadre de Hostifer, cette phase revêt une importance particulière, car la plateforme doit répondre simultanément aux exigences d'une infrastructure multi-tenant à haute disponibilité, aux contraintes réglementaires algériennes, et aux attentes d'utilisabilité propres aux développeurs. Les besoins ont été recueillis par une analyse itérative combinant l'étude des solutions existantes (menée au chapitre précédent) et la modélisation des cas d'usage réels identifiés lors de la conception.

2.1.1 Identification des acteurs

Au sens de la norme UML 2.5, un acteur est un *behaviored classifier* qui modélise le rôle joué par une entité externe au système, qu'il s'agisse d'un utilisateur humain, d'un système logiciel tiers ou d'un composant matériel, et qui interagit avec le sujet par échange de signaux et de données [17]. L'acteur est, par définition, placé en dehors de la frontière du système; il n'en fait pas partie mais en déclenche ou en consulte les comportements observables [18]. L'identification rigoureuse des acteurs est une étape préalable à toute modélisation des cas d'utilisation, car elle délimite le périmètre du système et constitue le point de départ de l'élicitation des besoins [19].

Pour Hostifer, l'analyse des diagrammes de séquence et des flux d'interaction a permis d'identifier deux catégories d'acteurs : les **acteurs humains**, qui initient des actions via une interface utilisateur, et les **acteurs systèmes**, qui interagissent avec la plateforme

de manière automatisée, soit en tant que systèmes externes déclencheurs, soit en tant que composants d'infrastructure participant aux workflows internes. Le tableau 2.1 présente la synthèse complète des acteurs identifiés.

Acteurs humains

L'Utilisateur (Développeur) est l'acteur principal de la plateforme. Il représente tout profil de développeur souhaitant déployer et opérer des applications web sans gérer directement l'infrastructure sous-jacente : ingénieur logiciel, freelance, membre d'une startup ou d'une PME. L'utilisateur interagit avec Hostifer exclusivement via le **frontend Next.js**, qui constitue le point d'entrée graphique de la plateforme. Ses actions couvrent l'ensemble du cycle de vie applicatif : authentification, création et configuration de projets, déclenchement de déploiements, consultation des journaux de build en temps réel, gestion des variables d'environnement, et activation du pipeline CI/CD automatique.

L'Administrateur de la plateforme est un acteur humain secondaire disposant de droits étendus sur l'ensemble du système. Il supervise la santé de l'infrastructure, gère les plans tarifaires et les quotas de ressources par tenant, et intervient en cas d'incident. L'administrateur interagit principalement avec les outils d'observabilité et d'orchestration (tableaux de bord Grafana, interface Argo Workflows UI) ainsi qu'avec les API d'administration exposées par le **backend NestJS**.

Acteurs systèmes externes

NextAuth.js est la bibliothèque d'authentification intégrée au frontend Next.js. Elle gère les sessions utilisateur côté client, orchestre le flux OAuth 2.0 vers GitHub, et délègue l'émission et le renouvellement des jetons JWT au backend NestJS via des callbacks configurables. Bien qu'elle soit un composant interne au frontend, son rôle de médiateur entre l'utilisateur, le fournisseur OAuth et le backend en fait un participant structurant des diagrammes d'interaction.

GitHub OAuth est le fournisseur d'identité externe utilisé pour l'authentification déléguée. Lors d'une connexion via OAuth 2.0, GitHub valide l'identité de l'utilisateur et retourne un code d'autorisation que NextAuth.js échange contre un jeton d'accès. En complément, **GitHub** agit comme système déclencheur du pipeline CI/CD : à chaque événement **push** sur une branche configurée, il émet une requête HTTP vers l'endpoint webhook du backend NestJS, initiant automatiquement un nouveau déploiement sans intervention humaine.

Acteurs systèmes d’orchestration et d’infrastructure

Argo Workflows est le moteur d’orchestration des pipelines de déploiement. Il reçoit les soumissions de *WorkflowTemplates* depuis le backend NestJS via l’API REST Argo, puis exécute séquentiellement les trois étapes du pipeline : *Build*, *Provision* et *DNS*. Argo Workflows interagit directement avec l’**API Kubernetes** pour créer et surveiller les ressources d’exécution de chaque étape.

Le Builder Job (Go) est le composant exécuté sous forme de Job Kubernetes lors de l’étape *Build*. Ce binaire Go orchestre la chaîne de construction complète : clonage du dépôt Git, invocation de **Railpack CLI** pour la détection du framework et la génération du plan de build, connexion au daemon **Buildkitd** pour la construction de l’image OCI, et push de l’image résultante vers le **registre interne (Registry OCI)**.

Helm est le gestionnaire de packages Kubernetes utilisé lors de l’étape *Provision*. Il déploie ou met à jour le chart Helm dédié à chaque tenant, créant l’ensemble des ressources Kubernetes nécessaires : *Namespace*, *Deployment*, *Service*, *NetworkPolicy*, *ResourceQuota*, *HorizontalPodAutoscaler* et *ExternalSecret*.

L’API Kubernetes est le point d’entrée central de l’infrastructure d’exécution. Elle est sollicitée par Argo Workflows, par Helm et directement par le backend NestJS pour la création, la mise à jour et la suppression de l’ensemble des ressources Kubernetes associées aux tenants [6].

Vault + ESO (External Secrets Operator) constituent le sous-système de gestion des secrets. HashiCorp Vault stocke les variables d’environnement sensibles des tenants dans un moteur KV v2 chiffré. L’External Secrets Operator synchronise ces secrets vers des ressources *Kubernetes Secret* natives, qui sont ensuite injectées dans les pods applicatifs via `envFrom`, sans que les valeurs ne transitent jamais en clair.

Traefik est le contrôleur d’entrée (*Ingress Controller*) du cluster. Il assure le routage du trafic HTTP/HTTPS entrant vers les services des tenants selon les règles d’Ingress-Route définies par le chart Helm, et s’intègre avec **cert-manager** pour le provisionnement et le renouvellement automatiques des certificats TLS via Let’s Encrypt.

Promtail (DaemonSet) et **Loki** forment le sous-système de collecte et d’indexation des journaux. Promtail, déployé en tant que DaemonSet sur chaque nœud du cluster, collecte les flux `stdout` des pods de construction et y attache des labels structurés (`build_id`, `tenant_id`, `log_type`). Les journaux sont transmis à Loki pour indexation et persistance, depuis lesquels le backend NestJS les interroge via l’API `query_range` pour les diffuser en temps réel à l’utilisateur via SSE (*Server-Sent Events*).

TABLEAU 2.1 – Synthèse des acteurs du système Hostifer

Acteur	Type	Catégorie	Rôle principal
Utilisateur	Humain	Principal	Déploie et opère ses applications via le frontend Next.js
Administrateur	Humain	Secondaire	Supervise la plateforme et l'infrastructure
NextAuth.js	Système interne	Secondaire	Gère les sessions et orchestre le flux OAuth côté frontend
GitHub OAuth	Système externe	Secondaire	Fournisseur d'identité OAuth 2.0
GitHub (web-hook)	Système externe	Déclencheur	Déclenche le CI/CD via événement push
Argo Workflows	Système interne	Orchestrateur	Exécute les pipelines Build / Provision / DNS
Builder Job (Go)	Système interne	Automatisé	Construit et pousse l'image OCI via Railpack et Buildkitd
Railpack CLI	Système interne	Automatisé	Détecte le framework et génère le plan de build
Buildkitd	Système interne	Automatisé	Construit l'image OCI à partir du plan Railpack
Registry (OCI)	Système interne	Automatisé	Stocke les images OCI des applications
Kubernetes API	Système interne	Infrastructure	Point d'entrée pour toutes les opérations sur les ressources cluster
Helm	Système interne	Infrastructure	Provisionne les ressources Kubernetes par tenant via chart
Vault + ESO	Système interne	Sécurité	Stocke et injecte les secrets applicatifs chiffrés

Suite à la page suivante

Acteur	Type	Catégorie	Rôle principal
cert-manager	Système interne	Infrastructure	Émet et renouvelle les certificats TLS automatiquement
Traefik	Système interne	Infrastructure	Route le trafic entrant et assure la terminaison TLS
Loki	Système interne	Observabilité	Indexe et persiste les journaux des pods
Promtail (DaemonSet)	Système interne	Observabilité	Collecte les journaux des pods et les transmet à Loki

2.1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent l'ensemble des services que le système doit fournir à ses utilisateurs ainsi que les comportements attendus en réponse à des entrées spécifiques [19]. Ils ont été structurés en sept domaines fonctionnels cohérents avec les grands cas d'utilisation de la plateforme, et priorisés selon la méthode MoSCoW (*Must have, Should have, Could have, Won't have*) [20], qui permet d'aligner les efforts de développement sur les fonctionnalités à plus fort impact métier.

Authentification et gestion des comptes. La plateforme doit permettre à tout utilisateur de s'inscrire et de s'authentifier, soit par une paire identifiant/mot de passe avec hachage sécurisé, soit par le protocole OAuth 2.0 délégué à GitHub. Le système doit émettre et renouveler des jetons d'accès JWT (*JSON Web Token*) de manière transparente, avec un mécanisme de rotation des jetons de rafraîchissement afin de maintenir la sécurité des sessions longues.

Gestion des projets. Un développeur authentifié doit pouvoir créer un projet en fournissant l'URL d'un dépôt GitHub, configurer ses paramètres (région de déploiement, plan tarifaire associé), consulter la liste de ses projets avec leur statut en temps réel, et supprimer un projet avec déprovisionnement complet des ressources Kubernetes associées.

Déploiement d'applications. La fonctionnalité centrale de la plateforme consiste à déployer automatiquement une application depuis un dépôt GitHub vers l'infrastructure Kubernetes. Ce processus doit couvrir la détection automatique du framework utilisé, la construction de l'image conteneur via Railpack et Buildkit, son stockage dans un registre interne, et son déploiement via un chart Helm dédié par tenant. L'utilisateur doit avoir accès à l'historique complet des déploiements et pouvoir effectuer un retour arrière vers une version stable antérieure.

Journalisation en temps réel. Pendant et après le processus de construction, l'utilisateur doit pouvoir consulter les journaux de build en continu, sans rechargement de page, via une connexion SSE (*Server-Sent Events*). Les journaux doivent être indexés et persistés dans Loki afin d'être consultables a posteriori.

Gestion des variables d'environnement. La plateforme doit permettre la création, la mise à jour et la suppression de variables d'environnement associées à chaque projet. Les valeurs sensibles (secrets) doivent être stockées de manière chiffrée dans HashiCorp Vault et jamais exposées en clair dans l'interface ou dans les journaux.

Intégration et livraison continues (CI/CD). Le système doit permettre la configuration d'un pipeline CI/CD automatique, déclenché par un événement `push` GitHub via webhook. L'utilisateur doit pouvoir activer ou désactiver ce comportement par projet, et choisir la branche de déploiement cible.

Administration de la plateforme. L'administrateur doit disposer d'une interface permettant de visualiser l'ensemble des tenants actifs, leur consommation de ressources, et les workflows Argo Workflows en cours d'exécution. Il doit pouvoir intervenir sur les ressources Kubernetes directement depuis les outils d'observabilité intégrés.

Le tableau 2.2 synthétise l'ensemble des besoins fonctionnels avec leur identifiant, leur description, leur priorité MoSCoW et l'acteur concerné.

TABLEAU 2.2 – Tableau des besoins fonctionnels

ID	Description	Priorité	Acteur concerné
BF-01	S'inscrire avec email/mot de passe	Must have	Développeur
BF-02	S'authentifier via OAuth GitHub	Must have	Développeur
BF-03	Émettre et renouveler des jetons JWT	Must have	Système (NestJS)
BF-04	Créer un projet depuis une URL GitHub	Must have	Développeur
BF-05	Consulter la liste des projets et leurs statuts	Must have	Développeur
BF-06	Déclencher manuellement un déploiement	Must have	Développeur

Suite à la page suivante

ID	Description	Priorité	Acteur concerné
BF-07	Détecter automatiquement le framework applicatif	Must have	Système (Builder Go)
BF-08	Construire l'image conteneur via Buildkit/Railpack	Must have	Système (Argo Workflows)
BF-09	Déployer l'application via chart Helm	Must have	Système (Argo Workflows)
BF-10	Consulter les journaux de build en temps réel (SSE)	Must have	Développeur
BF-11	Gérer les variables d'environnement (CRUD)	Must have	Développeur
BF-12	Stocker les secrets dans Hashi-Corp Vault	Must have	Système (NestJS/ESO)
BF-13	Configurer le déploiement automatique sur push GitHub	Should have	Développeur
BF-14	Consulter l'historique des déploiements	Should have	Développeur
BF-15	Effectuer un retour arrière vers une version précédente	Should have	Développeur
BF-16	Supprimer un projet avec déprovisionnement Kubernetes	Must have	Développeur
BF-17	Gérer les membres d'une équipe projet	Should have	Développeur
BF-18	Administrer les tenants et ressources (interface admin)	Must have	Administrateur
BF-19	Visualiser les workflows Argo en cours	Should have	Administrateur
BF-20	Configurer un domaine personnalisé	Could have	Développeur

2.1.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles le système doit se conformer, indépendamment des fonctionnalités qu’il offre. Ils couvrent des attributs tels que la performance, la disponibilité, la sécurité et la scalabilité, et constituent souvent les critères les plus déterminants pour l’expérience utilisateur réelle et la viabilité opérationnelle de la plateforme [19]. Dans le contexte d’une plateforme PaaS multi-tenant comme Hostifer, ces exigences sont particulièrement critiques, car une dégradation dans l’un de ces domaines peut simultanément affecter l’ensemble des tenants hébergés sur l’infrastructure partagée.

Performance. Le temps de déploiement de bout en bout, mesuré depuis la soumission du workflow Argo jusqu’au passage de l’application à l’état **LIVE**, ne doit pas excéder cinq minutes pour les frameworks courants (Express, NestJS, Next.js, Flask) sur un dépôt de taille standard. La latence de streaming des journaux de build via SSE doit rester inférieure à deux secondes entre l’émission d’un message par le pod de construction et son affichage dans le navigateur de l’utilisateur.

Disponibilité. La plateforme cible un SLA (*Service Level Agreement*) de disponibilité de 99,5% pour les plans Standard, et de 99,9% pour les plans Enterprise, ce qui correspond à une indisponibilité maximale tolérée respectivement de 43,8 heures et 8,76 heures par an. Ces objectifs imposent une architecture résiliente aux défaillances de composants individuels, notamment via la redondance des pods applicatifs et la gestion des interruptions planifiées.

Sécurité. L’isolation réseau entre tenants doit être garantie par des *NetworkPolicies* Kubernetes bloquant tout trafic inter-namespace non autorisé [21]. Les secrets applicatifs ne doivent jamais transiter en clair : leur cycle de vie complet (création, stockage, injection dans les pods) est pris en charge par HashiCorp Vault et l’opérateur External Secrets. Les communications entre l’utilisateur et la plateforme doivent être intégralement chiffrées via TLS, avec émission et renouvellement automatiques des certificats assurés par cert-manager et Let’s Encrypt. L’accès aux ressources Kubernetes est contrôlé par des politiques RBAC (*Role-Based Access Control*) granulaires, limitant strictement les permissions de chaque composant.

Scalabilité et autoscaling. La plateforme doit supporter une montée en charge dynamique à trois niveaux distincts [22], correspondant aux trois couches de l’architecture Kubernetes.

Au niveau des pods, le *Horizontal Pod Autoscaler* (HPA) ajuste automatiquement le nombre de répliques d’un déploiement en fonction de métriques observées telles que l’utilisation CPU ou mémoire, ou de métriques applicatives personnalisées exposées via

Prometheus Adapter [23]. Le HPA est configuré avec des seuils de tolérance adaptés afin d’éviter les phénomènes d’oscillation (*flapping*) dus à des variations métrique transitoires. En complément, le *Vertical Pod Autoscaler* (VPA) opère sur la dimension verticale en ajustant les champs `requests` et `limits` des conteneurs sur la base de l’historique de consommation réelle [24]. En mode `recommendation`, il fournit des suggestions sans perturber les pods en cours d’exécution ; en mode `auto`, il peut procéder à l’éviction et au redémarrage des pods pour appliquer les nouvelles valeurs. Afin d’éviter tout conflit avec le HPA sur les mêmes métriques, le VPA est configuré pour opérer sur des métriques distinctes lorsque les deux mécanismes coexistent [25].

Au niveau du cluster, des *node pools* classés par profil de ressources (small, medium, large) permettent d’affecter les workloads aux nœuds les mieux dimensionnés selon le plan tarifaire du tenant. Le *Cluster Autoscaler* surveille en permanence l’état des pods en attente de scheduling (*Pending*) et des nœuds sous-utilisés, ajoutant de nouveaux nœuds lorsque la demande dépasse la capacité disponible et en retirant lorsque des nœuds restent inactifs sur une période prolongée [22].

Les *ResourceQuotas* et les *LimitRanges* définis par namespace délimitent les bornes minimales et maximales de consommation de ressources par tenant. Ces contraintes co-opèrent avec le HPA et le VPA en empêchant les recommandations d’autoscaling de dépasser les quotas alloués par plan, garantissant ainsi la préservation de l’isolation multi-tenant même en période de forte charge [26].

Maintenabilité. L’architecture modulaire du backend NestJS (un module par domaine fonctionnel), l’utilisation de charts Helm paramétrables pour le provisionnement des tenants, et la séparation stricte des composants (frontend, backend, builder, infrastructure) doivent permettre la mise à jour et le débogage indépendants de chaque couche du système sans affecter les tenants en production.

Le tableau 2.3 récapitule l’ensemble des besoins non fonctionnels avec leurs critères de satisfaction mesurables.

TABLEAU 2.3 – Tableau des besoins non fonctionnels

ID	Catégorie	Description	Critère de satisfaction
BNF-01	Performance	Temps de déploiement de bout en bout	≤ 5 min pour frameworks standard
BNF-02	Performance	Latence de streaming des logs SSE	≤ 2 s entre émission pod et affichage navigateur

ID	Catégorie	Description	Critère de satisfaction
BNF-03	Disponibilité	SLA plan Standard	$\geq 99,5\%$ (43,8 h d'indisponibilité/an max.)
BNF-04	Disponibilité	SLA plan Entreprise	$\geq 99,9\%$ (8,76 h d'indisponibilité/an max.)
BNF-05	Sécurité	Isolation réseau inter-tenant	Zéro trafic inter-namespace validé par NetworkPolicy
BNF-06	Sécurité	Chiffrement des secrets	Secrets stockés dans Vault KV v2, jamais en clair
BNF-07	Sécurité	Chiffrement des communications	TLS obligatoire, certificats émis par cert-manager
BNF-08	Sécurité	Contrôle d'accès	RBAC Kubernetes par namespace et par composant
BNF-09	Scalabilité	Autoscaling horizontal des pods (HPA)	Augmentation du nombre de réplicas en ≤ 30 s sous charge CPU $> 70\%$
BNF-10	Scalabilité	Autoscaling vertical des pods (VPA)	Recommandations requests/limits appliquées sans interruption en mode recommendation
BNF-11	Scalabilité	Autoscaling du cluster (Cluster Autoscaler)	Ajout/suppression automatique de nœuds en ≤ 3 min selon utilisation des ressources

ID	Catégorie	Description	Critère de satisfaction
BNF-12	Scalabilité	Isolation des quotas multi-tenant	ResourceQuota empêche tout dépassement du plan tarifaire, même en cas de scaling
BNF-13	Maintenabilité	Modularité du backend	Mise à jour d'un module NestJS sans impact sur les autres
BNF-14	Maintenabilité	Paramétrabilité de l'infrastructure	Provisionnement complet d'un nouveau tenant via chart Helm sans modification manuelle

2.2 Modélisation UML des cas d'utilisation

2.2.1 Diagramme de cas d'utilisation global

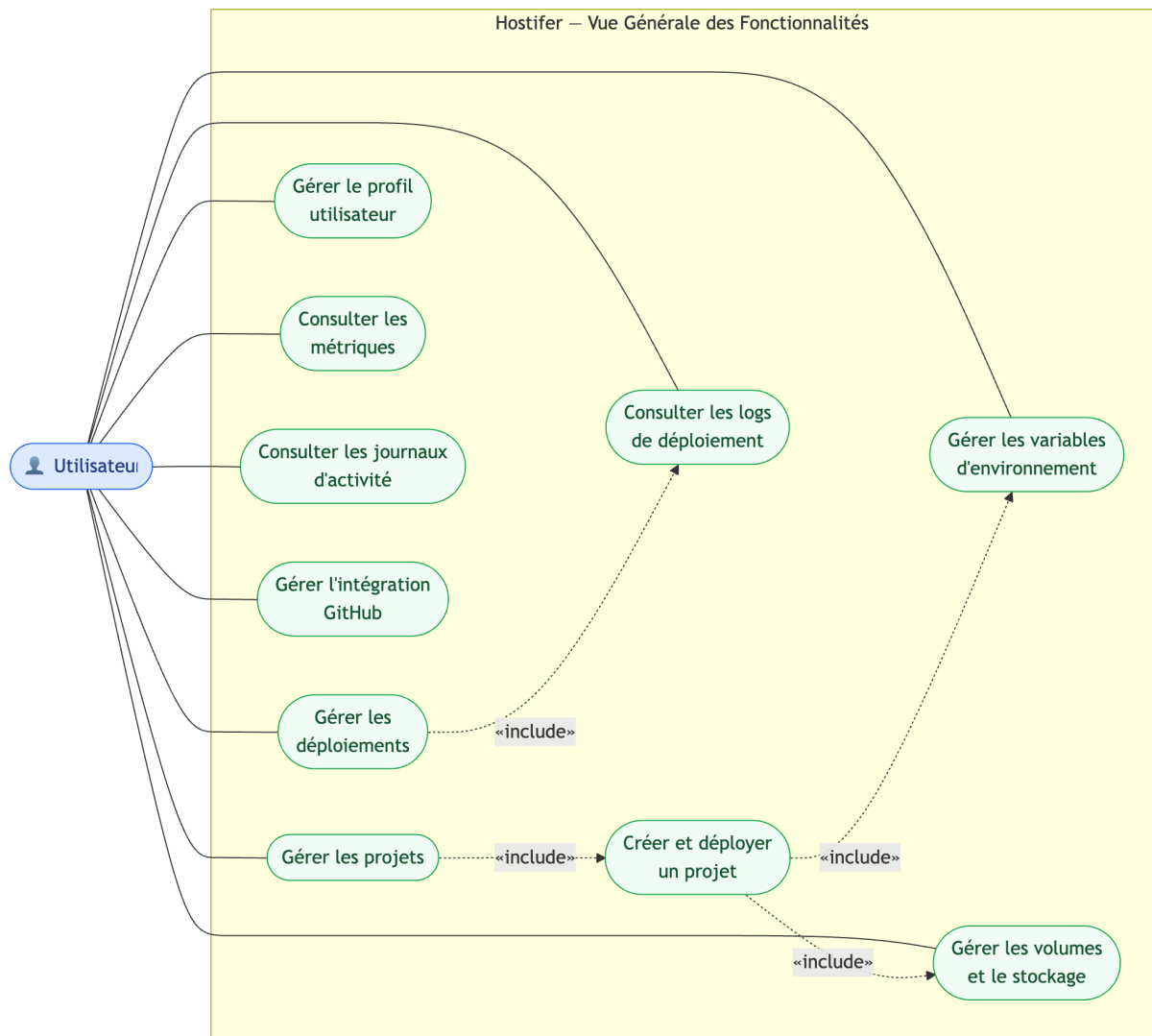


FIGURE 2.1 – Diagramme de cas d'utilisation global montrant tous les acteurs et leurs interactions avec le système (authentification, déploiement, gestion des projets, administration, CI/CD, logs)

2.2.2 Cas d'utilisation détaillés

2.2.2.1 CU01 : Créer et déployer un projet.png

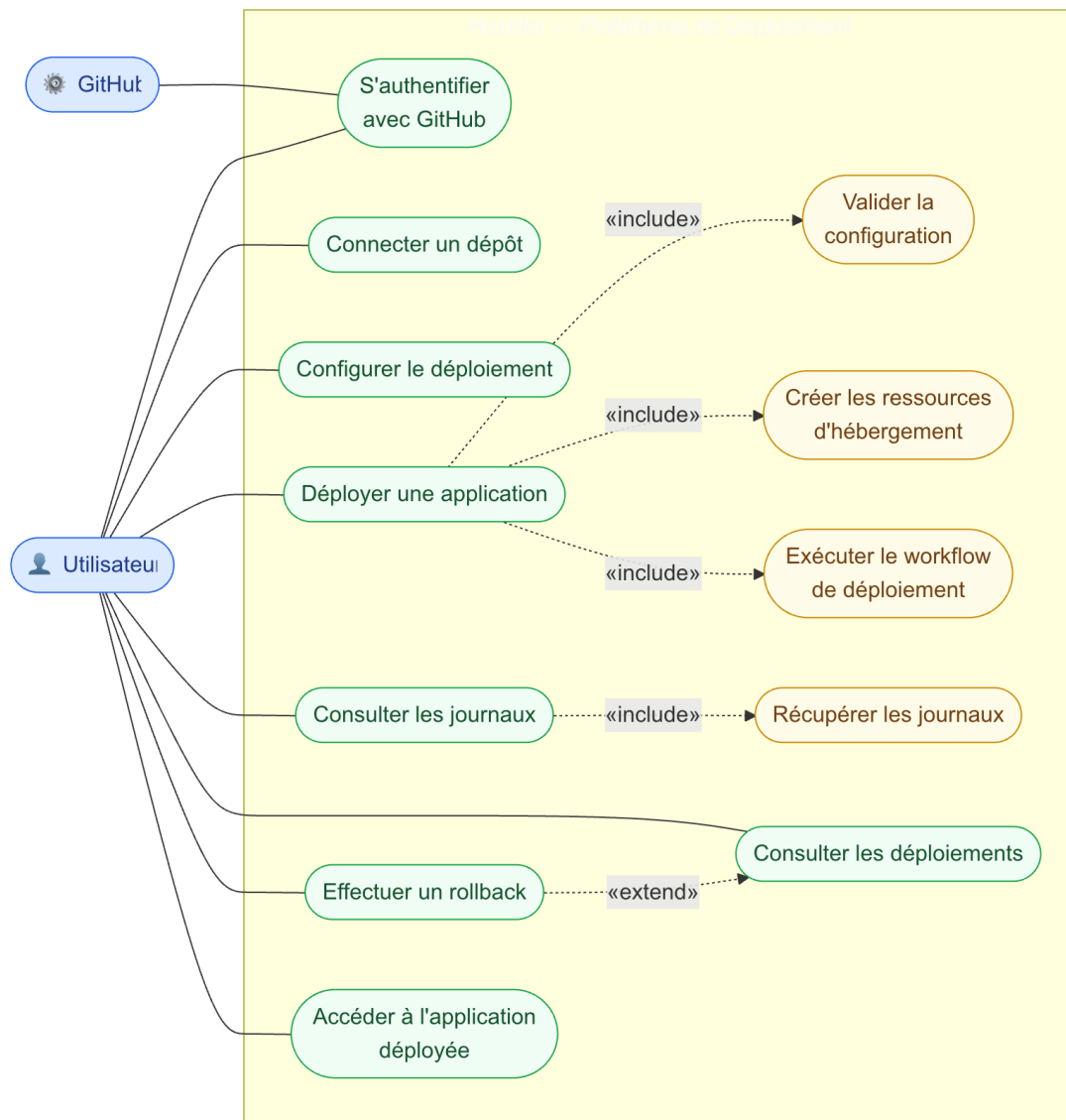


FIGURE 2.2 – Diagramme de cas d'utilisation montrant les interactions spécifiques de l'utilisateur avec le système pour créer un projet, configurer les paramètres, déclencher un déploiement et consulter les journaux en temps réel

2.2.2.2 CU02 : Gestion des Variables d'Environnement

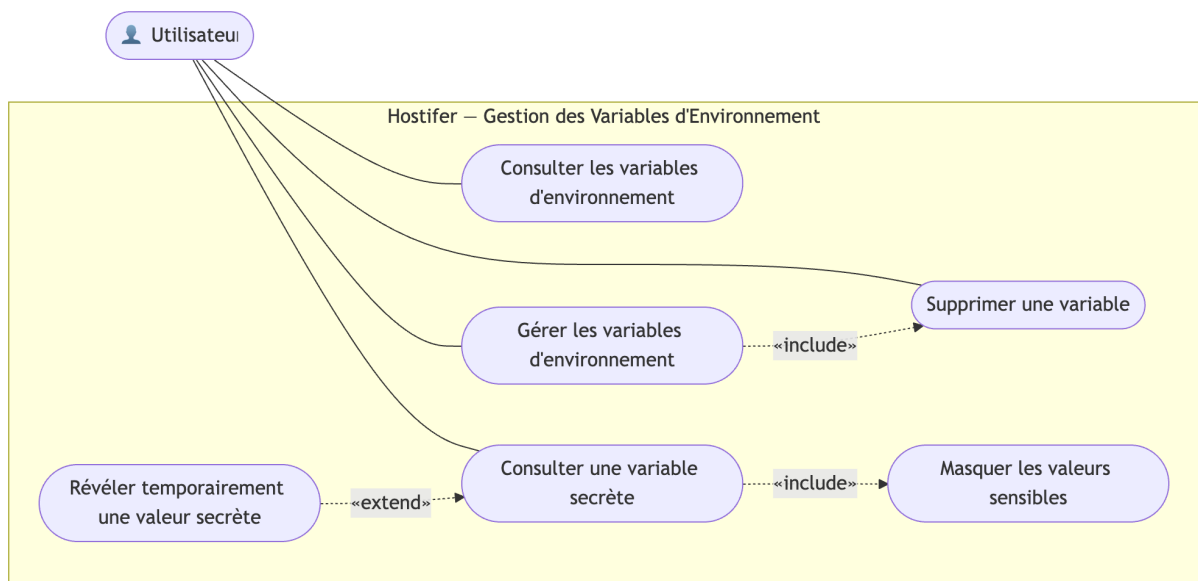


FIGURE 2.3 – Diagramme de cas d'utilisation montrant les interactions spécifiques de l'utilisateur avec le système pour gérer les variables d'environnement

2.2.2.3 CU03 : Gestion des Volumes et Stockage Persistant

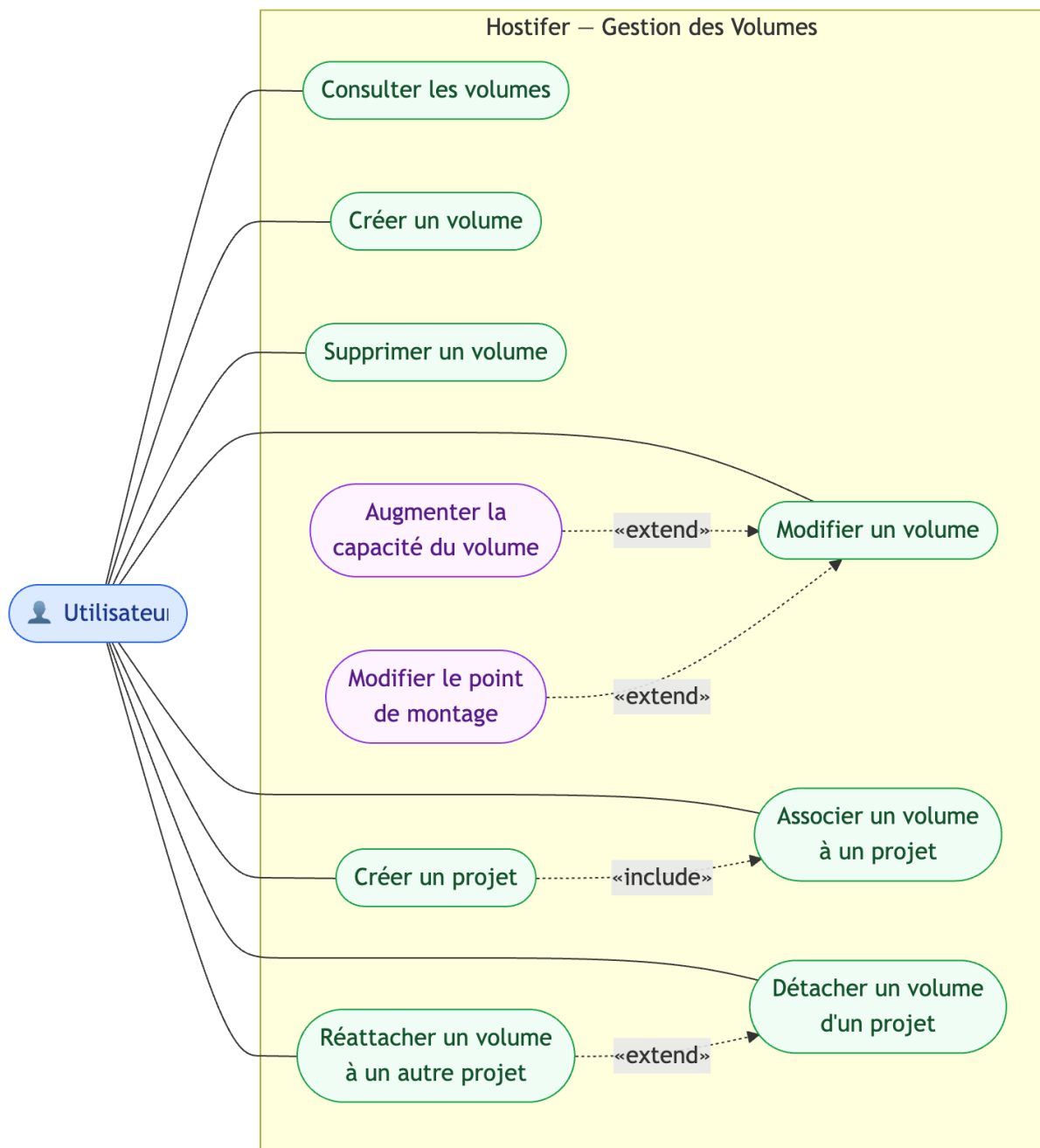


FIGURE 2.4 – Diagramme de cas d'utilisation montrant les interactions spécifiques de l'utilisateur avec le système pour gérer les volumes de stockage persistants associés à son projet

2.3 Modélisation des classes et données

2.3.1 Diagramme de classes

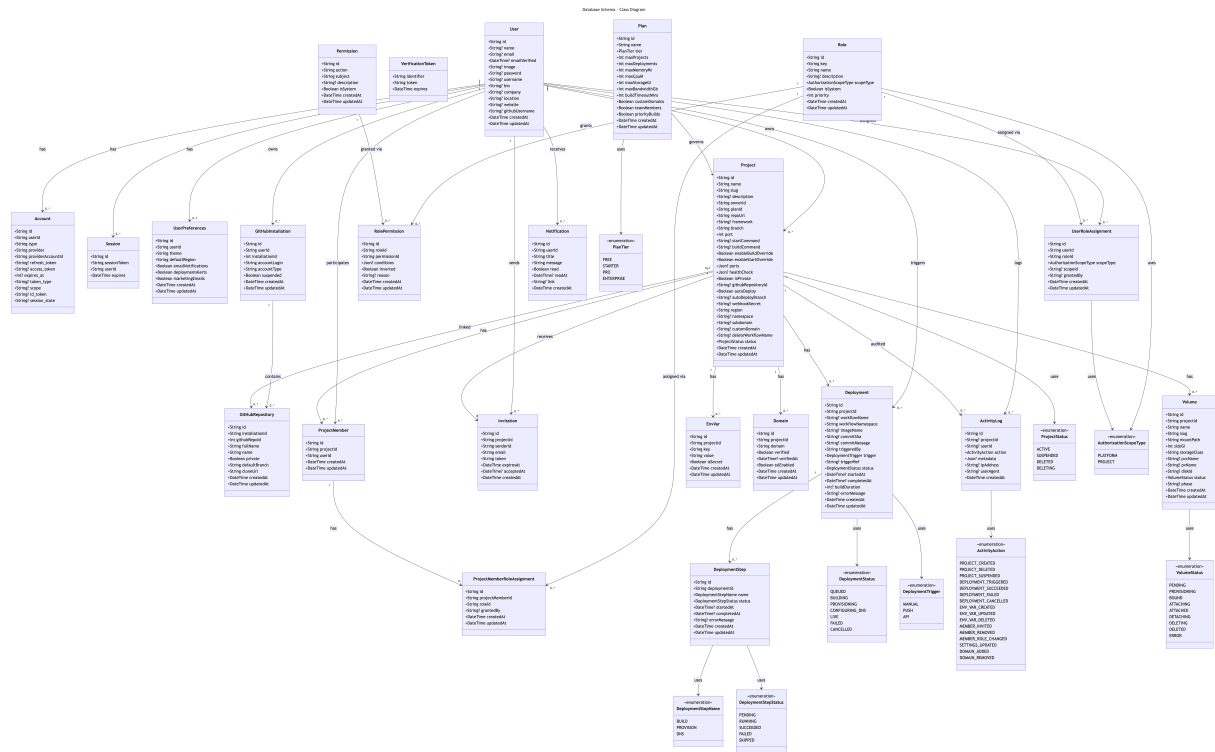


FIGURE 2.5 – Diagramme de classes UML complet (User, Project, Deployment, DeploymentStep, EnvVar, Plan, GitHubInstallation, GitHubRepository, ActivityLog, Notification, Domain)

2.3.2 Modèle Entité-Association

FIGURE 2.6 – Diagramme Entité-Association de la base de données PostgreSQL avec cardinalités

2.3.3 Description des entités principales

La modélisation des entités constitue le socle de la couche de persistance de la plateforme. Chaque entité est implémentée sous forme de modèle Prisma ORM et correspond à une table dans la base de données PostgreSQL. Les tableaux suivants décrivent, pour chaque entité principale, l'ensemble des attributs avec leurs types de données, leurs contraintes d'intégrité, et leurs relations avec les autres entités du schéma.

2.4 Modélisation comportementale

2.4.1 Diagrammes de séquence

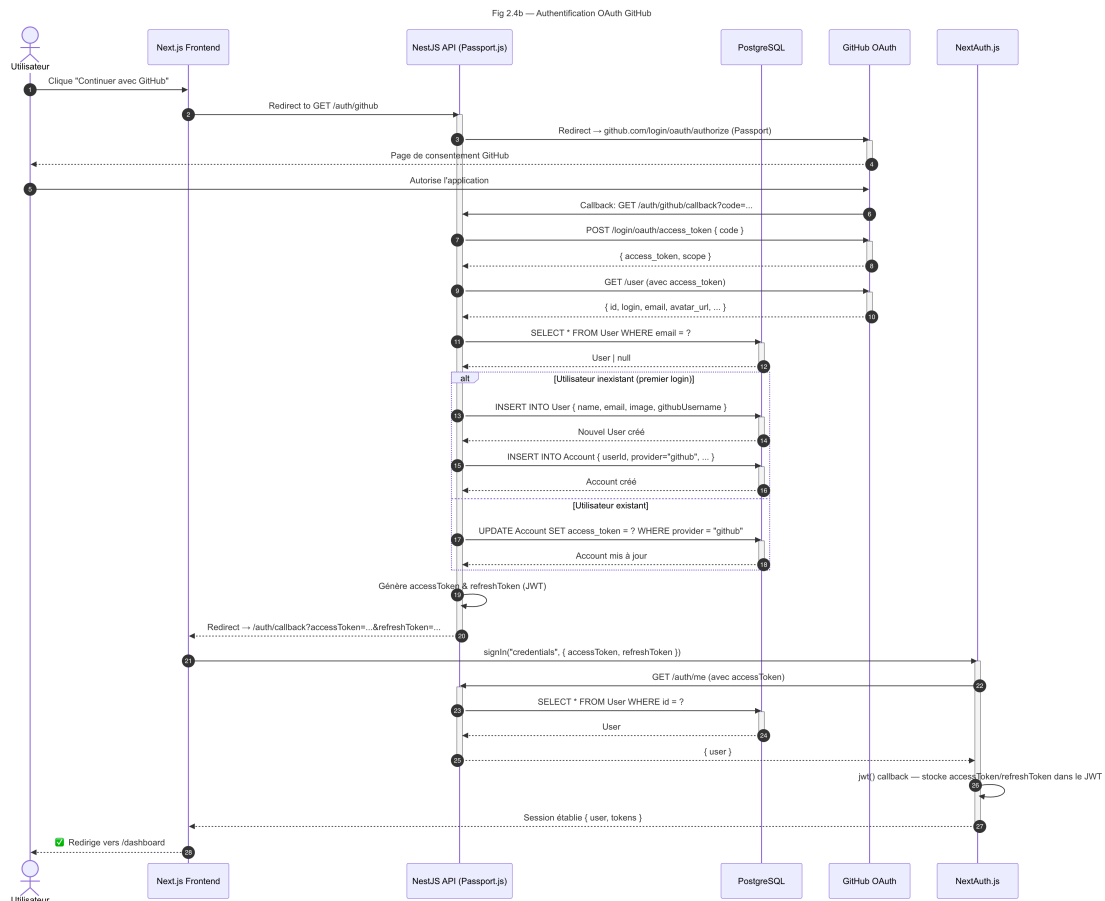


FIGURE 2.7 — Diagramme de séquence : Authentification (NextJS → NestJS → PostgreSQL → JWT)

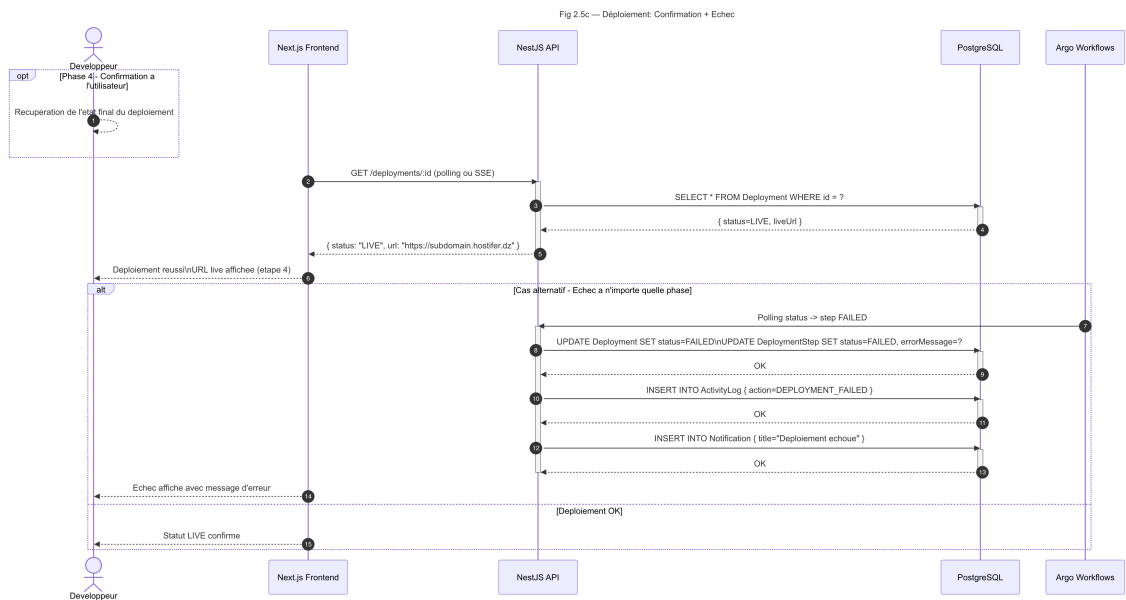


FIGURE 2.8 – Diagramme de séquence : Déclenchement d'un déploiement (Frontend → NestJS → Argo Workflows → Build Job → Registry → Helm → DNS)

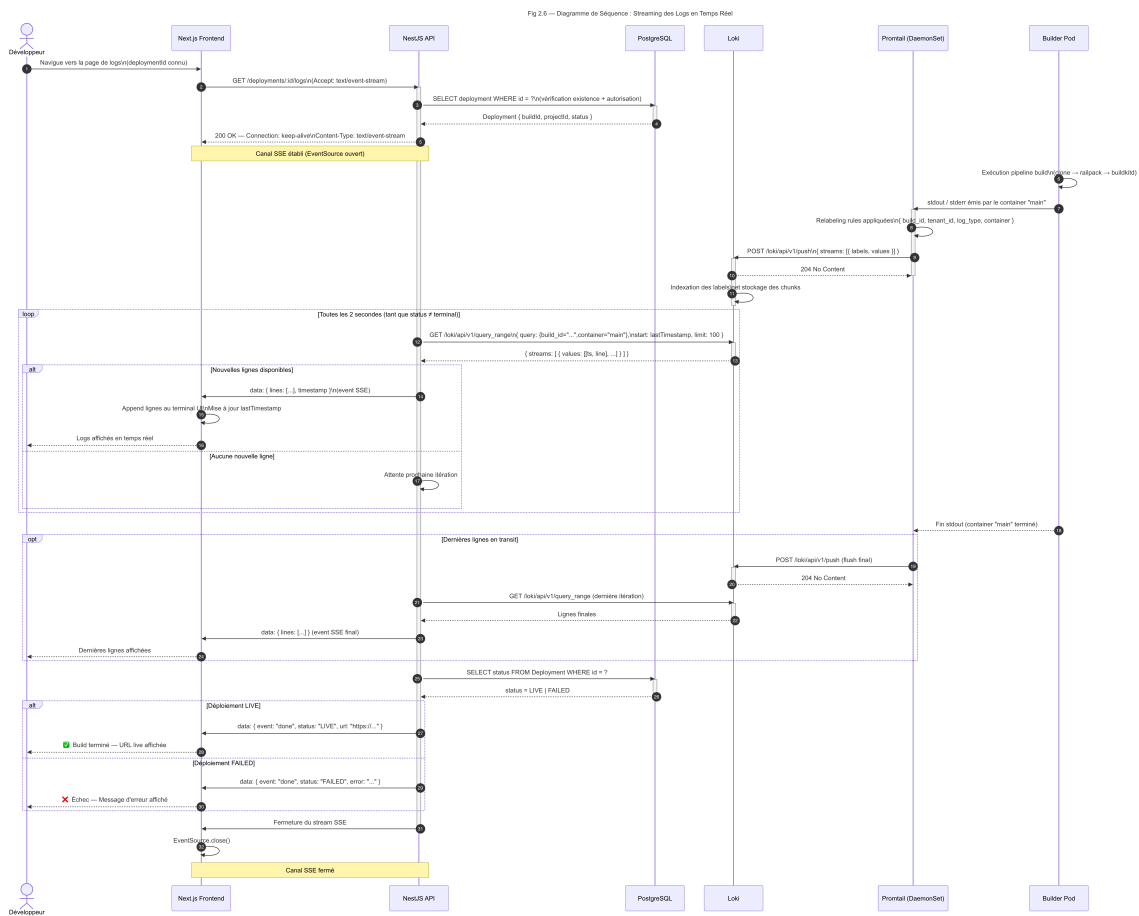


FIGURE 2.9 — Diagramme de séquence : Streaming des logs en temps réel (Builder Pod → Promtail → Loki → NestJS SSE → Browser)

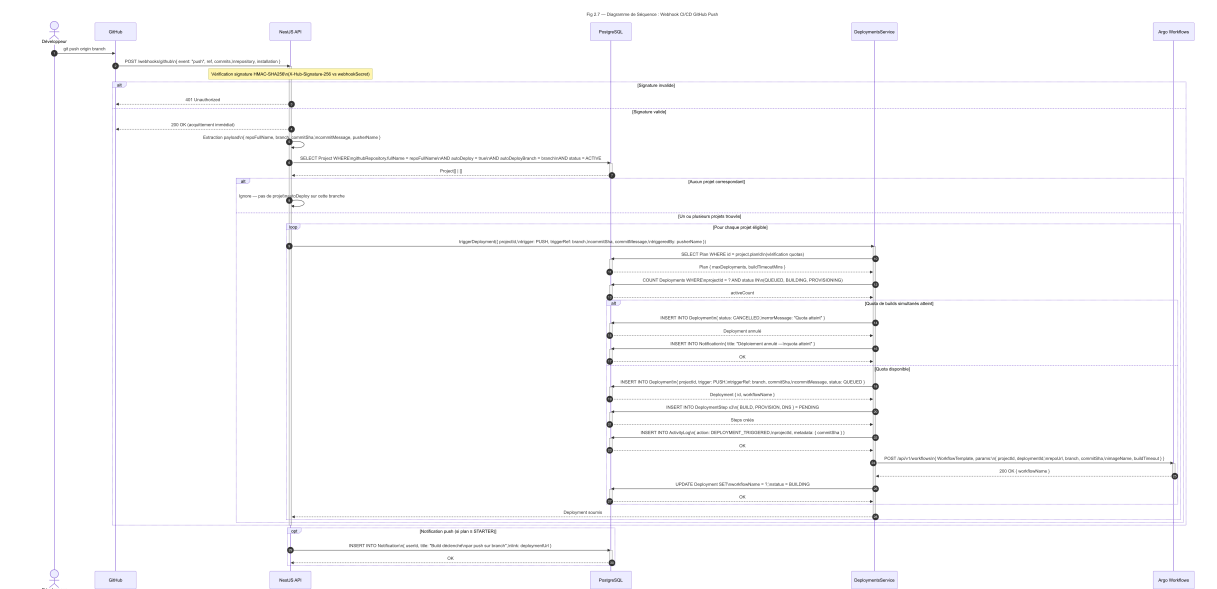


FIGURE 2.10 – Diagramme de séquence : Webhook CI/CD GitHub Push (GitHub → NestJS → DeploymentsService → Argo Workflows)

2.4.2 Diagramme d'états

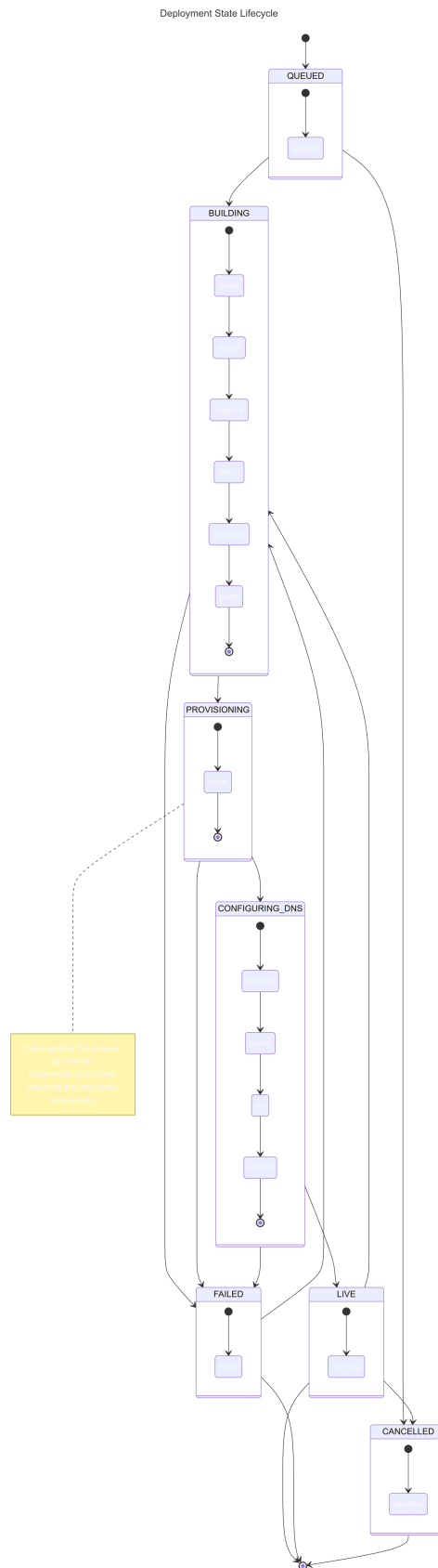


FIGURE 2.11 – Diagramme d'états d'un Deployment (QUEUED → BUILDING → PROVISIONING → CONFIGURING_DNS → LIVE / FAILED / CANCELLED)

2.5 Architecture technique globale

2.5.1 Vue d'ensemble de l'architecture

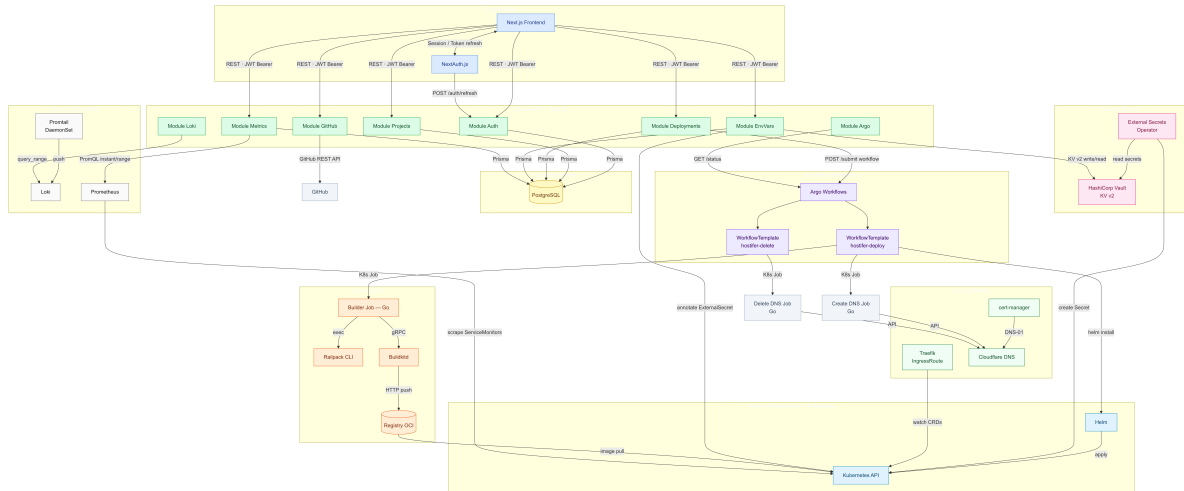
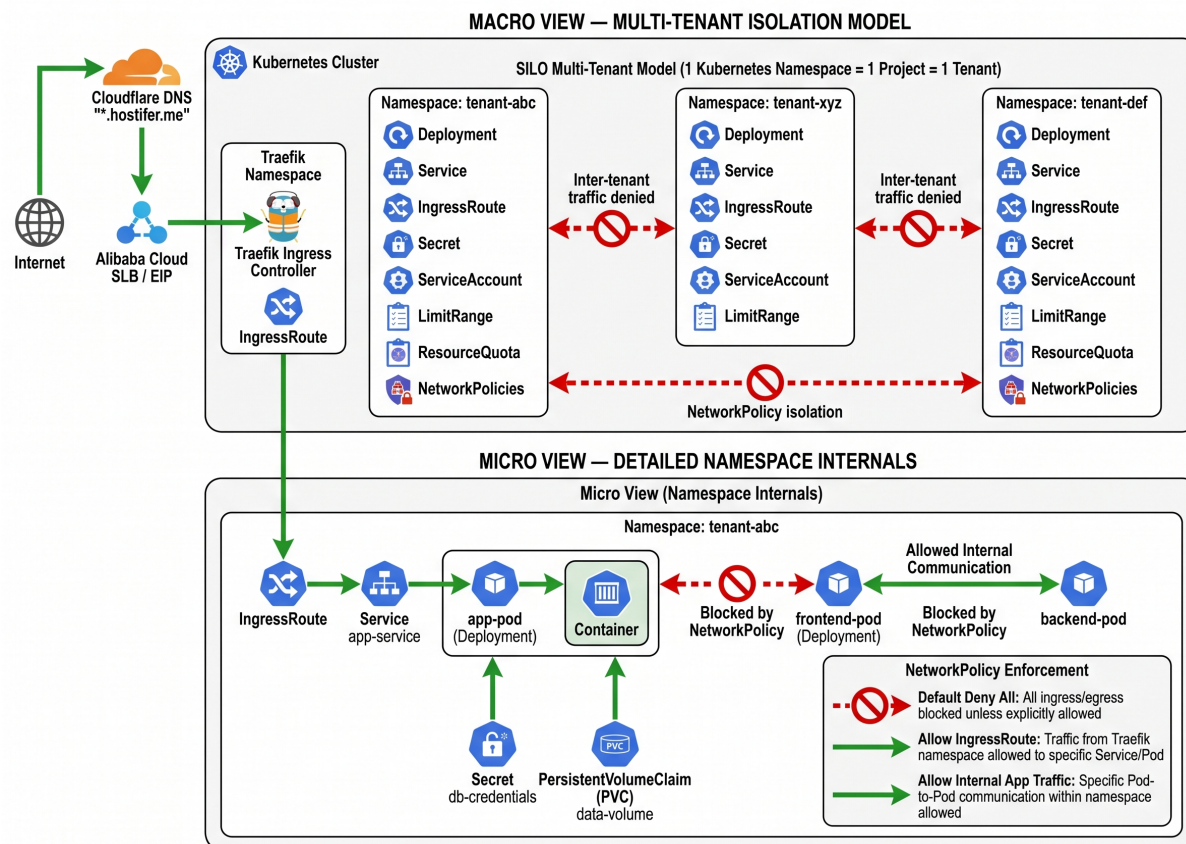


FIGURE 2.12 – Schéma d'architecture générale de Hostifer : couche client (NextJS), couche API (NestJS), couche orchestration (Argo Workflows), couche infrastructure (Kubernetes, Buildkit, Registry, Loki, Vault), couche réseau (Traefik, Cloudflare, cert-manager)

2.5.2 Architecture multi-tenant

Figure 2.12 — Multi-Tenant Isolation Architecture in Hostifer Kubernetes Platform**FIGURE 2.13** — Schéma d'isolation multi-tenant : Namespace par projet, NetworkPolicy bloquant le trafic inter-tenant, ressources quotas par plan**TABEAU 2.4** — Ressources allouées par profil d'allocation et comportements d'autoscaling

Profil	CPU request / limit	RAM request / limit	Stockage (PVC NAS)	Pods max	Réplicas HPA max	Autoscaling supporté
Free	100m / 500m	128Mi / 512Mi	—	2	1 (fixe)	Aucun autoscaling. Ressources fixes. HPA et VPA désactivés. Node pool small uniquement.

Suite à la page suivante

Profil	CPU request / limit	RAM request / limit	Stockage (PVC NAS)	Pods max	Réplicas HPA max	Autoscaling sup- porté
Standard	250m / 1	256Mi / 1Gi	1Gi	5	3	HPA activé — métriques CPU (<code>targetCPUUtilizationPercentage:70%</code>) et mémoire. VPA en mode <code>Off</code> (recommandations consultables, non appliquées automatiquement). Node pool <code>small</code> .
Pro	500m / 2	512Mi / 2Gi	5Gi	10	5	HPA activé — CPU, mémoire et métriques personnalisées via Prometheus Adapter (<code>http_request_latency_p99</code> , <code>deployment_queue_depth</code>). VPA en mode <code>recommendation</code> — ajustement des <code>requests/limits</code> suggéré sans éviction de pods. Node pool <code>medium</code> .

Suite à la page suivante

Profil	CPU request / limit	RAM request / limit	Stockage (PVC NAS)	Pods max	Réplicas HPA max	Autoscaling sup- porté
Enterprise	1 / 4	1Gi / 8Gi	20Gi	20	10	HPA activé — CPU, mémoire et métriques personnalisées Prometheus Adapter. VPA en mode <code>auto</code> — application automatique des recommandations avec éviction contrôlée (fenêtre de stabilisation configurable). Cluster Autoscaler éligible — pods schedulables sur <code>node pool large</code> avec <code>nodeAffinity</code> dédié.
Contraintes communes à tous les profils (LimitRange)						
CPU minimum par conteneur : 10m RAM minimum par conteneur : 16Mi Les <code>requests/limits</code> VPA ne peuvent jamais dépasser les plafonds du <code>ResourceQuota</code> du namespace. Les recommandations VPA sont bornées par <code>minAllowed</code> et <code>maxAllowed</code> alignés sur les valeurs <code>request/limit</code> du profil pour éviter les oscillations de scaling. <code>persistentvolumeclaims:0</code> pour le profil Free (stockage PVC désactivé).						

2.5.3 Architecture Du Réseaux Cloud

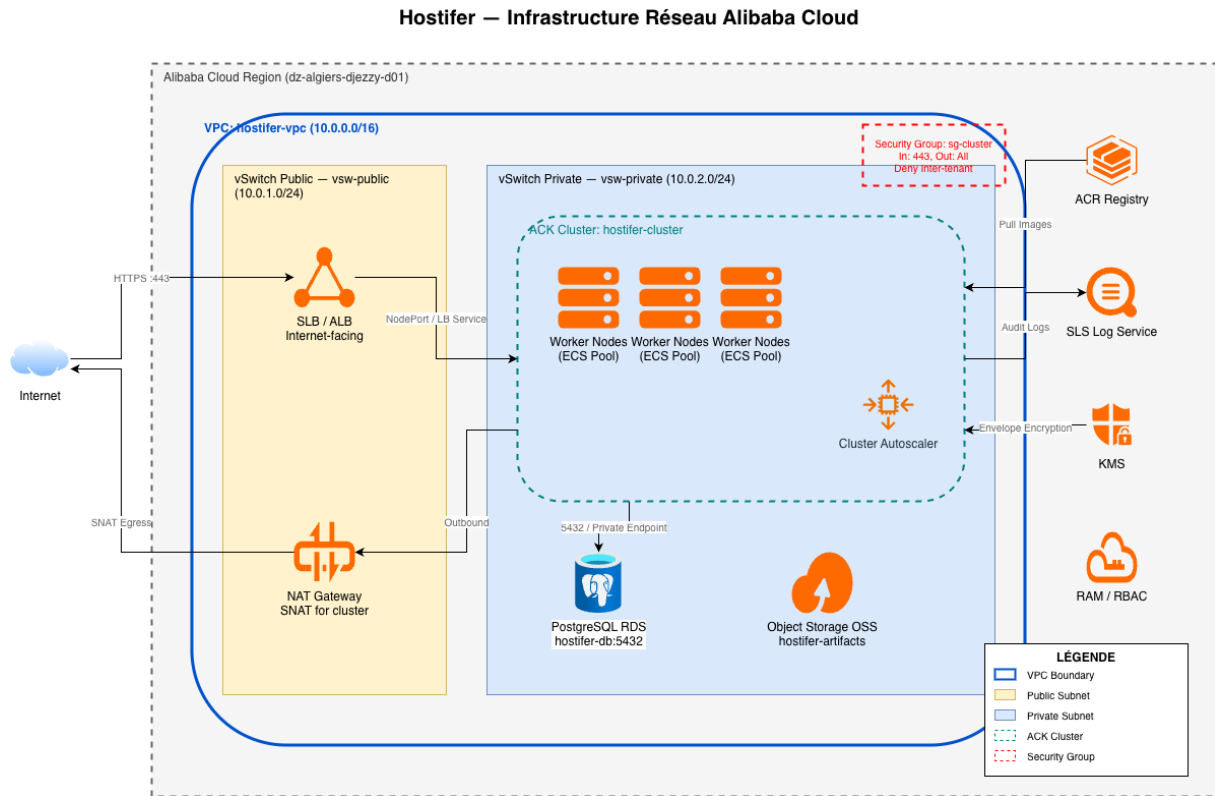


FIGURE 2.14 – Architecture réseau cloud : VPC, sous-réseaux publics/privés, Load Balancer, Worker Nodes

2.5.4 Architecture de gestion des secrets

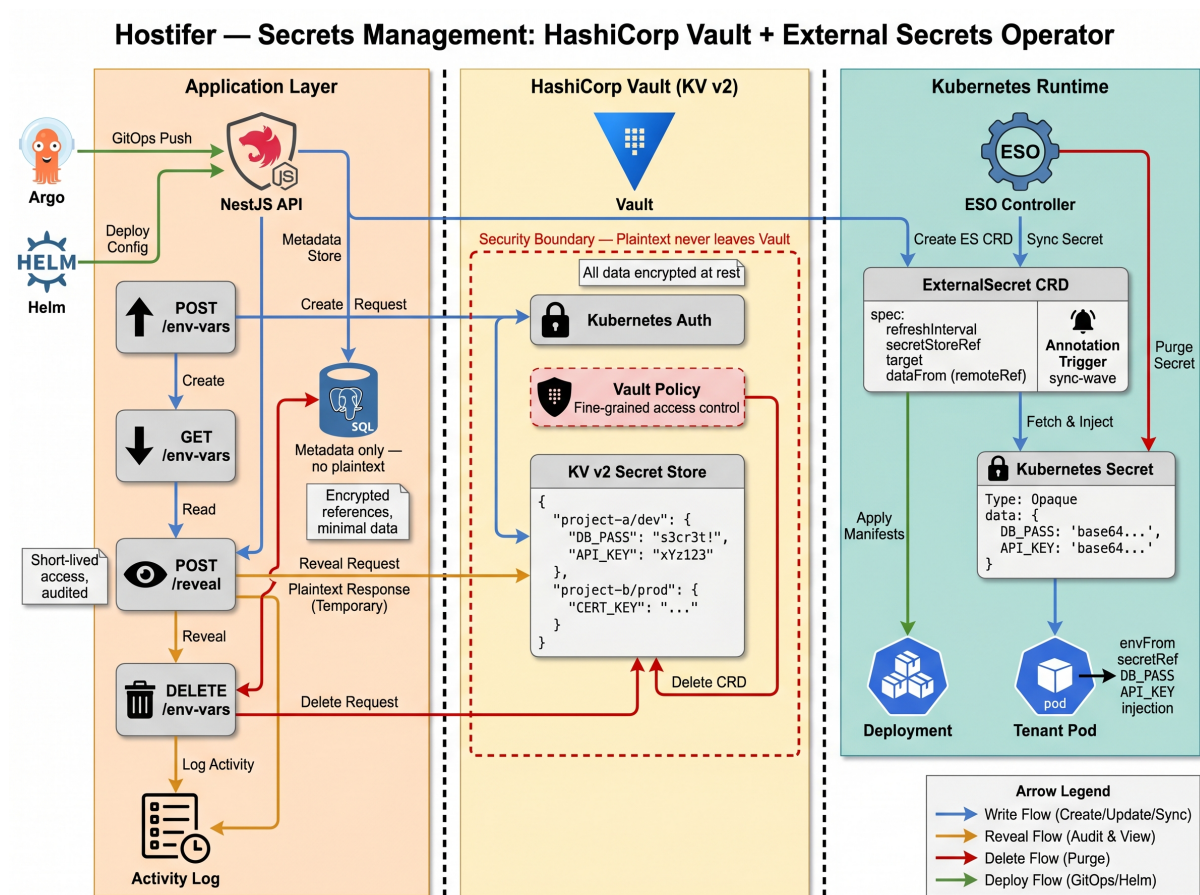


FIGURE 2.15 — Schéma de gestion des secrets avec HashiCorp Vault et External Secrets Operator

2.6 Choix technologiques

2.6.1 Justification des technologies retenues

Les choix technologiques de Hostifer résultent d'une évaluation comparative systématique des alternatives disponibles pour chaque composant de la plateforme. Chaque décision a été guidée par trois critères prioritaires : l'adéquation fonctionnelle aux exigences définies dans la section 2.1, la maturité et la maintenabilité à long terme, et la cohérence de l'ensemble de la pile technologique. Le Tableau 2.5 synthétise ces choix avec leurs justifications et les alternatives considérées.

TABLEAU 2.5 – Tableau des choix technologiques

Composant	Technologie retenue	Alternatives considérées	Justification du choix
Frontend	Next.js (TypeScript)	Nuxt.js, SvelteKit, Remix	Rendu hybride SSR/SSG natif, écosystème React mature, intégration NextAuth.js pour la gestion des sessions et des flux OAuth
Backend API	NestJS (TypeScript)	Express.js, Fastify, Hono	Architecture modulaire par domaine fonctionnel, injection de dépendances native, support SSE via <code>@Sse()</code> , décorateurs Passport pour les stratégies d'authentification, structuration facilitant la maintenabilité sur des équipes de petite taille [27]
ORM	Prisma	TypeORM, Drizzle, Sequelize	Schéma déclaratif centralisé (<code>schema.prisma</code>), client TypeScript auto-généré avec sécurité de type complète, migrations déterministes via <code>prismamigrate</code> , écosystème le plus mature pour NestJS [28]
Base de données	PostgreSQL	MySQL, MongoDB, CockroachDB	Système relationnel de référence avec support natif des transactions ACID, des index partiels et des requêtes JSON ; support natif de Prisma et de la plupart des outils d'observabilité [29]

Suite à la page suivante

Composant	Technologie retenue	Alternatives considérées	Justification du choix
Orchestration de conteneurs	Kubernetes (ACK)	Docker Swarm, Nomad, k3s	Standard industriel CNCF pour l'orchestration multi-tenant, support natif des primitives d'isolation (Namespace, NetworkPolicy, ResourceQuota, HPA, VPA), intégration native avec l'ensemble de l'écosystème cloud-native [6]
Pipeline CI/CD tenant	Argo Workflows	Tekton, Jenkins X, GitHub Actions (self-hosted)	Orchestration de workflows Kubernetes-native, interface UI intégrée, paramétrage dynamique via <i>WorkflowTemplates</i> , gestion fine des politiques de cycle de vie des pods (<code>podGC</code> , <code>ttlStrategy</code>) [30]
Outil de build d'images OCI	Buildkit + Railpack	Kaniko, BuildKit seul, Buildah, Cloud Native Buildpacks	Buildkit assure la construction parallèle avec cache de couches efficace; Railpack ajoute la détection automatique de framework sans Dockerfile requis, éliminant la friction d'onboarding pour les développeurs [5] [16]

Suite à la page suivante

Composant	Technologie retenue	Alternatives considérées	Justification du choix
Langage du builder	Go	Python, Node.js, Shell	Compilation en binaire statique sans dépendances runtime, performance élevée pour les opérations I/O intensive (clonage Git, appels gRPC BuildKit), démarrage quasi instantané comme Job Kubernetes [31]
Packaging d'infrastructure	Helm (charts OCI)	Kustomize, Terraform, Pulumi	Templating déclaratif paramétrable, distribution via registre OCI (GHCR), rollback natif, écosystème chart le plus mature pour Kubernetes [32]
Livraison continue (GitOps)	ArgoCD	Flux CD, Jenkins X	Réconciliation continue de l'état déclaré (dépôt Git) vers l'état réel du cluster, interface de visualisation des diffs, gestion des rollbacks, modèle GitOps natif [33]
Ingress Controller	Traefik	Nginx Ingress, HA-Proxy, Envoy	Support natif des CRDs <code>IngressRoute</code> pour le routage dynamique par sous-domaine, rechargement de configuration à chaud sans interruption de service, intégration native avec cert-manager pour la gestion TLS [34]

Suite à la page suivante

Composant	Technologie retenue	Alternatives considérées	Justification du choix
Gestion des certificats TLS	cert-manager + Let's Encrypt	Gestion manuelle, AWS ACM	Émission et renouvellement automatiques des certificats X.509, support du challenge DNS-01 Cloudflare pour les wildcards, intégration Kubernetes native via CRDs [35]
Gestion des secrets	HashiCorp Vault + ESO	Kubernetes Secrets (natifs), AWS Secrets Manager, Sealed Secrets	Chiffrement KV v2 des secrets applicatifs, authentification Kubernetes Auth native, synchronisation automatique vers les Kubernetes Secrets via External Secrets Operator sans exposition des valeurs en clair [36]
Collecte des journaux	Promtail + Loki	Elasticsearch + Fluentd (EFK), Datadog	Stack légère conçue pour les environnements Kubernetes, indexation par labels (sans index full-text coûteux), interrogation LogQL expressive, intégration native avec Grafana pour les dashboards [37]
Collecte des métriques	Prometheus + Grafana	Datadog, New Relic, InfluxDB	Standard CNCF pour la collecte de métriques Kubernetes, scraping via ServiceMonitors, exposition de métriques personnalisées au HPA via Prometheus Adapter, dashboards Grafana configurables [38]

Suite à la page suivante

Composant	Technologie retenue	Alternatives considérées	Justification du choix
DNS externe	Cloudflare (API)	Route 53, Gandi, OVH	API REST fiable et documentée, support du challenge DNS-01 ACME pour cert-manager, latence de propagation DNS parmi les plus faibles du marché, tarification gratuite pour la gestion DNS [39]
Fournisseur cloud	Alibaba Cloud ACK	AWS EKS, Google GKE, infrastructure bare-metal	Seul fournisseur hyperscaler disposant d'une infrastructure de datacenter en Algérie, permettant la conformité aux lois 18-07 et 25-11 sur la résidence des données ; service Kubernetes managé certifié CNCF avec Cluster Autoscaler intégré [40]
Registre interne OCI	Distribution (registry :2)	Harbor, Quay.io, GHCR	Registre léger déployable en intra-cluster sans authentification (HTTP interne), éliminant la latence de pull depuis un registre externe lors du déploiement des images construites par Buildkit

Chapitre 3

Réalisation et Implémentation

3.1 Environnement de développement et de déploiement

3.1.1 Infrastructure matérielle

L’infrastructure de déploiement de Hostifer repose entièrement sur le service *Container Service for Kubernetes* (ACK) d’Alibaba Cloud, un service Kubernetes managé certifié conforme au programme *Kubernetes Conformance* du CNCF [40]. Le choix d’ACK est motivé par son intégration native avec l’ensemble des briques réseau, stockage et sécurité d’Alibaba Cloud, ainsi que par la prise en charge managée du plan de contrôle, qui délègue à Alibaba Cloud la responsabilité opérationnelle des composants critiques (`kube-apiserver`, `kube-controller-manager`, `kube-scheduler`, `etcd`) [41].

L’architecture de déploiement s’articule autour d’un réseau privé virtuel (*Virtual Private Cloud*, VPC) unique constituant le périmètre d’isolation réseau de l’ensemble des ressources de la plateforme [42]. Au sein de ce VPC, un *vSwitch* dédié héberge les nœuds du cluster en leur attribuant des adresses IP privées, conformément aux bonnes pratiques de planification réseau ACK préconisant la séparation entre le sous-réseau des nœuds et les blocs CIDR des pods et des services [43].

Cluster ACK et node pools

Le cluster ACK est configuré en mode *managed Pro*, dans lequel Alibaba Cloud héberge et maintient intégralement le plan de contrôle dans son infrastructure méta-cluster (*Kube-on-Kube*), garantissant un SLA de disponibilité du plan de contrôle de 99,95% en configuration multi-zone [41]. Le plan de données est composé de trois node pools distincts, dimensionnés selon les profils de charge de la plateforme :

- **Node pool du plan de contrôle ACK (3 nœuds)** : ce pool est réservé aux composants internes du plan de contrôle gérés par ACK (notamment *CCM*, *etcd* et les services système Kubernetes associés), et n'héberge pas les workloads applicatifs de la plateforme.
- **Node pool des services principaux (3 nœuds)** : trois instances ECS configurées avec 4 vCPU et 12 Go de RAM, dédiées aux fonctionnalités cœur de la plateforme (backend NestJS, Argo Workflows, Loki, Vault, cert-manager, Traefik, registre OCI). Ce dimensionnement offre un compromis entre capacité mémoire et coût pour des composants d'orchestration et d'observabilité exécutés en continu.
- **Node pool des tenants (3 nœuds, extensible)** : trois instances ECS configurées avec 4 vCPU et 16 Go de RAM, dédiées à l'exécution des workloads applicatifs des tenants et des jobs de construction Buildkit. Ce node pool est géré par le Cluster Autoscaler d'ACK, qui ajoute ou supprime des nœuds automatiquement en fonction de l'état des pods *Pending* et du taux d'utilisation des ressources, permettant une élasticité horizontale sans intervention manuelle [40].

Architecture réseau

La topologie réseau de la plateforme repose sur deux *Server Load Balancers* (SLB) [44] et trois *Elastic IP Addresses* (EIP) [45], dont les rôles sont strictement distincts :

SLB d'entrée applicative (ports 80 et 443) : ce premier équilibreur de charge reçoit l'ensemble du trafic HTTP et HTTPS provenant des utilisateurs finaux et des webhooks GitHub. Il distribue le trafic entrant vers le pod Traefik déployé sur les nœuds du cluster, qui assure ensuite le routage vers les services des tenants selon les règles d'IngressRoute. Une EIP entrante dédiée lui est associée, constituant le point d'entrée public unique de la plateforme.

SLB d'accès à l'API Kubernetes (port 6443) : ce second équilibreur de charge expose le serveur API Kubernetes (*kube-apiserver*) pour les connexions *kubectl* et les appels d'administration depuis les outils de déploiement (Helm, Argo Workflows CLI). Une EIP entrante dédiée lui est associée, permettant l'accès sécurisé au plan de contrôle depuis les environnements de développement et de CI/CD.

NAT Gateway et EIP sortante : les nœuds de travail ne disposent d'aucune adresse IP publique directe. Leurs connexions sortantes (clonage de dépôts Git depuis GitHub, téléchargement d'images de base depuis des registres publics) sont acheminées via une passerelle NAT Internet (*Internet NAT Gateway*) à laquelle une troisième EIP sortante est associée [42]. Ce mécanisme de SNAT centralise et masque les origines des requêtes sortantes, réduisant la surface d'exposition publique du cluster.

Infrastructure de stockage

La plateforme mobilise trois types de stockage complémentaires, sélectionnés selon les caractéristiques d'accès et de persistance requises par chaque composant [46] :

NAS (*Network Attached Storage*) : le stockage NAS est utilisé pour les *Persistent-VolumeClaims* (PVC) nécessitant un accès concurrent depuis plusieurs pods, notamment pour le registre OCI interne (stockage des couches d'images) et pour la persistance des données Loki. Le protocole NFS sous-jacent permet le montage simultané depuis plusieurs nœuds (*ReadWriteMany*), ce qui est incompatible avec les disques EBS par nature mono-nœud [47].

Cloud Disks EBS (ESSD) : les volumes EBS de type ESSD (*Enhanced SSD*) sont utilisés pour les workloads nécessitant des accès disque à faible latence et à haute performance I/O, notamment les jobs de construction Buildkitd pour lesquels le cache des couches de build est stocké localement sur des volumes bloc dédiés. Chaque volume EBS est attaché à un nœud unique (*ReadWriteOnce*), ce qui les rend adaptés aux charges à fort débit séquentiel ou aléatoire [46].

Object Storage Service (OSS) : le stockage objet OSS est utilisé pour deux cas d'usage distincts. D'une part, le stockage occasionnel de fichiers artefacts générés par la plateforme (plans de build exportés, archives de logs). D'autre part, le stockage dit *cold* pour les journaux et données d'observabilité archivés à long terme, pour lesquels les contraintes de latence d'accès sont relâchées mais le coût de stockage par gigaoctet doit être minimisé [47].

Le tableau 3.1 récapitule les caractéristiques de l'environnement de déploiement.

TABLEAU 3.1 – Caractéristiques de l'environnement de déploiement

Composant	Paramètre	Valeur
Cluster Kubernetes	Service	Alibaba Cloud ACK (<i>Managed Pro</i>)
	Distribution	Kubernetes 1.31+ (ACK patch x.y.z-aliyun.n)
	Runtime	containerd
Node pool contrôle	Nœuds	3 instances ECS
	CPU	4 vCPU / nœud
	RAM	16 Go / nœud

Suite à la page suivante

Composant	Paramètre	Valeur
	OS	Alibaba Cloud Linux
Node pool travail	Nœuds	3 instances ECS (extensible)
	CPU	4 vCPU / nœud
	RAM	8 Go / nœud
	OS	Alibaba Cloud Linux
	Autoscaling	Cluster Autoscaler ACK (ajout/suppression automatique de nœuds)
Réseau	VPC	1 VPC isolé, 1 vSwitch nœuds
	SLB applicatif	1 SLB — ports 80/443 (trafic HTTP/HTTPS entrant)
	SLB API server	1 SLB — port 6443 (accès kubectl)
	EIP entrante (app)	1 EIP associée au SLB ports 80/443
	EIP entrante (API)	1 EIP associée au SLB port 6443
	EIP sortante	1 EIP associée au NAT Gateway (SNAT worker nodes)
Stockage	NAS (NFS)	PVC multi-nœuds (<i>ReadWriteMany</i>) — registre OCI, Loki
	EBS (ESSD)	Volumes bloc haute performance (<i>ReadWriteOnce</i>) — cache Buildkitd
	OSS	Stockage objet — artefacts occasionnels et archivage <i>cold</i>

3.1.2 Outils de développement

TABLEAU 3.2 – Environnement de développement

Catégorie	Outil / Technologie	Rôle
IDE	Visual Studio Code	Éditeur principal pour le développement frontend, backend et infrastructure
Runtimes	Node.js 24 LTS	Environnement d'exécution JavaScript pour le frontend Next.js et le backend NestJS
	Go (dernière version LTS)	Environnement d'exécution du binaire builder
Frameworks	Next.js	Framework React full-stack pour le frontend de la plateforme
	NestJS	Framework Node.js modulaire pour le backend API REST et SSE
Base de données	PostgreSQL	Système de gestion de base de données relationnelle principal de la plateforme
Gestionnaires de paquets	npm	Gestion des dépendances Node.js (frontend Next.js et backend NestJS)
	Go Modules (gomod)	Gestion des dépendances du binaire builder Go
	Helm	Gestion et déploiement des charts Kubernetes de la plateforme
Outils CLI	kubectl	Administration et inspection des ressources du cluster Kubernetes
	helm	Déploiement, mise à jour et test des charts Helm en local et en production

Suite à la page suivante

Catégorie	Outil / Technologie	Rôle
	<code>argo</code>	Soumission, suivi et débogage des workflows Argo Workflows
	<code>crictl</code>	Inspection bas niveau des conteneurs et images via l'interface CRI
	<code>docker</code>	Construction et test local des images conteneurs avant push vers le registre
	<code>git</code>	Gestion du code source et interaction avec les dépôts GitHub
	<code>vault</code>	Interaction avec HashiCorp Vault pour la gestion et le test des secrets
	<code>railpack</code>	Test local de la détection de framework et de la génération des plans de build
Chaîne CI/CD	GitHub Actions	Intégration continue : lint, build, tests et publication des images OCI sur GHCR
	ArgoCD	Livraison continue : synchronisation GitOps des manifests Kubernetes vers le cluster ACK

3.1.3 Organisation des dépôts GitHub

Le code source de Hostifer est organisé en plusieurs dépôts GitHub distincts, chacun correspondant à un composant logique de la plateforme. Cette séparation permet une gestion indépendante des cycles de release, des pipelines CI/CD et des droits d'accès pour chaque composant. Les artefacts produits par chaque dépôt — images Docker et charts Helm — sont publiés sur le registre *GitHub Container Registry* (GHCR) sous le namespace `ghcr.io/hostifer`, conformément au standard OCI (*Open Container Initiative*) [48].

Dépôt frontend (Next.js). Ce dépôt contient le code source de l'interface utilisateur

de la plateforme, développée avec Next.js. Le pipeline GitHub Actions associé construit et publie l'image Docker du frontend sur GHCR à l'adresse `ghcr.io/hostifer/frontend:<tag>`. Cette image est ensuite déployée sur le cluster ACK via ArgoCD lors de chaque release.

Dépôt backend (NestJS). Ce dépôt contient le code source de l'API backend, développée avec NestJS. Son pipeline CI/CD produit deux images Docker distinctes publiées sur GHCR : l'image principale du backend à l'adresse `ghcr.io/hostifer/backend:<tag>`, et une image de migration de schéma de base de données à l'adresse `ghcr.io/hostifer/migrate:<tag>`. Cette dernière est exécutée en tant que Job Kubernetes avant chaque mise à jour du backend afin d'appliquer les migrations Prisma sans interruption de service.

Dépôt helm (chart principal). Ce dépôt contient le chart Helm de déploiement des composants principaux de la plateforme (frontend, backend, ingress, secrets). Le chart est publié sous forme de package OCI sur GHCR à l'adresse `oci://ghcr.io/hostifer/charts/helm` et est consommé par ArgoCD pour la synchronisation GitOps de l'état du cluster.

Dépôt tenant (chart tenant). Ce dépôt contient le chart Helm dédié au provisionnement des tenants. À chaque déploiement d'application utilisateur, Argo Workflows invoque ce chart publié à l'adresse `oci://ghcr.io/hostifer/charts/tenant` pour créer l'ensemble des ressources Kubernetes du namespace tenant : *Deployment*, *Service*, *IngressRoute*, *NetworkPolicy*, *ResourceQuota*, *HorizontalPodAutoscaler* et *ExternalSecret*.

Dépôt build-job (Go). Ce dépôt contient le code source du binaire Go exécuté en tant que Job Kubernetes lors de l'étape *Build* du pipeline Argo Workflows. Il orchestre le clonage du dépôt Git, l'invocation de Railpack CLI pour la détection du framework et la génération du plan de build, puis la construction et le push de l'image OCI de l'application via Buildkitd. L'image du builder est publiée sur GHCR à l'adresse `ghcr.io/hostifer/build-job:<tag>`.

Dépôt create-dns-job (Go). Ce dépôt contient le code source du Job Go responsable de la création des enregistrements DNS pour le sous-domaine de l'application déployée, lors de l'étape *DNS* du pipeline Argo Workflows. Son image Docker est publiée à l'adresse `ghcr.io/hostifer/create-dns:<tag>`.

Dépôt delete-dns-job (Go). Ce dépôt contient le code source du Job Go déclenché lors de la suppression d'un projet (CU08), chargé de supprimer les enregistrements DNS associés au sous-domaine de l'application. Son image Docker est publiée à l'adresse `ghcr.io/hostifer/delete-dns-job:<tag>`.

Dépôt gitops (ArgoCD). Ce dépôt ne produit aucun artefact compilé. Il contient exclusivement les manifests Kubernetes et les fichiers de valeurs Helm (`values.yaml`)

décrivant l'état cible du cluster. ArgoCD surveille ce dépôt en continu et synchronise l'état réel du cluster ACK vers l'état déclaré dans le dépôt, selon le paradigme GitOps [33].

Dépôt documentation (Mintlify). Ce dépôt contient la documentation technique et utilisateur de la plateforme, générée avec Mintlify. Il ne produit aucun artefact OCI et est déployé indépendamment via le service d'hébergement Mintlify.

Le tableau 3.3 récapitule l'ensemble des dépôts, leurs technologies et les artefacts publiés.

TABLEAU 3.3 – Organisation des dépôts GitHub et artefacts GHCR

Dépôt	Langage	Artefacts publiés (GHCR)
frontend	TypeScript	ghcr.io/hostifer/frontend:<tag>
backend	TypeScript	ghcr.io/hostifer/backend:<tag> ghcr.io/hostifer/migrate:<tag>
helm	Helm	oci://ghcr.io/hostifer/charts/helm
tenant	Helm	oci://ghcr.io/hostifer/charts/tenant
build-job	Go	ghcr.io/hostifer/build-job:<tag>
create-dns-job	Go	ghcr.io/hostifer/create-dns:<tag>
delete-dns-job	Go	ghcr.io/hostifer/delete-dns-job:<tag>
gitops	YAML	— (aucun artefact, manifests ArgoCD uniquement)
documentation	MDX	— (aucun artefact, hébergement Mintlify)

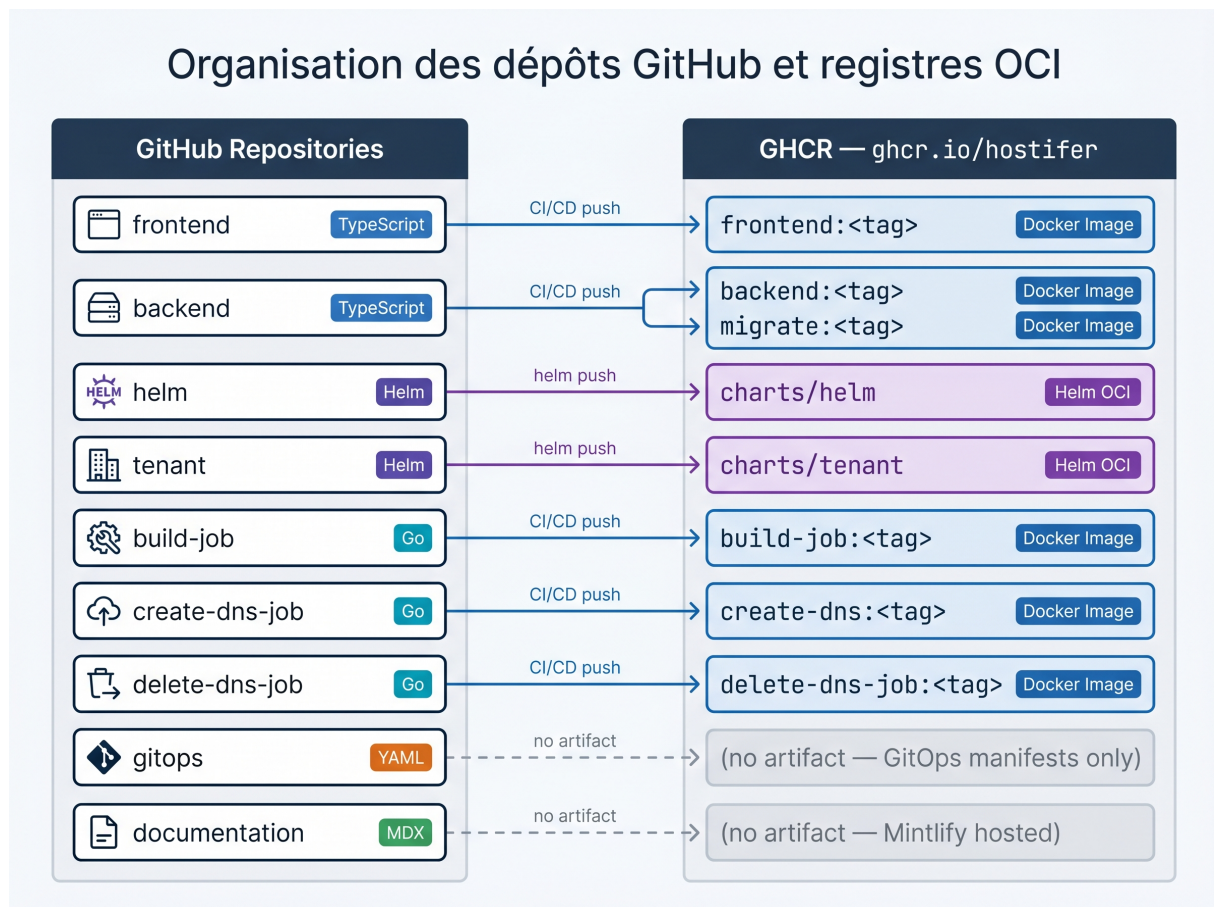


FIGURE 3.1 — Schéma d'organisation des dépôts GitHub et registres OCI

3.2 Implémentation du backend NestJS

3.2.1 Structure modulaire de l'application

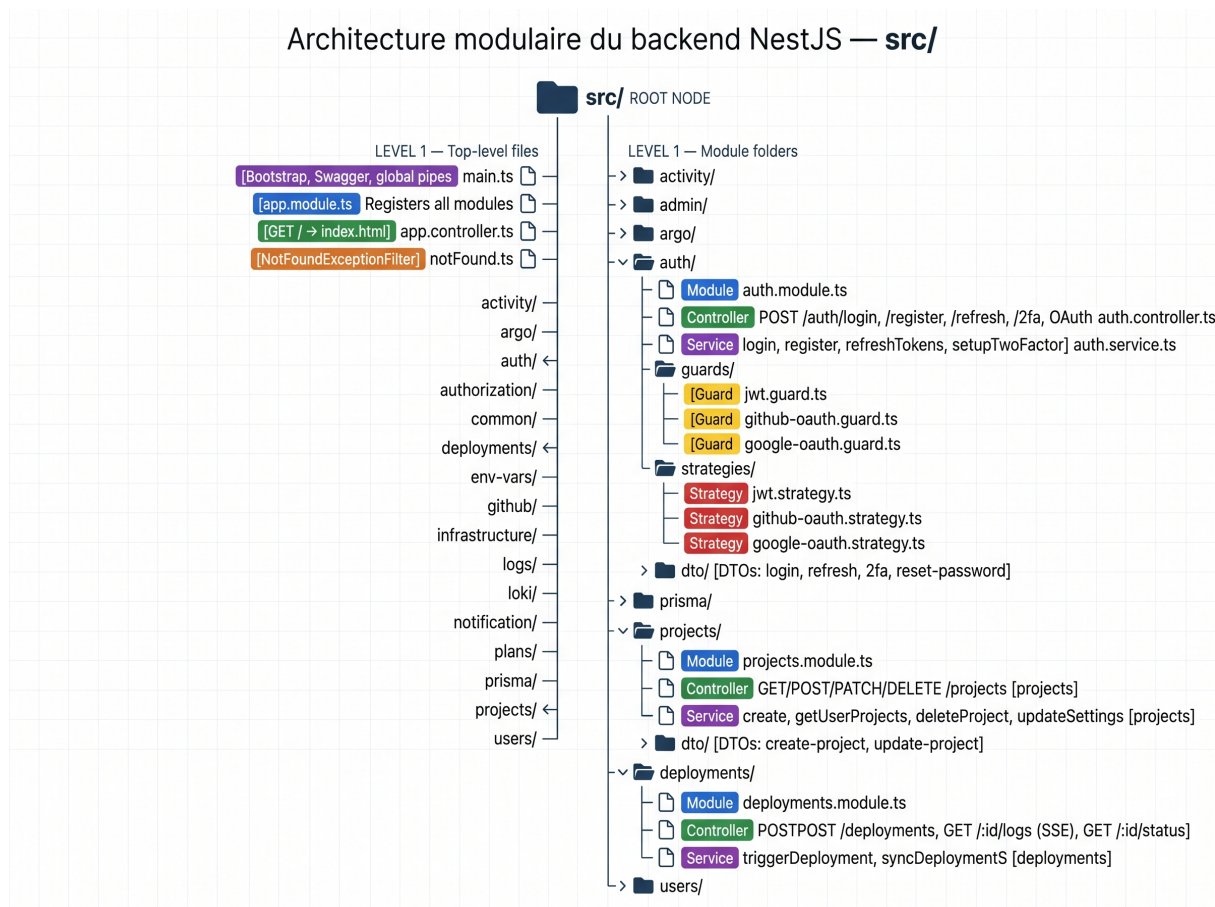


FIGURE 3.2 — Arborescence du projet NestJS avec description de chaque module (Auth, Users, Projects, Deployments, Logs, EnvVars, GitHub, Plans, Infrastructure)

3.2.2 Module d'authentification

Le module d'authentification constitue l'une des couches les plus critiques de la plateforme. Il est implémenté dans le module NestJS `auth/` et repose sur la bibliothèque Passport.js, intégrée à NestJS via le package `@nestjs/passport`, qui expose un mécanisme de stratégies d'authentification interchangeableables et extensibles [27]. Trois mécanismes d'authentification distincts sont implémentés : l'authentification par identifiant et mot de passe avec émission de jetons JWT, l'authentification déléguée via OAuth 2.0 GitHub, et l'authentification à deux facteurs (2FA) basée sur TOTP (*Time-based One-Time Password*).

Authentification JWT et stratégie Passport

L'authentification principale repose sur la stratégie `passport-jwt`, configurée dans `jwt.strategy.ts`. Lors d'une connexion par identifiant et mot de passe, le service `auth.service.ts` vérifie les identifiants soumis contre l'enregistrement User en base de

données PostgreSQL via Prisma, en comparant le mot de passe soumis au hachage bcrypt stocké. En cas de succès, il émet deux jetons distincts via le service `@nestjs/jwt` [49] :

- Un **jeton d'accès** (*access token*) JWT de courte durée de vie, signé avec un secret `JWT_ACCESS_SECRET` dédié. Son payload contient uniquement l'identifiant et l'email de l'utilisateur, conformément au principe de minimisation des données dans le payload JWT [50]. Ce jeton est transmis par le client dans l'en-tête `Authorization: Bearer<token>` à chaque requête vers les endpoints protégés.
- Un **jeton de rafraîchissement** (*refresh token*) de longue durée de vie, signé avec un secret `JWT_REFRESH_SECRET` distinct. Avant persistance, ce jeton est haché en base de données dans le champ `refreshToken` de l'entité `User`, de sorte qu'une fuite de la base de données n'expose jamais de jetons de rafraîchissement en clair [51].

La stratégie JWT est enregistrée comme guard global via `JwtGuard` (`guards/jwt.guard.ts`), qui étend `AuthGuard('jwt')` de Passport. Ce guard est appliqué sur tous les endpoints protégés du backend, garantissant que seules les requêtes portant un jeton d'accès valide et non expiré peuvent atteindre les handlers NestJS.

Rotation des jetons de rafraîchissement

La rotation des jetons de rafraîchissement (*refresh token rotation*) est un mécanisme de sécurité selon lequel chaque utilisation d'un jeton de rafraîchissement entraîne son invalidation immédiate et l'émission d'une nouvelle paire de jetons [52]. Ce mécanisme est implémenté dans la méthode `refreshTokens()` de `auth.service.ts`, invoquée via l'endpoint `POST/auth/refresh`.

Lors d'un appel à cet endpoint, NestJS valide le jeton de rafraîchissement soumis en le comparant au hachage stocké en base de données. Si la vérification réussit, une nouvelle paire jeton d'accès / jeton de rafraîchissement est générée : le nouveau jeton de rafraîchissement est haché et remplace l'ancien en base, tandis que l'ancien est définitivement invalidé. Ce mécanisme garantit que la réutilisation d'un jeton de rafraîchissement volé soit détectable : si un attaquant tente d'utiliser un jeton déjà consommé, la vérification du hachage échoue et la session est révoquée [53].

Délégation OAuth GitHub au backend

Le flux OAuth 2.0 GitHub est entièrement délégué au backend NestJS plutôt que géré directement par NextAuth.js côté frontend. Ce choix architectural garantit que les tokens GitHub ne transitent jamais en clair dans le navigateur et que la logique de création ou de mise à jour du compte utilisateur reste dans la couche backend, où elle peut être

auditée et testée indépendamment. La stratégie OAuth GitHub est implémentée dans `github-oauth.strategy.ts` via `PassportStrategy` et le package `passport-github2`.

Le flux se déroule en deux étapes. Lors de l'initiation, le frontend redirige l'utilisateur vers `GET/auth/github`, qui déclenche le guard `GithubOAuthGuard` (`guards/github-oauth.guard.ts`) et redirige vers le serveur d'autorisation GitHub avec les scopes requis. Après consentement de l'utilisateur, GitHub retourne un code d'autorisation vers l'URL de callback `GET/auth/github/callback`. NestJS échange ce code contre un token d'accès GitHub et invoque la méthode `validateOrCreateOAuthUser()` du service d'authentification, qui crée ou met à jour l'enregistrement `User` en base de données en associant le `githubId` retourné par GitHub au profil. NestJS émet ensuite une paire de jetons JWT propres à la plateforme et les retourne au frontend, de sorte que les appels API ultérieurs du client utilisent exclusivement les JWT Hostifer et non les tokens GitHub.

Rotation des jetons dans le callback `NextAuth.js`

Côté frontend `Next.js`, la gestion des sessions est assurée par `NextAuth.js`, configuré avec un `CredentialsProvider` personnalisé qui délègue la validation des identifiants au backend NestJS. `NextAuth.js` persiste les jetons JWT émis par le backend dans sa session chiffrée côté serveur, évitant toute exposition dans le `localStorage` du navigateur, vulnérable aux attaques XSS [54].

La rotation des jetons est orchestrée dans le `callback jwt` de `NextAuth.js`, qui est invoqué à chaque validation de session [?]. Ce callback implémente la logique suivante : si la date d'expiration du jeton d'accès courant est dépassée, `NextAuth.js` appelle automatiquement l'endpoint `POST/auth/refresh` du backend NestJS en transmettant le jeton de rafraîchissement stocké en session. Si NestJS retourne une nouvelle paire de jetons, le callback met à jour la session avec les nouveaux jetons et la nouvelle date d'expiration, de manière totalement transparente pour l'utilisateur. En cas d'échec du rafraîchissement (jeton révoqué ou expiré), le callback positionne un champ `error: "RefreshAccessTokenError"` dans le token `NextAuth.js`, que le callback `session` propage au client, déclenchant une déconnexion forcée et une redirection vers la page de connexion [55].

Cette architecture à deux couches — rotation au niveau NestJS pour la sécurité de l'infrastructure, et rafraîchissement automatique au niveau `NextAuth.js` pour la transparence de l'expérience utilisateur — garantit des sessions longues sans compromettre la sécurité des jetons.

3.2.3 Module de déploiement et orchestration Argo Workflows

Le module `deployments/` constitue le cœur fonctionnel du backend Hostifer. Il est responsable de l'orchestration complète du cycle de vie des déploiements : soumission des workflows à Argo Workflows, synchronisation périodique de leur statut, et mise à jour transactionnelle des enregistrements `Deployment` et `DeploymentStep` en base de données PostgreSQL. Il expose également l'endpoint SSE de streaming des journaux de build, décrit dans la section suivante.

Soumission des workflows via l'API REST Argo

Argo Workflows expose un serveur API REST dédié (`argo-server`), distinct de l'API Kubernetes, qui offre des endpoints pour la soumission, l'interrogation et l'annulation des workflows [30]. La soumission d'un workflow depuis NestJS n'est donc pas effectuée via le client Kubernetes, mais via une requête HTTP POST vers l'endpoint `/api/v1/workflows/{namespace}/submit` de l'API Argo, en transmettant un corps JSON indiquant le *WorkflowTemplate* à instancier ainsi que les paramètres dynamiques du déploiement [56].

La méthode `submitDeployWorkflow(payload)` du service `argo.service.ts` construit ce corps de requête en injectant les paramètres propres à chaque déploiement : identifiant du projet, tag de l'image à construire, namespace Kubernetes du tenant, référence au dépôt Git et branche cible. L'authentification auprès de l'API Argo est assurée par un jeton de service (*ServiceAccount token*) transmis dans l'en-tête `Authorization: Bearer`. En retour, l'API Argo retourne le manifeste du *Workflow* instancié, dont le champ `metadata.name` est extrait et stocké dans le champ `argoWorkflowName` de l'enregistrement `Deployment` en base de données. Ce nom est utilisé comme clé d'identification pour toutes les interrogations de statut ultérieures.

De manière symétrique, la méthode `submitDeleteWorkflow(payload)` soumet un *WorkflowTemplate* de suppression lors de la demande de déprovisionnement d'un projet (CU08), orchestrant la suppression de la release Helm, du namespace Kubernetes et des enregistrements DNS via les jobs Go dédiés.

Synchronisation du statut par polling

Argo Workflows ne propose pas de mécanisme de callback ou de webhook natif permettant à un système externe d'être notifié de l'évolution du statut d'un workflow. Le statut est exposé via l'endpoint `GET/api/v1/workflows/{namespace}/{name}` qui retourne l'objet *Workflow* complet, dont le champ `status.phase` reflète l'état courant : `Pending`, `Running`, `Succeeded`, `Failed` ou `Error` [30]. Le champ `status.nodes`

contient quant à lui l'état détaillé de chaque nœud du workflow, correspondant à une étape individuelle du pipeline.

La synchronisation du statut est implémentée dans la méthode `syncDeploymentStatus(deploymentId)` de `deployments.service.ts`, selon un mécanisme de *polling* périodique. Cette méthode est invoquée régulièrement par le service NestJS après la soumission d'un workflow. À chaque appel, elle interroge l'API Argo via la méthode `getWorkflowStatus(workflowName)` du service `argo.service.ts`, parse la réponse JSON retournée, et en extrait les informations de statut nécessaires à la mise à jour de la base de données.

La correspondance entre les phases Argo et les statuts internes de Hostifer est définie comme suit. Tant que le workflow est en phase `Running`, le service examine l'état des nœuds individuels (`status.nodes`) pour déterminer quelle étape du pipeline est en cours d'exécution (*Build*, *Provision* ou *DNS*) et met à jour le statut du `Deployment` en conséquence (`BUILDING`, `PROVISIONING` ou `CONFIGURING_DNS`). Lorsque le workflow atteint la phase `Succeeded`, le statut est mis à jour à `LIVE` et le timestamp `finishedAt` est enregistré. En cas de phase `Failed` ou `Error`, le statut passe à `FAILED` et le message d'erreur est extrait via la méthode privée `extractWorkflowFailureMessage()`, qui parcourt les nœuds du workflow à la recherche du premier nœud en état d'échec et récupère son champ `message`. En cas d'annulation explicite, le statut passe à `CANCELLED`.

Mise à jour des enregistrements `Deployment` et `DeploymentStep`

Chaque appel à `syncDeploymentStatus()` se conclut par une mise à jour transactionnelle des enregistrements Prisma correspondant au déploiement. Deux entités sont mises à jour simultanément lors de chaque cycle de synchronisation.

L'entité `Deployment` est mise à jour avec le nouveau statut, le timestamp `updatedAt`, et — lorsque le déploiement atteint un état terminal — les champs `finishedAt` et `totalDuration` calculés comme la différence en secondes entre `finishedAt` et `createdAt`.

L'entité `DeploymentStep` représente l'état individuel de chacune des trois étapes du pipeline (*Build*, *Provision*, *DNS*). Pour chaque étape, un enregistrement `DeploymentStep` est créé ou mis à jour avec le statut du nœud Argo correspondant, son heure de démarrage (`startedAt`), son heure de fin (`finishedAt`), et un message d'erreur optionnel. Cette granularité permet à l'interface utilisateur d'afficher une progression détaillée par étape, et non uniquement le statut global du déploiement.

Le flux complet de déclenchement d'un déploiement, depuis la réception de la requête `POST/deployments` jusqu'au passage en état `LIVE`, est illustré dans la Figure 2.5 (diagramme de séquence du déploiement).

TABEAU 3.4 – Correspondance entre les phases Argo Workflows et les statuts internes Hostifer

Phase Argo	Nœud actif	Statut Hostifer (DeploymentStatus)
Pending	—	QUEUED
Running	build	BUILDING
Running	provision	PROVISIONING
Running	dns	CONFIGURING_DNS
Succeeded	—	LIVE
Failed	—	FAILED
Error	—	FAILED
(annulé)	—	CANCELLED

3.2.4 Module de streaming des logs

Le streaming des journaux de build en temps réel constitue une fonctionnalité centrale de l'expérience utilisateur de Hostifer. Elle repose sur une architecture à deux composants distincts : l'endpoint SSE (*Server-Sent Events*) exposé par le contrôleur NestJS, et le générateur asynchrone de journaux Loki implémenté dans le service `loki.service.ts`. La séparation entre ces deux couches respecte le principe de responsabilité unique : le contrôleur gère le protocole de transport SSE, tandis que le service Loki encapsule toute la logique d'interrogation et de pagination des journaux.

Endpoint SSE avec le décorateur `@Sse()`

NestJS fournit un support natif du protocole SSE via le décorateur `@Sse()` du package `@nestjs/common` [57]. Ce protocole, standardisé par le W3C et nativement supporté par les navigateurs via l'API `EventSource`, permet au serveur de pousser des messages vers le client sur une connexion HTTP persistante unidirectionnelle, sans les surcoûts protocolaires des WebSockets [58]. Il est particulièrement adapté au cas d'usage du streaming de journaux, qui est par nature unidirectionnel (serveur $\rightarrow$ client) et ne nécessite pas de communication bidirectionnelle.

L'endpoint de streaming est déclaré dans `deployments.controller.ts` à la route `GET/deployments/:id/logs`, protégé par le `JwtGuard`. Le handler retourne un `Observable<MessageEvent>` de RxJS, que NestJS sérialise automatiquement en événements

SSE conformes à la spécification `text/event-stream` [57]. Chaque événement transmis au client contient un champ `data` encapsulant un objet `LogLine` sérialisé en JSON, comprenant le contenu de la ligne de journal, son timestamp nanoseconde, et les labels Loki associés.

La conversion du générateur asynchrone retourné par `loki.service.ts` en `Observable` RxJS est réalisée via l'opérateur `from()` de RxJS, qui accepte nativement les itérables asynchrones. Le flux SSE est clos automatiquement lorsque le générateur asynchrone se termine, c'est-à-dire lorsque le déploiement atteint un état terminal ou lorsque Loki ne retourne plus de nouvelles entrées pour l'identifiant de build concerné.

Connexion WebSocket et endpoint Loki /tail

La collecte des journaux depuis Loki est implémentée dans la méthode `streamDeploymentLogs(deploymentId, options)` du service `loki.service.ts`, sous la forme d'un générateur asynchrone (`asyncfunction*`) qui produit des objets `LogLine` au fur et à mesure de leur disponibilité.

Loki expose un endpoint WebSocket `GET/loki/api/v1/tail` conçu pour le streaming de logs en temps réel, en contraste avec l'endpoint HTTP `GET/loki/api/v1/query_range` utilisé pour les requêtes temporelles historiques. L'endpoint `/tail` accepte une requête `LogQL` et maintient une connexion WebSocket persistante, diffusant chaque nouvelle ligne de journal au fur et à mesure qu'elle est ingérée par Loki. Cette approche élimine les délais de polling et réduit significativement la surcharge réseau par rapport à des requêtes HTTP répétées.

NestJS établit et gère une connexion WebSocket unique par flux de journaux (identifié par un `streamKey` dérivé de l'identifiant du déploiement). Cette gestion centralisée est assurée par le service `LokiTailPool`, qui implémente un pattern de pooling et de fan-out : plusieurs clients (*subscribers*) abonnés au même flux partagent une unique connexion WebSocket vers Loki, réduisant ainsi la consommation de ressources. Chaque *subscriber* maintient sa propre file d'attente d'événements (*queue*) de sorte qu'un client lent ne bloque pas les autres.

La connexion WebSocket est établie paresseusement, c'est-à-dire uniquement lors du premier abonnement à un flux donné. Les frames de données reçus depuis Loki sont parsés et normalisés en objets `LogLine`, puis distribués de manière asynchrone à chaque subscriber via sa file de réception. En cas de déconnexion réseau, le service implémente une stratégie de reconnexion avec backoff exponentiel, garantissant la continuité du flux en cas de défaillance transitoire.

Requête LogQL et stratégie de filtrage

LogQL, le langage de requête de Loki, structure toute requête autour d'un sélecteur de flux de journaux (*log stream selector*) obligatoire, suivi d'un pipeline de traitement optionnel. Le sélecteur de flux est une chaîne de paires clé-valeur délimitée par des accolades, dont la combinaison unique identifie un flux de journaux.

La requête LogQL construite par `loki.service.ts` pour chaque déploiement est de la forme :

```
{build_id="<deployment_id>", container="main"}
```

Le label `build_id` est un label personnalisé attaché par Promtail lors de la collecte des journaux du pod Builder Job, via une règle de *relabeling* dans la configuration du DaemonSet Promtail. Il permet d'isoler précisément les journaux d'un déploiement donné parmi l'ensemble des logs du cluster [59]. Le label `container="main"` restreint la sélection au conteneur principal du pod, excluant les conteneurs système injectés par Argo Workflows (conteneur `wait`, conteneur `init`) dont les journaux sont de nature infrastructurelle et sans intérêt pour l'utilisateur final.

Cette stratégie de filtrage par labels indexés est préférable à un filtrage par pipeline (`!=`) appliqué après sélection d'un flux large, car l'application des filtres sur les labels indexés en amont du pipeline de traitement réduit significativement le volume de données à parcourir par Loki. En ciblant directement le flux `\{build_id,container\}`, la requête évite que Loki ne charge et ne filtre l'intégralité des journaux du namespace, ce qui serait coûteux dans un environnement multi-tenant à charge élevée.

Les objets `LogLine` normalisés produits par le service Loki contiennent le contenu de la ligne de journal, son timestamp en nanosecondes, et les labels Loki associés. Le générateur `yield` ces objets un par un vers le contrôleur, lequel les diffuse au navigateur via la connexion SSE.

Gestion du cycle de vie de la session WebSocket

La session WebSocket gérée par `LokiTailPool` termine son cycle de vie selon deux conditions distinctes. Premièrement, lorsque le statut du déploiement en base de données atteint un état terminal (`LIVE`, `FAILED` ou `CANCELLED`), indiquant que le pipeline Argo Workflows a conclu son exécution et qu'aucune nouvelle ligne de journal ne sera produite ; dans ce cas, le service interroge périodiquement la base de données et détecte cette transition pour fermer proprement la connexion. Deuxièmement, après une période d'inactivité détectée via le service `IntervalTicker`, qui contrôle les pauses prolongées entre les émissions de logs et émet un signal de heartbeat pour confirmer que le flux est toujours actif ;

lors d'un silence prolongé sans détection de changement de statut, le générateur interprète cela comme la fin effective de la construction et termine.

Dans les deux cas, la terminaison du générateur asynchrone provoque la complétion de l'`Observable` `RxJS` côté contrôleur, ce qui entraîne automatiquement la fermeture propre de la connexion SSE. Côté backend, lorsque le dernier subscriber se désabonne (ou que tous les clients fermés leur connexion SSE), la session `WebSocket` `LokiTailSession` est supprimée du pool et la connexion `WebSocket` vers Loki est fermée, libérant les ressources associées.

3.2.5 Module de gestion des variables d'environnement avec Vault

3.2.6 Versioning et rollback des déploiements

3.3 Implémentation du builder Go

3.3.1 Structure du binaire

Le binaire `hostifer-builder` est organisé en cinq fichiers Go à responsabilités strictement séparées, complétés par les manifests de configuration et d'exécution Kubernetes. Cette organisation minimale reflète la philosophie du projet : un seul chemin d'exécution, sans framework, sans injection de dépendances, et sans état partagé entre les composants.

`main.go` constitue le point d'entrée du programme et l'unique orchestrateur du pipeline de build. Il commence par valider l'ensemble des variables d'environnement requises via la fonction `requireEnv()`, qui retourne une erreur immédiate si l'une d'elles est absente, évitant tout démarrage dans un état indéfini. Il instancie ensuite le logger structuré, applique un contexte Go avec un *timeout* global de vingt minutes sur l'ensemble du pipeline — au-delà duquel le Job est annulé et retourne un code de sortie non nul — puis enchaîne séquentiellement les trois étapes : clonage, préparation Railpack, et build Buildkit. Le programme attend six variables d'environnement à l'exécution : `REPO_URL` (URL du dépôt à cloner), `IMAGE_NAME` (référence complète de l'image OCI à produire), `BUILDKIT_HOST` (adresse gRPC du daemon Buildkitd), `BUILD_ID` et `TENANT_ID` (identifiants utilisés pour la corrélation des journaux dans Loki), et optionnellement `WORK_DIR` (répertoire de travail, `/tmp/build` par défaut).

`git.go` encapsule la logique de clonage du dépôt source. La fonction `CloneRepo(ctx, repoURL, destDir, log)` supprime d'abord tout répertoire de travail existant, garantissant que les tentatives de relance du job démarrent depuis un état propre, puis exécute un clonage superficiel (`git clone --depth 1`) pour ne récupérer que le dernier commit de la

branche cible. Cette approche réduit significativement le volume de données transférées pour les dépôts à historique long [60].

`railpack.go` est le fichier le plus volumineux du projet. Il prend en charge l'intégralité de l'interaction avec Railpack CLI : exécution de `railpackprepare`, gestion des tentatives en cas d'échec transitoire via `GenerateBuildWithRetry()`, et post-traitement du plan généré via `patchRailpackPlanRunAsUser()`. Ce fichier expose également les fonctions utilitaires `injectNodeMemoryFlag()` et `nodeMemoryLimitMb()`, décrites en détail dans la section suivante.

`buildkit.go` gère la connexion au daemon Buildkitd et l'exécution du build OCI. La fonction principale `BuildAndPush(ctx, buildkitHost, contextDir, imageName, log)` établit une connexion gRPC vers le daemon Buildkitd à l'adresse fournie par `BUILDKIT_HOST`, charge les credentials Docker depuis la configuration runtime du conteneur, soumet le contexte de build au frontend Railpack via l'API LLB de Buildkit, et pousse l'image résultante vers le registre interne de la plateforme [5]. La progression de la construction est capturée ligne par ligne par le type `buildkitProgressLogWriter` et émise vers le logger Zap, ce qui permet à Promtail de la collecter et de la transmettre à Loki pour le streaming SSE.

`logger.go` instancie un logger Zap en mode production via la fonction `NewLogger(buildID, tenantIDstring)`. Ce logger attache systématiquement les champs `build_id` et `tenant_id` à chaque entrée de journal émise, sous forme de JSON structuré sur `stdout`. Cette structuration est indispensable pour que les règles de *relabeling* Promtail puissent extraire ces valeurs comme labels Loki indexables, permettant le filtrage précis par déploiement décrit dans la section 3.2.4 [61].

L'image Docker du builder est construite en deux étapes (*multi-stage build*) : une première étape compile le binaire Go dans une image builder Go officielle, et une seconde étape produit l'image runtime minimaliste contenant uniquement le binaire compilé, `git`, `curl`, les certificats CA, et le binaire `railpack` installé dans `/usr/local/bin`. Une étape de préchauffage (*warmup*) Railpack est exécutée à la fin de la construction de l'image pour s'assurer que le binaire est opérationnel dès le premier démarrage du Job, sans délai d'initialisation.

Le tableau 3.5 récapitule la structure du projet builder et le rôle de chaque fichier.

TABLEAU 3.5 – Structure du projet builder Go

Fichier	Type	Rôle principal
<code>main.go</code>	Point d'entrée	Validation des variables d'environnement, timeout global 20 min, orchestration du pipeline
<code>git.go</code>	Module	Clonage superficiel du dépôt Git (<code>--depth 1</code>), nettoyage du répertoire de travail
<code>railpack.go</code>	Module	Exécution de <code>railpackprepare</code> , gestion des tentatives (max. 3), patch du plan généré
<code>buildkit.go</code>	Module	Connexion gRPC à Buildkitd, build LLB via frontend Railpack, push vers le registre OCI
<code>logger.go</code>	Utilitaire	Initialisation du logger Zap avec champs <code>build_id</code> et <code>tenant_id</code>
Dockerfile	Image runtime	Build multi-stage : compilation Go + image runtime avec <code>git</code> , <code>railpack</code> , <code>curl</code>
<code>go.mod</code>	Dépendances	Dépendances : <code>moby/buildkit</code> , <code>docker/cli</code> , <code>uber/zap</code> , <code>grpc</code>

3.3.2 Détection de framework

La détection automatique du framework applicatif est entièrement déléguée à Railpack, un outil open source développé par Railway et successeur de Nixpacks [16]. La prise en charge des langages et frameworks dans Railpack est assurée par un système de *providers*, chacun associé à un langage spécifique (Node.js, Python, PHP, Go, Ruby, Java, etc.). Chaque provider analyse le dépôt source et détermine s'il correspond à son domaine en inspectant les fichiers présents à la racine du projet.

Le mécanisme de détection de Railpack repose sur l'analyse statique du système de fichiers du dépôt cloné, sans exécution de code applicatif. Railpack détecte automatiquement le langage (Node.js, Python, Go, PHP, Java, Ruby, etc.), le framework (Next.js,

Laravel, Django, Spring Boot, etc.) et les dépendances via les fichiers de manifeste propres à chaque écosystème (`package.json`, `pyproject.toml`, `go.mod`, etc.).

Dans le builder Hostifer, la détection est déclenchée par l’invocation de la commande `railpackprepare<srcDir>` depuis la fonction `GenerateBuild()` de `railpack.go`. Cette commande produit deux fichiers dans le répertoire source : `railpack-plan.json`, qui contient le plan de build complet sérialisé en JSON, et `railpack-info.json`, qui contient les métadonnées de détection incluant le langage et le framework identifiés. Le plan de build est un objet JSON contenant l’ensemble des informations nécessaires à la génération de l’image, notamment les étapes de build (*steps*), les commandes à exécuter, les dépendances entre étapes et les instructions de mise en cache.

La commande `railpackprepare` est exécutée dans la fonction `runRailpack()`, qui lance le binaire Railpack comme sous-processus Go via `exec.CommandContext`, capture sa sortie standard et son flux d’erreur, et normalise le résultat en lignes de journal émises vers le logger Zap. Si Railpack retourne un code de sortie non nul, la fonction retourne une erreur Go qui déclenche le mécanisme de tentatives implémenté dans `GenerateBuildWithRetry()`. Ce mécanisme relance l’intégralité de la phase de préparation jusqu’à trois fois (`maxAttempts=3`), en nettoyant les fichiers de sortie partiels entre chaque tentative, pour absorber les défaillances transitoires liées à la résolution de dépendances réseau ou à des conditions de concurrence dans l’environnement Kubernetes [16].

Un cas particulier est géré pour la compatibilité avec les versions antérieures de Railpack : si la première invocation échoue parce que le flag `--hide-pretty-plan` n’est pas supporté par la version installée dans l’image, `runRailpack()` relance la commande sans ce flag. Cette logique de *fallback* garantit la compatibilité du builder avec différentes versions du binaire Railpack sans nécessiter de mise à jour de l’image.

Le résultat de la détection — le framework identifié — est extrait du fichier `railpack-info.json` par le backend NestJS après la fin du workflow Argo, et persisté dans le champ `framework` de l’entité `Project` en base de données PostgreSQL. Cette information est ensuite affichée dans l’interface utilisateur sur la page de détail du projet, offrant à l’utilisateur une confirmation visuelle que son framework a bien été reconnu automatiquement.

3.3.3 Génération du plan Railpack

La génération du plan de build constitue la phase la plus déterminante du pipeline de construction, car elle produit la spécification complète de l’image OCI à construire. Elle se déroule en deux temps : l’invocation de Railpack CLI pour la détection et la génération du plan brut, suivie d’un post-traitement du plan JSON par le builder Go pour y appliquer

des contraintes de sécurité et d'optimisation mémoire.

Invocation de `railpackprepare`

La génération du plan est déclenchée par la méthode `GenerateBuild(ctx, srcDir, log)` de `railpack.go`, qui invoque la commande `railpackprepare<srcDir>` via `exec.CommandContext`. Cette commande produit deux fichiers en sortie : `railpack-plan.json`, qui contient le plan de build sérialisé, et un fichier d'information `railpack-info.json`. Le plan de build est un objet JSON contenant l'ensemble des informations nécessaires à la génération de l'image, notamment les étapes (*steps*), les commandes à exécuter séquentiellement au sein de chaque étape, les dépendances entre étapes, et les directives de mise en cache permettant à Buildkit d'exécuter les étapes indépendantes en parallèle.

La version du binaire Railpack à utiliser est contrôlable via la variable d'environnement `RAILPACK_VERSION`. La référence de l'image frontend Railpack à transmettre à Buildkit est construite par la fonction `railpackFrontendImage()` à partir de cette version, garantissant la cohérence entre la version CLI ayant généré le plan et la version de l'image frontend interprétant ce même plan lors de la construction.

L'ensemble de la phase de préparation est encapsulée dans `GenerateBuildWithRetry(ctx, srcDir, log, maxAttempts)`, qui relance l'intégralité du processus en cas d'échec, jusqu'à un maximum de trois tentatives. Entre chaque tentative, les fichiers de sortie partiels (`railpack-plan.json`, `railpack-info.json`) sont supprimés afin d'éviter que Buildkit consomme un plan corrompu ou incomplet lors d'une tentative ultérieure.

Patch du plan JSON pour UID 1000

À l'issue de la génération réussie du plan, la fonction `patchRailpackPlanRunAsUser(srcDir, log)` lit, modifie et réécrit le fichier `railpack-plan.json`. Ce patch poursuit trois objectifs de sécurité et de compatibilité.

Premièrement, le plan est modifié pour forcer l'exécution du processus applicatif sous l'UID 1000 et le GID 1000. Par défaut, Railpack peut générer des images dont le processus s'exécute en tant que `root` (UID 0), ce qui constitue une mauvaise pratique de sécurité dans un environnement multi-tenant et est incompatible avec les *PodSecurityPolicies* restrictives définies par Kubernetes sur certains clusters [62]. L'exécution sous un UID non privilégié réduit la surface d'attaque en cas de faille applicative permettant une évvasion de conteneur.

Deuxièmement, une étape `chown-R1000:1000/app` est injectée dans le plan lorsque la structure de celui-ci le permet, afin de transférer la propriété du répertoire applicatif

à l'UID cible avant le démarrage du processus. Cette étape est nécessaire car les outils de build (gestionnaires de paquets, compilateurs) opèrent généralement en tant que `root` pendant la phase de construction ; sans ce transfert de propriété, le processus en UID 1000 ne pourrait pas écrire dans son propre répertoire de travail à l'exécution.

Troisièmement, les commandes de démarrage (*start commands*) du plan sont inspectées pour détecter les invocations de `node`, `npm` ou `yarn`. Si une telle invocation est détectée, le flag `NODE_OPTIONS` est injecté via la fonction `injectNodeMemoryFlag()`, décrite ci-après.

Injection de `NODE_OPTIONS` pour la gestion mémoire

L'injection de la limite de tas mémoire Node.js répond à un problème spécifique aux environnements conteneurisés sous Kubernetes. Sans configuration explicite, le moteur JavaScript V8 de Node.js ne connaît pas les limites mémoire du conteneur et ne déclenche pas le garbage collector de manière agressive lorsque la consommation approche la limite du conteneur, ce qui provoque un crash OOM (*Out-of-Memory Kill*) du conteneur par le noyau Linux avant que V8 n'ait pu libérer de la mémoire.

La solution implémentée dans `railpack.go` consiste à injecter la variable d'environnement `NODE_OPTIONS` avec la valeur `--max-old-space-size=<limitMb>` dans la commande de démarrage du plan Railpack. L'utilisation de `NODE_OPTIONS` est préférée à l'injection directe du flag dans la commande `node`, car elle est prise en compte par tous les processus Node.js enfants créés dans l'environnement, y compris ceux démarrés par `npm` ou `yarn`.

La valeur de la limite en mégaoctets est déterminée par la fonction `nodeMemoryLimitMb()`, qui lit la variable d'environnement `MEMORY_LIMIT_MB` exposée au conteneur par le manifest Kubernetes du Job via le mécanisme `resourceFieldRef` de Kubernetes. Cette approche rend la limite dynamique et proportionnelle aux ressources allouées au Job par le plan tarifaire du tenant, sans hard-coding d'une valeur fixe dans l'image. Node.js depuis la version 12 est dit *container-aware* et peut utiliser les limites cgroups du conteneur, mais l'injection explicite de `--max-old-space-size` reste recommandée pour découpler précisément la limite du tas V8 de la limite mémoire totale du conteneur. En l'absence de la variable `MEMORY_LIMIT_MB`, la fonction retourne une valeur par défaut conservative afin de garantir un comportement prévisible même sans configuration explicite.

```
1 {
2   "$schema": "https://schema.railpack.com",
3
4   "steps": [
5     {
6       "name": "packages:mise",
7       "commands": [
8         { "cmd": "mise install",
9           "customName": "install mise packages: bun, node" }
10      ],
11      "assets": {
12        "mise.toml": "[tools]\n  bun = \"1.3.14\"\n  node = \"22.22.3\""
13      }
14    },
15    {
16      "name": "install",
17      "commands": [
18        { "dest": "bun.lock", "src": "bun.lock" },
19        { "dest": "package.json", "src": "package.json" },
20        { "cmd": "bun install --frozen-lockfile" }
21      ],
22      "caches": ["bun-install"]
23    },
24    {
25      "name": "build",
26      "commands": [
27        { "cmd": "bun run build" },
28        {
29          "cmd": "chown -R 1000:1000 /app",
30          "customName": "set ownership to uid 1000"
31        }
32      ],
33      "variables": { "NEXT_TELEMETRY_DISABLED": "1" },
34      "secrets": ["*"],
35      "caches": ["node-modules", "next"]
36    }
37  ],
```

Listing 1 – Extrait du plan railpack-plan.json – Partie 1 : etapes de build

```

46  "deploy": {
47    "startCommand": "bun run start",
48    "user": "1000",
49    "variables": {
50      "NODE_ENV": "production",
51      "CI": "true"
52    },
53    "inputs": [
54      { "step": "packages:mise",
55        "include": ["/mise/shims", "/mise/installs"] },
56      { "step": "build",
57        "include": ["/app/node_modules"] },
58      { "step": "build",
59        "include": ["."],
60        "exclude": ["node_modules", ".yarn"] }
61    ]
62  },
63
64  "caches": {
65    "next": { "directory": "/app/.next/cache",
66             "type": "shared" },
67    "bun-install": { "directory": "/root/.bun/install/cache",
68                    "type": "shared" },
69    "node-modules": { "directory": "/app/node_modules/.cache",
70                     "type": "shared" }
71  }
72 }

```

Listing 2 – Extrait du plan `railpack-plan.json` – Partie 2 : configuration de déploiement et caches

3.3.4 Construction et push de l'image via Buildkit

La construction de l'image OCI et son push vers le registre interne constituent la dernière étape du pipeline de build. Elle est entièrement implémentée dans `buildkit.go` via le client Go officiel de Buildkit, exposé par le package `github.com/moby/buildkit/client` [5].

Connexion au daemon Buildkitd

Buildkit est déployé sur le cluster Kubernetes en tant que *Deployment* avec un pod Buildkitd persistant, accessible depuis les autres pods du cluster via une adresse de service

Kubernetes. Le daemon Buildkitd expose son API gRPC sur un socket TCP, permettant aux clients distants de lui soumettre des requêtes de build via une connexion réseau standard.

La fonction `BuildAndPush(ctx, buildkitHost, contextDir, imageName, log)` établit la connexion au daemon via `client.New(ctx, buildkitHost)`, où `buildkitHost` est la valeur de la variable d'environnement `BUILDKIT_HOST`. Cette variable est injectée dans le Job par le ConfigMap Kubernetes `build-configmap`, qui contient l'adresse du service Buildkitd au format `tcp://<service>:<port>`. La connexion est établie de manière asynchrone avec gestion des tentatives de reconnexion intégrée au client Buildkit.

Les credentials Docker nécessaires pour pousser l'image vers le registre interne sont chargés depuis la configuration Docker runtime du conteneur via le package `github.com/docker/cli/config`, puis transmis à Buildkit sous forme de `session.AuthProvider` attaché à la requête de build. Ce mécanisme de délégation d'authentification évite que les credentials ne soient exposés dans les logs ou dans les variables d'environnement du Job.

Résolution du frontend Railpack via gateway.v0

Buildkit distingue deux types de frontends pour l'interprétation des définitions de build [63]. Le frontend `dockerfile.v0` est le frontend officiel pour les Dockerfiles, tandis que le frontend `gateway.v0` est un frontend générique permettant d'utiliser n'importe quelle image OCI comme interpréteur de build, en lui déléguant la conversion de la définition de build en LLB (*Low-Level Build*).

Le builder Hostifer utilise le frontend `gateway.v0` en lui fournissant l'image frontend Railpack référencée par `railpackFrontendImage()` comme source d'interprétation. Buildkit télécharge cette image depuis GHCR si elle n'est pas déjà présente dans son cache local, l'instancie, et lui transmet le répertoire source du dépôt cloné ainsi que le fichier `railpack-plan.json` patché. Le frontend Railpack interprète ce plan et génère le graphe de dépendances LLB correspondant, que Buildkit exécute pour produire l'image OCI finale. LLB (*Low-Level Build*) est le format intermédiaire binaire de Buildkit, défini comme un protocol buffer, qui représente le graphe de dépendances des opérations de build sous forme d'un DAG (*Directed Acyclic Graph*) permettant l'exécution parallèle des étapes indépendantes et la réutilisation du cache de couches.

L'image résultante est poussée directement vers le registre interne OCI de la plateforme via l'option d'export `type=image,name=<imageName>,push=true,registry.insecure=true`. Le flag `registry.insecure=true` est nécessaire car le registre interne de la plateforme est exposé en HTTP sans TLS au sein du cluster, selon le modèle de registre interne non authentifié courant dans les environnements Kubernetes isolés [5].

Streaming des journaux vertex par vertex

Le suivi de la progression de la construction est assuré par le type `buildkitProgressLogWriter`, implémenté dans `buildkit.go`. Ce type implémente l'interface `io.Writer` et est passé à la fonction de solve Buildkit comme récepteur de la progression. Buildkit émet sa progression sous la forme d'une séquence de *vertex* — chaque vertex correspondant à une étape atomique du graphe LLB (installation d'un package, exécution d'une commande, copie de fichiers, etc.) — accompagnés de leurs logs de sortie standard.

La méthode `Write(p[]byte)` reçoit ces données de progression brutes, les accumule dans un buffer interne, et émet chaque ligne complète (délimitée par `\n`) vers le logger Zap via `log.Info()`. La méthode `Flush()` est appelée à la fin du build pour s'assurer qu'aucune ligne partielle ne reste dans le buffer. La fonction utilitaire `stripTrailingCR()` normalise les fins de ligne en supprimant les caractères `\r` résiduels produits par certains outils de build qui utilisent le retour chariot pour l'affichage de barres de progression en terminal.

Chaque ligne émise par le logger Zap est collectée par Promtail, qui l'indexe dans Loki avec les labels `build_id` et `tenant_id` attachés par le logger. Ces journaux deviennent ainsi immédiatement disponibles pour le streaming SSE décrit dans la section 3.2.4, offrant à l'utilisateur une visibilité en temps réel sur chaque étape de construction de son image OCI.

3.4 Implémentation de l'infrastructure Kubernetes

L'infrastructure Kubernetes de Hostifer est entièrement décrite sous forme déclarative et provisionnée de manière automatisée. Chaque composant — qu'il s'agisse de l'isolation d'un tenant, du pipeline d'orchestration Argo Workflows, de la collecte des journaux ou de la gestion des certificats TLS — est défini par des manifests versionnés et déployés via GitOps. Cette section décrit l'implémentation concrète de chacun de ces composants.

3.4.1 Chart Helm multi-tenant

Le chart Helm `tenant-chart` est la pièce centrale du modèle de provisionnement de Hostifer. Chaque déploiement d'application utilisateur déclenche l'invocation de ce chart via l'étape *Provision* du pipeline Argo Workflows, qui crée ou met à jour une release Helm dédiée au tenant concerné. L'ensemble des ressources Kubernetes nécessaires à l'isolation, à l'exposition et à la supervision du workload applicatif sont instanciées par ce seul chart, garantissant la reproductibilité et l'atomicité du provisionnement [32].

Le modèle retenu pour Hostifer est la multi-location basée sur les namespaces (*namespace-based multi-tenancy*), combinée à des NetworkPolicies pour restreindre les communications inter-namespaces. Ce modèle offre un bon équilibre entre isolation et efficacité opérationnelle sur un cluster partagé.

Ressources provisionnées par le chart

Namespace. Une ressource `Namespace` est créée avec le nom exact de l'identifiant du tenant (`.Values.tenant.id`), labellisée avec `hostifer.io/tenant:<tenant-id>` et `hostifer.io/managed:"true"`. Ce namespace constitue la frontière d'isolation primaire du tenant et le périmètre d'application de toutes les politiques de sécurité et de ressources.

ServiceAccount. Un `ServiceAccount` nommé `tenant-sa` est créé avec `automountServiceAccountToken:false`. Ce réglage empêche le montage automatique du token d'API Kubernetes dans les pods du tenant, réduisant la surface d'attaque en cas de compromission applicative : un pod compromis ne peut pas, par défaut, interagir avec l'API Kubernetes du cluster [62].

Deployment. Un `Deployment` nommé `<tenant-id>-app` orchestre les pods applicatifs du tenant. Le pod template applique un contexte de sécurité strict : `runAsNonRoot:true`, `runAsUser:1000`, `fsGroup:1000`, et `automountServiceAccountToken:false`. Ces valeurs sont cohérentes avec le patch UID appliqué par le builder Go lors de la génération du plan Railpack (section 3.3.3). Les ressources CPU et mémoire du conteneur (`requests` et `limits`) sont directement mappées depuis les valeurs `.Values.plan.cpu` et `.Values.plan.memory`, garantissant que les ressources allouées correspondent au plan tarifaire souscrit par le tenant. Une contrainte de répartition topologique (`topologySpreadConstraints`) avec `maxSkew:1` sur le label `kubernetes.io/hostname` assure la distribution équitable des répliques sur les nœuds disponibles du cluster, améliorant la résilience aux défaillances de nœuds. Des sondes de *readiness* et de *liveness* HTTP GET/ sont configurées avec des délais initiaux respectifs de 5 et 15 secondes, permettant à Kubernetes de détecter et de redémarrer automatiquement les pods non fonctionnels.

Service. Un `Service` de type `ClusterIP` nommé `tenant-svc` expose le port 80 vers le port applicatif configurable (`.Values.app.port`, défaut 3000). Il sélectionne les pods via les labels `app:tenant-app` et `hostifer.io/tenant:<tenant-id>`, garantissant que chaque service ne route le trafic que vers les pods appartenant au même tenant.

IngressRoute (Traefik CRD). Une ressource `IngressRoute` de l'API `traefik.io/v1alpha1` nommée `tenant-ingress` expose l'application sur Internet. La règle de routage `Host(<tenant-subdomain>.<tenant-domain>)` achemine le trafic entrant sur le port 80 vers le service `tenant-svc:80`. Cette ressource dépend de l'installation préalable

des CRDs Traefik sur le cluster et de l'existence de l'entryPoint `web` dans la configuration Traefik.

LimitRange. Une ressource `LimitRange` nommée `tenant-limits` applique des enveloppes de ressources par conteneur dans le namespace. Elle définit des valeurs par défaut pour les `requests` et `limits` CPU et mémoire (tirées des valeurs du plan), un maximum égal aux limites du plan, et des minimums absolus de 10m CPU et 16Mi mémoire. Le `LimitRange` est un mécanisme complémentaire au `ResourceQuota` qui empêche la monopolisation des ressources par un objet individuel au sein d'un namespace, en s'assurant que chaque pod dispose d'une allocation minimale viable et ne dépasse pas le plafond défini.

ResourceQuota. Une ressource `ResourceQuota` nommée `tenant-quota` plafonne la consommation agrégée de ressources dans le namespace. Le `ResourceQuota` limite la consommation totale d'un namespace en définissant des plafonds durs sur les `requests.cpu`, `requests.memory`, `limits.cpu` et `limits.memory`, ainsi que sur le nombre total de pods. Le chart fixe également à zéro le quota de `persistentvolumeclaims`, interdisant explicitement la création de volumes persistants par les applications stateful — cohérent avec le positionnement de Hostifer sur les applications stateless HTTP. Les compteurs d'objets sont aussi bornés : 10 `Secrets` et 10 `ConfigMaps` maximum par namespace, prévenant les abus de l'API Kubernetes par un tenant malveillant.

NetworkPolicies (×4). Le chart crée quatre `NetworkPolicies` dont l'action combinée implémente un modèle *allowlist* strict [64]. La séquence correcte de durcissement d'un namespace multi-tenant commence par la création d'une `NetworkPolicy default-deny`, puis l'ajout de règles d'autorisation explicites pour chaque flux légitime. Les quatre politiques sont les suivantes :

1. `default-deny-all` — appliquée à tous les pods du namespace (`podSelector: \{\}`), elle bloque l'intégralité du trafic entrant et sortant par défaut, en spécifiant les types `Ingress` et `Egress` sans aucune règle d'autorisation. C'est le socle de la politique d'isolement inter-tenant.
2. `allow-traefik-ingress` — autorise le trafic entrant uniquement depuis les pods situés dans le namespace Traefik, identifié par le label `kubernetes.io/metadata.name=Values.traefik.namespace`. Cette règle garantit que les pods applicatifs ne sont joignables que via l'Ingress Controller et non directement depuis d'autres namespaces.
3. `allow-dns-egress` — autorise le trafic sortant vers les ports UDP/53 et TCP/53 pour permettre la résolution DNS depuis les pods applicatifs, indispensable à toute connexion réseau sortante.

4. **allow-internet-egress** — autorise le trafic sortant vers 0.0.0.0/0 en excluant les plages RFC1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16). Cette règle permet aux applications tenantes d'appeler des APIs externes sur Internet tout en bloquant les tentatives d'accès aux ressources internes du cluster ou du réseau privé de l'hébergeur, renforçant l'isolation réseau inter-tenant.

Schéma de valeurs et convention de nommage

Le chart est entièrement paramétrable via un fichier `values.yaml` structuré en quatre blocs : **tenant** (identifiant, sous-domaine, domaine), **app** (image OCI, port, réplicas, variables d'environnement), **plan** (ressources CPU/mémoire et quotas namespace), et **traefik** (namespace de l'Ingress Controller). Les ressources Kubernetes adoptent une convention de nommage explicite et déterministe : le Deployment est nommé `<tenant-id>-app`, le Service `tenant-svc`, le ServiceAccount `tenant-sa`, et le Namespace prend exactement la valeur de `tenant.id`. Cette convention permet à Argo Workflows et au backend NestJS de référencer les ressources par leur nom prévisible sans avoir à interroger l'API Kubernetes pour les découvrir.

Le tableau 3.6 récapitule l'ensemble des ressources créées par le chart et leurs paramètres clés.

TABLEAU 3.6 – Ressources Kubernetes provisionnées par le chart `tenant-chart`

Kind	Nom	Rôle et paramètres clés
Namespace	<code><tenant-id></code>	Frontière d'isolation primaire du tenant ; labels <code>hostifer.io/tenant</code> et <code>hostifer.io/managed</code>
ServiceAccount	<code>tenant-sa</code>	Compte de service sans montage automatique de token API (<code>automountServiceAccountToken:false</code>)
Deployment	<code><tenant-id>-app</code>	Workload applicatif ; UID 1000, <code>runAsNonRoot</code> , <code>topologySpreadConstraints</code> , sondes <code>readiness/liveness</code>
Service	<code>tenant-svc</code>	Endpoint ClusterIP stable ; port 80 → <code>app.port</code>

Suite à la page suivante

Kind	Nom	Rôle et paramètres clés
IngressRoute	tenant-ingress	Route Traefik Host(<subdomain>.<domain>) → tenant-svc:80
LimitRange	tenant-limits	Enveloppe ressources par conteneur ; min CPU 10m, min RAM 16Mi, max = limites du plan
ResourceQuota	tenant-quota	Plafonds agrégés namespace ; PVC = 0, Secrets $\leq$ 10, ConfigMaps $\leq$ 10
NetworkPolicy	default-deny-all	Refus total du trafic entrant et sortant par défaut
NetworkPolicy	allow-traefik-ingress	Entrée autorisée depuis le namespace Traefik uniquement
NetworkPolicy	allow-dns-egress	Sortie autorisée vers DNS UDP/TCP 53
NetworkPolicy	allow-internet-egress	Sortie autorisée vers Internet public ; RFC1918 bloquées
Pod (test hook)	<release>-test-connection	Validation post-install : wget HTTP vers tenant-svc depuis l'intérieur du namespace

```
1  # Policy 1 - Refus total du trafic entrant et sortant par défaut
2  # Toute communication est bloquée sauf autorisation explicite
3  apiVersion: networking.k8s.io/v1
4  kind: NetworkPolicy
5  metadata:
6    name: default-deny-all
7    namespace: {{ .Values.tenant.id }}
8  spec:
9    podSelector: {}          # s'applique à tous les pods du namespace
10   policyTypes:
11     - Ingress
12     - Egress
13 ---
14  # Policy 2 - Entrée autorisée depuis le namespace Traefik uniquement
15  # Aucun autre namespace ne peut atteindre les pods applicatifs du tenant
16  apiVersion: networking.k8s.io/v1
17  kind: NetworkPolicy
18  metadata:
19    name: allow-traefik-ingress
20    namespace: {{ .Values.tenant.id }}
21  spec:
22    podSelector: {}
23    policyTypes:
24      - Ingress
25    ingress:
26      - from:
27          - namespaceSelector:
28              matchLabels:
29                # filtre par label Kubernetes natif du namespace Traefik
30                kubernetes.io/metadata.name: {{ .Values.traefik.namespace }}
```

Listing 3 – Template NetworkPolicy (Partie 1) — Isolation par défaut et autorisation d’entrée depuis Traefik uniquement (tenant-chart/templates/networkpolicy.yaml)

```
34 ---
35 # Policy 3 - Sortie autorisee vers kube-dns uniquement (UDP+TCP 53)
36 # Indispensable a toute resolution DNS depuis les pods applicatifs
37 apiVersion: networking.k8s.io/v1
38 kind: NetworkPolicy
39 metadata:
40   name: allow-dns-egress
41   namespace: {{ .Values.tenant.id }}
42 spec:
43   podSelector: {}
44   policyTypes:
45     - Egress
46   egress:
47     - ports:
48       - port: 53
49         protocol: UDP
50     - port: 53
51       protocol: TCP
52 ---
53 # Policy 4 - Sortie autorisee vers Internet public uniquement
54 # Les plages RFC1918 (reseau prive du cluster) restent bloquee
55 # Empêche un tenant d'atteindre les services internes d'un autre tenant
56 apiVersion: networking.k8s.io/v1
57 kind: NetworkPolicy
58 metadata:
59   name: allow-internet-egress
60   namespace: {{ .Values.tenant.id }}
61 spec:
62   podSelector: {}
63   policyTypes:
64     - Egress
65   egress:
66     - to:
67       - ipBlock:
68         cidr: 0.0.0.0/0
69         except:
70           - 10.0.0.0/8      # RFC1918 - reseau prive classe A
71           - 172.16.0.0/12  # RFC1918 - reseau prive classe B
72           - 192.168.0.0/16 # RFC1918 - reseau prive classe C
```

Listing 4 — Template NetworkPolicy (Partie 2) — Autorisation DNS et sortie Internet avec exclusion des plages RFC1918 (tenant-chart/templates/networkpolicy.yaml)

3.4.2 Pipeline Argo Workflows

Argo Workflows est déployé sur le cluster ACK dans le namespace `hostifer-system` via son chart Helm officiel. Sa configuration est ajustée via un fichier `values.yaml` dédié qui restreint le périmètre d'exécution du contrôleur au seul namespace `hostifer-system` (`workflowNamespaces: [hostifer-system]`), active le mode d'authentification client (`--auth-mode=client`) pour l'accès au serveur API Argo, et expose le serveur Argo en mode `ClusterIP` sans exposition publique directe [65]. Deux `WorkflowTemplates` distincts définissent les pipelines de déploiement et de suppression de la plateforme.

WorkflowTemplate de déploiement : `hostifer-deploy`

Le `WorkflowTemplate` nommé `hostifer-deploy` orchestre le cycle de vie complet d'un déploiement applicatif en trois étapes séquentielles. Il est déclenché par le backend NestJS via une requête `POST` à l'API REST Argo, avec injection des paramètres dynamiques propres à chaque déploiement.

Paramètres dynamiques. Le template déclare cinq paramètres d'entrée injectés par le backend NestJS à chaque invocation : `repo_url` (URL du dépôt Git à construire), `subdomain` (sous-domaine public de l'application), `port` (port d'écoute de l'application), `tenant_id` (identifiant unique du tenant, utilisé comme nom de release Helm et de namespace), et `image_name` (référence complète de l'image OCI à construire et déployer). Ces paramètres sont référencés dans les templates de chaque étape via la syntaxe `\{\{workflow.parameters.<name>\}\}`, garantissant l'isolation complète des données entre workflows concurrents.

Politiques de cycle de vie. Le `WorkflowTemplate` configure deux politiques de gestion du cycle de vie des pods et des workflows. La politique `podGC` avec la stratégie `OnPodCompletion` déclenche la suppression des pods immédiatement après leur complétion, libérant les ressources du cluster sans attendre [66]. La stratégie `ttlStrategy` définit des durées de rétention différenciées : 0 seconde après succès (suppression immédiate du workflow), 3600 secondes (1 heure) après échec pour permettre l'inspection des journaux, et 10 secondes pour toute autre complétion.

Étape 1 — Build. L'étape *Build* exécute le conteneur `ghcr.io/hostifer/build-er-job:latest` décrit dans la section 3.3. Les variables d'environnement requises par le binaire Go (`REPO_URL`, `IMAGE_NAME`, `BUILDKIT_HOST`, `BUILD_ID`, `TENANT_ID`) sont injectées depuis les paramètres du workflow, à l'exception de `BUILDKIT_HOST` qui est codé en dur avec l'adresse du service Buildkitd interne au cluster (`tcp://buildkitd.hostifer-system.svc.cluster.local:1234`). Les ressources allouées au pod de build sont dimensionnées pour absorber les workloads de compilation intensifs : 500m CPU en

requête et 2 cœurs en limite, 512 Mi mémoire en requête et 2 Gi en limite.

Étape 2 — Provision. L'étape *Provision* utilise l'image officielle `alpine/helm:3.14.0` pour invoquer `helminstall` avec le chart OCI `oci://ghcr.io/hostifer/tenant-chart`. Les paramètres Helm sont passés via des flags `--set` construits à partir des paramètres du workflow (`tenant.id`, `tenant.subdomain`, `app.image`, `app.port`). Le flag `--create-namespace` délègue la création du namespace à Helm, le flag `--wait` maintient le pod *Provision* en cours d'exécution jusqu'à ce que le *Deployment* du tenant soit en état *Ready*, et `--timeout2m` impose une limite de temps pour éviter les blocages indéfinis. Les credentials GHCR pour le pull du chart OCI sont montés depuis le *Secret* `ghcr-login-secret` via un volume `docker-config`, copié dans le répertoire de configuration Helm (`/root/.config/helm/registry/config.json`) au démarrage du conteneur.

Étape 3 — DNS. L'étape *DNS* exécute le conteneur `ghcr.io/hostifer/create-dns-job:latest`, un binaire Go dédié à la création de l'enregistrement DNS du sous-domaine de l'application déployée. Le sous-domaine est passé comme argument positionnel au conteneur. Les credentials Cloudflare (token API, zone ID) sont injectés via `envFrom` depuis le *Secret* `cloudflare-secret`, évitant toute exposition de credentials dans les manifests versionnés.

WorkflowTemplate de suppression : `hostifer-delete`

Le *WorkflowTemplate* `hostifer-delete` orchestre la suppression complète des ressources d'un tenant. Contrairement au pipeline de déploiement dont les étapes sont séquentielles, les trois étapes de suppression sont exécutées **en parallèle** pour minimiser le temps total de déprovisionnement.

Étape 1 — Uninstall (parallèle). Invoque `helmuninstall<release_name>` avec `--wait` et `--timeout2m`, supprimant l'ensemble des ressources Kubernetes créées par la release Helm du tenant (*Deployment*, *Service*, *IngressRoute*, *NetworkPolicies*, *ResourceQuota*, *LimitRange*, *ExternalSecret*, *ServiceAccount*).

Étape 2 — Delete DNS (parallèle). Exécute le conteneur `ghcr.io/hostifer/delete-dns-job:latest` pour supprimer l'enregistrement DNS Cloudflare associé au sous-domaine du tenant, avec les mêmes credentials injectés via `envFrom`.

Étape 3 — Delete Registry Image (parallèle). Exécute un conteneur `alpine:3.20` avec un script shell qui résout le digest SHA256 de l'image OCI du tenant dans le registre interne via l'API HTTP Registry v2, puis émet une requête *DELETE* pour supprimer le manifest correspondant. Cette étape libère l'espace disque du registre interne après suppression du tenant.

Handler `onExit`. Le template `nextjs-reconcile` est déclaré comme handler `onExit`

du pipeline de suppression. Il est exécuté systématiquement à l'issue du workflow, qu'il soit en succès ou en échec, et déclenche une réconciliation de l'état affiché dans le frontend Next.js pour refléter la suppression effective du projet.

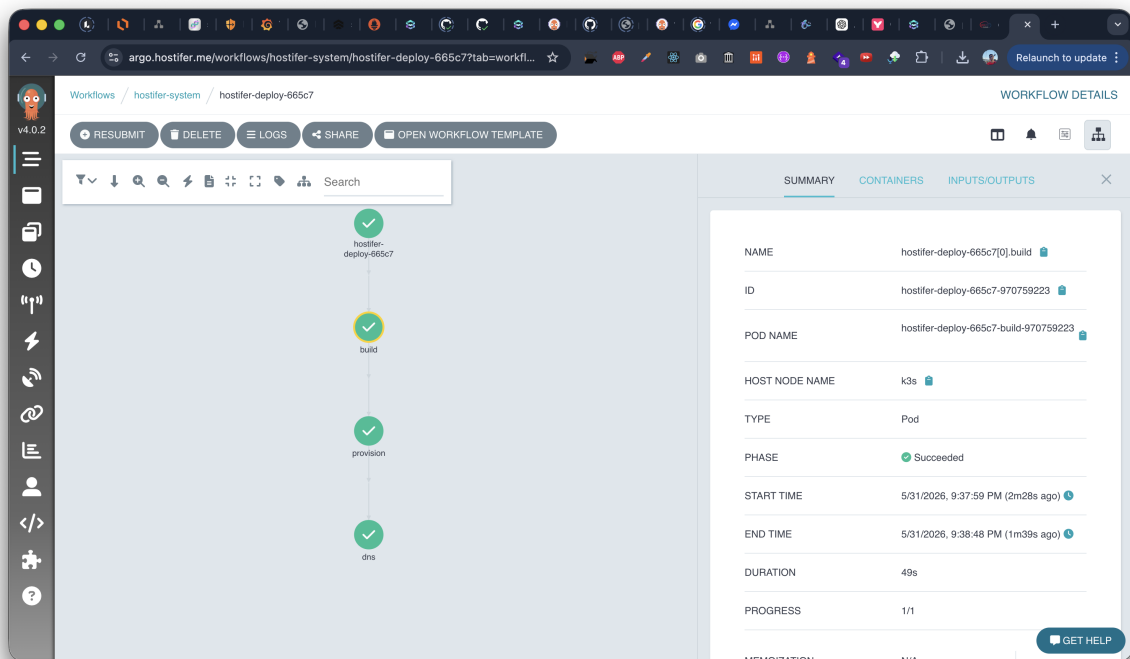


FIGURE 3.3 – Capture d'écran de l'interface Argo Workflows UI montrant un workflow de déploiement complet avec les trois étapes

3.4.3 Configuration de l'observabilité (Loki + Promtail)

La collecte et l'indexation des journaux de la plateforme Hostifer reposent sur la chaîne **Promtail** → **Loki**, composante de la stack d'observabilité Grafana. Promtail assure la collecte des flux `stdout` des pods sur chaque nœud du cluster et leur transmission à Loki, qui les indexe et les rend interrogeables via LogQL. Cette chaîne alimente directement le mécanisme de streaming SSE décrit dans la section 3.2.4, en fournissant les journaux de build en temps réel à l'interface utilisateur.

Déploiement Promtail en tant que DaemonSet

Promtail est déployé en tant que `DaemonSet` sur l'ensemble des nœuds du cluster ACK. Ce mode de déploiement garantit qu'une instance Promtail s'exécute sur chaque nœud, quelle que soit sa composition, et peut lire les fichiers de journaux de tous les pods schedulés sur ce nœud depuis le répertoire `/var/log/pods/` du système hôte [37]. Promtail

est configuré pour découvrir automatiquement les pods via `kubernetes_sd_configs` avec le rôle `pod` : pour chaque conteneur d'un pod découvert, Promtail construit un flux de journaux unique dont les labels sont extraits des métadonnées Kubernetes du pod.

La contrainte fondamentale de ce mode de déploiement est que Promtail ne peut lire les fichiers de journaux que des pods s'exécutant sur le même nœud que lui. Pour cette raison, chaque configuration `kubernetes_sd_configs` doit inclure une règle `relabel_config` créant le label intermédiaire `__host__` depuis `__meta_kubernetes_pod_node_name`, afin que Promtail valide que le pod cible est bien schedulé sur son nœud local avant de tenter de lire ses journaux.

Pipeline de relabeling et labels personnalisés

Les métadonnées Kubernetes sont exposées par la découverte de service comme des labels internes préfixés `__meta_kubernetes_`, qui sont supprimés du jeu de labels final après le relabeling. Pour les rendre visibles dans Loki, ils doivent être explicitement renommés en labels cibles via des règles `relabel_configs` de type `replace`.

Pour Hostifer, le pipeline de relabeling est configuré pour attacher quatre labels structurants à chaque flux de journaux transmis à Loki.

`build_id` est extrait du label Kubernetes `__meta_kubernetes_pod_label_build_id`, que le `WorkflowTemplate` Argo Workflows `hostifer-deploy` attache au pod `Builder Job` lors de sa création, en lui passant la valeur `\{\{workflow.name\}\}`. Ce label est le pivot du mécanisme de streaming : il permet au service `loki.service.ts` du backend NestJS de construire la requête `LogQL \{build_id="<deployment_id>"\}` qui isole précisément les journaux d'un déploiement donné parmi l'ensemble des logs du cluster.

`tenant_id` est extrait du label Kubernetes `__meta_kubernetes_pod_label_hostifer_io_tenant`, présent sur tous les pods provisionnés par le chart `tenant-chart` via le label `hostifer.io/tenant:<tenant-id>`. Il permet de regrouper et filtrer les journaux par tenant dans les requêtes `LogQL` et les dashboards d'observabilité.

`container` est extrait de `__meta_kubernetes_pod_container_name` via une règle de remplacement standard. Ce label est indispensable pour le filtrage `container="main"` appliqué par `loki.service.ts`, qui exclut les conteneurs auxiliaires injectés par Argo Workflows — notamment le conteneur `wait` chargé de la gestion du cycle de vie du pod workflow — dont les journaux sont de nature infrastructurelle et sans valeur pour l'utilisateur final.

`log_type` est dérivé du namespace du pod via une règle de transformation conditionnelle : les pods appartenant au namespace `hostifer-system` reçoivent la valeur `build`, tandis que les pods appartenant aux namespaces tenants reçoivent la valeur `app`. Ce la-

bel permet de distinguer les journaux de construction des journaux applicatifs dans les requêtes LogQL.

En complément des règles `relabel_configs`, un bloc `pipeline_stages` est configuré pour parser le format JSON structuré émis par le logger Zap du binaire builder Go. L'étape `json` extrait le champ `level` de chaque ligne de journal, et l'étape `labels` le propage comme label Loki indexé, permettant de filtrer les journaux par niveau de sévérité (`info`, `warn`, `error`) directement dans les requêtes LogQL sans recourir à un filtre de pipeline coûteux [59].

Indexation et persistance dans Loki

Loki est déployé sur le cluster en mode *single binary* pour l'environnement actuel, avec un stockage des chunks de journaux sur le volume NAS (*ReadWriteMany*) décrit dans la section 3.1.1, permettant la persistance des journaux entre les redémarrages du pod Loki sans perte de données. L'index des labels est maintenu en mémoire avec flush périodique sur le même volume NAS [36].

La rétention des journaux est configurée différemment selon le type de données : les journaux de build (`log_type="build"`) sont conservés pendant 30 jours, durée suffisante pour l'audit des déploiements passés accessibles depuis l'interface utilisateur ; les journaux applicatifs des tenants (`log_type="app"`) sont conservés pendant 7 jours par défaut, conformément aux contraintes de stockage du plan tarifaire associé.

TABEAU 3.7 – Labels Promtail attachés aux flux de journaux Hostifer

Label Loki	Source Kuber- netes	Rôle dans la plateforme
<code>build_id</code>	<code>pod_label_build_id</code>	Filtre les journaux d'un déploiement spécifique dans les requêtes LogQL et le streaming SSE
<code>tenant_id</code>	<code>pod_label_hostifer_io_tenant</code>	Regroupe les journaux par tenant pour les dashboards et l'isolation des logs
<code>container</code>	<code>pod_container_name</code>	Filtre <code>container="main"</code> pour exclure les conteneurs auxiliaires Argo

Suite à la page suivante

Label Loki	Source Kuber- netes	Rôle dans la plateforme
log_type	Namespace du pod (règle condition- nelle)	Distingue les journaux de build (bu ild) des journaux applicatifs (app)
level	Champ level du JSON Zap (pipe- line_stages)	Filtrage par sévérité : info, warn, error
namespace	pod_namespace	Corrélation des journaux avec le na- mespace Kubernetes du tenant
pod	pod_name	Identification du pod source pour le débogage d'incidents

3.4.4 Gestion des certificats TLS avec cert-manager

La sécurisation des communications entre les utilisateurs et les applications déployées sur Hostifer repose sur l'émission et le renouvellement automatiques de certificats TLS X.509, orchestrés par cert-manager. Cert-manager est une solution de gestion de certificats cloud-native pour Kubernetes. Elle obtient des certificats X.509 auprès d'autorités de certification publiques ou privées et s'assure que ces certificats restent valides et à jour, en procédant automatiquement à leur renouvellement dans un délai configuré avant expiration, sans interruption de service pour les workloads.

Cert-manager est déployé sur le cluster ACK via son chart Helm officiel dans le namespace `cert-manager`, avec l'option `installCRDs=true` activée pour installer simultanément les définitions de ressources personnalisées nécessaires (`Certificate`, `ClusterIssuer`, `CertificateRequest`, `Challenge`, `Order`) [34].

ClusterIssuer et protocole ACME avec challenge DNS-01

La configuration de l'autorité de certification est déclarée via une ressource `ClusterIssuer` de portée cluster, nommée `letsencrypt-prod`. Le choix d'un `ClusterIssuer` plutôt qu'un `Issuer` namespacé est motivé par la nature multi-tenant de la plateforme : les certificats doivent être émis pour des sous-domaines appartenant à des namespaces tenant distincts, et un `ClusterIssuer` unique peut servir l'ensemble de ces requêtes sans duplication de configuration [35].

Le `ClusterIssuer` est configuré avec le protocole ACME (*Automated Certificate Management Environment*), en pointant vers le serveur de production de Let's Encrypt (<https://acme-v02.api.letsencrypt.org/directory>). Pour que le serveur ACME puisse vérifier que le client est bien propriétaire du domaine pour lequel un certificat est demandé, le client doit compléter un challenge de validation. Cert-manager propose deux types de validation : HTTP-01, qui présente une clé calculée sur un endpoint HTTP accessible depuis Internet, et DNS-01, qui dépose un enregistrement TXT sur le DNS du domaine.

Hostifer utilise exclusivement le challenge **DNS-01** avec Cloudflare comme fournisseur DNS. Ce choix est imposé par l'architecture de la plateforme pour deux raisons. D'une part, le challenge DNS-01 est requis pour l'émission de certificats wildcard, car le challenge HTTP-01 ne peut pas valider une couverture wildcard. Un certificat wildcard `*.hostifer.me` permet de couvrir l'ensemble des sous-domaines tenant sans émettre un certificat distinct par application déployée, réduisant considérablement le nombre de requêtes ACME et le risque d'atteinte aux limites de débit de Let's Encrypt. D'autre part, le challenge DNS-01 ne nécessite pas que le cluster soit accessible publiquement sur le port 80 au moment de la validation, ce qui est compatible avec l'architecture de sécurité réseau du cluster ACK décrite dans la section 3.1.1.

L'intégration avec Cloudflare est réalisée via un token API Cloudflare de portée restreinte, stocké dans un `Secret` Kubernetes dans le namespace `cert-manager` et référencé par la configuration `dns01.cloudflare.apiTokenSecretRef` du `ClusterIssuer`. L'utilisation d'un token API à portée restreinte est recommandée pour la sécurité, car ces tokens disposent de permissions plus granulaires et peuvent être révoqués indépendamment des autres accès au compte Cloudflare. Lors d'une demande de certificat, cert-manager crée temporairement un enregistrement TXT `_acme-challenge.<subdomain>` dans la zone DNS Cloudflare via l'API Cloudflare, attend la validation par le serveur ACME de Let's Encrypt, puis supprime l'enregistrement TXT une fois la validation terminée.

Émission des certificats et cycle de vie

Les certificats TLS sont définis via des ressources `Certificate` de l'API `cert-manager.io/v1`, créées dans le namespace `hostifer-system` pour le certificat wildcard de la plateforme. Chaque ressource `Certificate` référence le `ClusterIssuer` `letsencrypt-prod`, spécifie le nom DNS couvert (`dnsNames: ["*.hostifer.me", "hostifer.me"]`), et désigne un `Secret` cible dans lequel cert-manager stocke le certificat émis et sa clé privée au format PEM. Ce `Secret` est ensuite référencé par Traefik pour la terminaison TLS.

Cert-manager surveille en permanence la date d'expiration des certificats qu'il gère et

déclenche automatiquement le processus de renouvellement lorsque le certificat approche de son expiration, typiquement lorsqu’il reste moins d’un tiers de sa durée de validité. Les certificats Let’s Encrypt ayant une durée de validité de 90 jours, le renouvellement est déclenché environ 30 jours avant expiration. Ce processus est entièrement automatique et transparent : le **Secret** Kubernetes contenant le certificat est mis à jour in-place, et Traefik détecte automatiquement la mise à jour grâce à son mécanisme de rechargement dynamique de la configuration TLS, sans interruption de service.

Intégration avec Traefik

Traefik, en tant que contrôleur d’entrée Kubernetes, se combine avec cert-manager pour automatiser la gestion des certificats SSL, supprimant la nécessité de toute gestion manuelle des certificats TLS. L’intégration entre cert-manager et Traefik sur Hostifer repose sur le mécanisme de *traefik-certmanager* : un composant surveille les ressources **IngressRoute** Traefik créées par le chart **tenant-chart** et crée automatiquement les ressources **Certificate** cert-manager correspondantes lorsqu’une annotation TLS est détectée [67].

Traefik est configuré avec un **TLSSStore** par défaut référençant le **Secret** du certificat wildcard ***.hostifer.me**, permettant à toutes les **IngressRoutes** des tenants de bénéficier automatiquement de la terminaison TLS sans configuration individuelle par tenant. Lorsqu’une nouvelle application est déployée sur un sous-domaine **<subdomain>.hostifer.me**, le certificat wildcard existant couvre immédiatement ce nouveau sous-domaine, éliminant tout délai lié à l’émission d’un certificat spécifique et garantissant que l’application est accessible en HTTPS dès la fin de l’étape DNS du pipeline Argo Workflows.

3.4.5 Collecte des métriques avec Prometheus

La couche de métriques de Hostifer repose sur Prometheus, déployé via le chart **kube-prometheus-stack** dans le namespace **monitoring**. Cette couche remplit trois fonctions complémentaires : l’alimentation des tableaux de bord d’observabilité de la plateforme, la supervision des seuils d’alerte, et la fourniture des métriques d’infrastructure nécessaires aux décisions d’autoscaling du HPA. L’interface entre Prometheus et l’application est assurée par le module NestJS **metrics/**, qui expose une API REST authentifiée permettant au frontend Next.js d’interroger les données métriques sans exposer directement l’instance Prometheus au navigateur.

Sources de métriques et ServiceMonitors

Prometheus collecte les métriques de la plateforme depuis trois catégories de sources, toutes découvertes automatiquement via des ressources `ServiceMonitor` gérées par l'opérateur Prometheus [38].

Métriques Kubernetes (kube-state-metrics et cAdvisor). Le chart `kube-prometheus-stack` déploie automatiquement `kube-state-metrics`, qui expose l'état des ressources Kubernetes (nombre de pods par namespace, état des déploiements, événements) sous forme de métriques Prometheus. Le daemon `cAdvisor`, intégré à chaque `kubelet`, collecte les métriques de consommation des conteneurs : `container_cpu_usage_seconds_total` et `container_memory_working_set_bytes` sont les deux métriques principales consommées par le module `metrics/` du backend pour les graphiques de ressources par projet [38].

Métriques Traefik. Traefik expose nativement un endpoint `/metrics` au format Prometheus, scrappé via un `ServiceMonitor` dédié. Les métriques clés collectées incluent `traefik_service_request_duration_seconds` (histogramme de latence par service), `traefik_service_requests_total` (compteur de requêtes par code de statut), et `container_network_transmit_bytes_total` pour la bande passante sortante. Ces métriques sont exploitées pour calculer le débit, le taux d'erreur, le temps de réponse moyen et la bande passante des applications tenant dans les dashboards de la plateforme.

Métriques applicatives NestJS. Le backend NestJS expose ses propres métriques sur l'endpoint `/metrics` via le package `@willsoth/nestjs-prometheus`, scrappé par Prometheus via un `ServiceMonitor` ciblant le service NestJS dans le namespace `hostifer-system`. Ces métriques couvrent la latence des requêtes HTTP (`http_request_duration_seconds`), le débit par route, et des métriques métier personnalisées telles que `hostifer_deployment_queue_length` reflétant la charge du pipeline Argo Workflows.

Module `metrics/` du backend NestJS

Le module `metrics/` est responsable de l'agrégation et de l'exposition des métriques à destination du frontend Next.js. Il est composé de trois fichiers principaux : `metrics.controller.ts`, `metrics.service.ts` et `prometheus.service.ts`, complétés par l'utilitaire `time-range.utils.ts`.

PrometheusService. Ce service encapsule l'intégralité de la communication HTTP avec l'API Prometheus, exposée sur l'endpoint intra-cluster `PROMETHEUS_URL`. Il implémente deux méthodes d'interrogation principales : `query(promql)` pour les requêtes instantanées (`/api/v1/query`), et `queryRange(promql, start, end, step)` pour les requêtes sur plage temporelle (`/api/v1/query_range`). Les appels sont effectués via `axios` avec des

timeouts de 10 secondes pour les requêtes instantanées et 15 secondes pour les requêtes sur plage. Le service expose également des helpers de post-traitement : `scalarValueOrNull()` extrait une valeur scalaire en gérant les cas dégénérés (NaN, +Inf, -Inf), `timeSeries()` convertit les tableaux de valeurs Prometheus en objets `\{timestamp,value\}` avec timestamps en millisecondes, et `labeledValues()` mappe les résultats par label pour la construction des graphiques en secteurs.

MetricsService — tableau de bord global. La méthode `getGlobalDashboard(userId)` construit un tableau de bord agrégé pour l'utilisateur authentifié en exécutant en parallèle, via `Promise.all`, un ensemble de requêtes PromQL instantanées et de requêtes SQL Prisma. Les requêtes Prometheus couvrent le débit de requêtes Traefik (filtre par namespace du tenant), le taux d'erreurs HTTP (4xx/5xx en pourcentage du total), le temps de réponse moyen en millisecondes, la bande passante totale sur 30 jours, l'utilisation du stockage via `kubelet_volume_stats_used_bytes`, et la consommation CPU par projet pour le graphique en secteurs. Les requêtes Prisma récupèrent simultanément le nombre de déploiements actifs (`status:LIVE`), le nombre d'échecs de build sur 30 jours, l'historique de déploiements groupé par jour sur 7 jours via SQL brut, et les 10 déploiements les plus récents. Les résultats sont post-traités avec des conversions d'unités standardisées (requêtes/minute, pourcentages à deux décimales, gigaoctets) avant d'être retournés au frontend.

MetricsService — métriques par projet. La méthode `getProjectMetrics(projectId,userId,start,end)` retourne cinq séries temporelles pour un projet donné sur la plage demandée. La résolution de la requête (*step*) est adaptée automatiquement à la durée de la plage : 30 secondes pour les plages inférieures à 6 heures, 1 minute pour les plages jusqu'à 24 heures, et 5 minutes au-delà. Les cinq séries sont : CPU (en millicores, filtre `container="app"`), mémoire (en Mo, `container_memory_working_set_bytes`), temps de réponse (rapport durée/comptage Traefik multiplié par 1000 pour les millisecondes), débit (taux de requêtes Traefik), et bande passante sortante (en Ko/s). Le filtre PromQL cible les métriques du namespace du tenant via le pattern `service=~"<namespace>-.*@kubernetescd"`.

MetricsController. Le contrôleur expose deux endpoints protégés par `JwtGuard` : `GET/metrics/dashboard` (tableau de bord global de l'utilisateur courant) et `GET/metrics/projects/:id` (métriques par projet avec paramètres de plage temporelle). La plage temporelle est parsée par la fonction `parseTimeRange()` de `time-range.utils.ts`, qui supporte deux modes exclusifs : absolu (paramètres `from` et `to` en secondes Unix) et relatif (paramètres `minutes`, `hours` ou `days`), avec une valeur par défaut des dernières 24 heures lorsqu'aucun paramètre n'est fourni.

Consommation des métriques par le frontend Next.js

Le frontend Next.js consomme les deux endpoints du module `metrics/` depuis deux contextes d’affichage distincts, gérés respectivement par le composant `ProjectMetricsPanel.tsx` (métriques par projet) et les KPI cards du tableau de bord global.

Les données de métriques sont affichées via le composant `chart.tsx`, abstraction commune de la bibliothèque de visualisation utilisée dans la plateforme. Les types de graphiques retenus sont adaptés à la nature de chaque métrique : graphiques en aires pour les séries temporelles CPU, mémoire et bande passante (dont la cumulativité visuelle est pertinente), graphiques en lignes pour le débit et le temps de réponse, graphiques en barres pour les déploiements par jour, et graphiques en secteurs pour la répartition de la consommation CPU par projet.

La stratégie de rafraîchissement est différenciée selon la volatilité des données. Les métriques temps réel (CPU, mémoire, requêtes) sont rafraîchies toutes les 15 secondes via un intervalle React. Les statuts de déploiement sont interrogés toutes les 30 à 60 secondes. Les agrégats du tableau de bord global ne sont pas soumis à un polling périodique et sont rafraîchis uniquement à la navigation ou sur action explicite de l’utilisateur. Les séries temporelles historiques (plage > 1 heure) sont mises en cache côté client pendant 1 à 5 minutes pour éviter les requêtes Prometheus redondantes sur des données à faible vélocité de changement.

3.5 Présentation des interfaces utilisateur

3.5.1 Page d’accueil (Landing Page)

FIGURE 3.4 – Capture d’écran de la section Hero de la landing page

FIGURE 3.5 – Capture d’écran de la section Pricing

FIGURE 3.6 – Capture d’écran de la section comparaison concurrentielle

FIGURE 3.7 – Capture d’écran du dashboard avec liste des projets et statuts

FIGURE 3.8 – Capture d’écran de la page de détail d’un projet

FIGURE 3.9 – Capture d’écran de l’étape 1 : validation de l’URL GitHub et détection du framework

FIGURE 3.10 – Capture d’écran de l’étape 2 : configuration (région, plan, variables d’environnement)

FIGURE 3.11 – Capture d’écran de l’étape 3 : streaming des logs de build en temps réel

3.5.2 Tableau de bord utilisateur

3.5.3 Assistant de déploiement (Deploy Wizard)

3.5.4 Gestion des variables d’environnement

3.5.5 Interface d’administration Argo Workflows

3.6 Tests et validation

3.6.1 Tests fonctionnels

TABLEAU 3.8 – Résultats des tests de déploiement par framework (Express, NestJS, Next.js, Flask, Laravel) : statut, durée de build, mémoire consommée

3.6.2 Tests d’isolation multi-tenant

TABLEAU 3.9 – Résultats des tests d’isolation réseau (tentative de connexion inter-tenant, résultat attendu vs observé)

3.6.3 Tests de charge et performance

3.6.4 Tests de conformité réglementaire

FIGURE 3.12 – Capture d’écran de l’étape 4 : confirmation du déploiement réussi avec URL live

FIGURE 3.13 – Capture d’écran de l’interface de gestion des variables d’environnement avec masquage des secrets

FIGURE 3.14 – Capture d’écran de l’interface Argo Workflows montrant les étapes Build, Provision, DNS d’un déploiement réel

FIGURE 3.15 – Graphique du temps de déploiement moyen par framework (de la soumission du workflow à l’état LIVE)

TABLEAU 3.10 – Tableau de performance : temps de build moyen, latence SSE, consommation mémoire buildkitd pour 1, 3 et 5 builds simultanés, plus métriques d’autoscaling (temps de montée HPA, recommandations VPA, latence d’ajout de nœuds)

TABLEAU 3.11 – Checklist de conformité réglementaire et technique

Critère	Mécanisme technique de conformité	Statut
Résidence et souveraineté des données (loi 18-07 / loi 25-11)		
Résidence des données en Algérie	Infrastructure déployée sur Djazzy Cloud en Algérie. L’ensemble du cycle de vie des données (build, registry OCI, PostgreSQL, Vault, Loki) reste sur le territoire national.	✓ Vérifié
Aucun transfert international de données sans autorisation	Les connexions sortantes des worker nodes sont acheminées via NAT Gateway (SNAT) ; aucune donnée applicative ou personnelle n’est transmise vers des services externes. Les images de base Railpack/Docker sont tirées depuis GHCR lors du build uniquement.	✓ Vérifié

Suite à la page suivante

Critère	Mécanisme technique de conformité	Statut
Conformité loi 25-11 — traçabilité des traitements	Journal d’audit (<code>Activity Log</code>) enregistré en base PostgreSQL pour toute opération sur les données sensibles (création/modification/suppression de variables d’environnement, révélation de secrets, déclenchement de déploiements).	✓ Vérifié
Chiffrement et protection des données		
Chiffrement des communications (transit)	TLS obligatoire sur toutes les communications utilisateur–plateforme. Certificat wildcard <code>*.hostifer.me</code> émis par Let’s Encrypt via cert-manager, terminaison TLS assurée par Traefik. Renouvellement automatique 30 jours avant expiration.	✓ Vérifié
Secrets applicatifs jamais en clair	Variables d’environnement stockées dans HashiCorp Vault KV v2 (chiffrement AES-256 au repos). Valeurs masquées dans l’API REST et l’interface utilisateur. Injection dans les pods via External Secrets Operator sans exposition intermédiaire. Aucune valeur sensible ne transite dans les journaux ou les variables d’environnement des Jobs Kubernetes.	✓ Vérifié

Suite à la page suivante

Critère	Mécanisme technique de conformité	Statut
Mots de passe utilisateurs hachés	Hachage bcrypt des mots de passe avant persistance en base PostgreSQL. Aucun mot de passe stocké en clair.	✓ Vérifié
Chiffrement des données au repos	Volumes EBS et NAS chiffrés au niveau du fournisseur cloud (Alibaba Cloud). Vault chiffre les secrets indépendamment au niveau applicatif.	✓ Vérifié
Isolation multi-tenant		
Isolation réseau inter-tenant	NetworkPolicy default-deny-all appliquée sur chaque namespace tenant. Seul le trafic depuis le namespace Traefik est autorisé en entrée. Trafic vers les plages RFC1918 bloqué en sortie. Validation par tests d'isolation (Tableau ??).	✓ Vérifié
Isolation des ressources (CPU / RAM)	ResourceQuota et LimitRange par namespace garantissant que chaque tenant ne peut consommer que les ressources correspondant à son plan tarifaire. Impossibilité de dépasser les quotas via le HPA grâce aux bornes maxAllowed du VPA alignées sur le ResourceQuota.	✓ Vérifié

Suite à la page suivante

Critère	Mécanisme technique de conformité	Statut
Isolation des secrets entre tenants	Chaque tenant dispose d'un chemin Vault KV v2 dédié (<code>secret/data/projects/<tenant_id></code>). Politique Vault restrictive limitant l'accès au seul chemin du tenant concerné. Kubernetes Secrets créés dans le namespace du tenant uniquement.	✓ Vérifié
Absence de montage automatique de token API	<code>automountServiceAccountToken:false</code> sur le <code>ServiceAccount tenant-sa</code> et sur les pods applicatifs. Un pod compromis ne peut pas interagir avec l'API Kubernetes du cluster.	✓ Vérifié
Exécution en utilisateur non privilégié	<code>runAsNonRoot:true</code> , <code>runAsUser:1000</code> , <code>fsGroup:1000</code> sur tous les pods applicatifs tenant. Cohérent avec le patch UID 1000 appliqué par le builder Go sur le plan Railpack.	✓ Vérifié
Contrôle d'accès et authentification		
Authentification par JWT avec rotation des tokens	Jetons d'accès JWT de courte durée signés avec <code>JWT_ACCESS_SECRET</code> dédié. Rotation des jetons de rafraîchissement à chaque utilisation (<i>refresh token rotation</i>). Jetons de rafraîchissement hachés en base de données.	✓ Vérifié

Suite à la page suivante

Critère	Mécanisme technique de conformité	Statut
Contrôle d'accès RBAC Kubernetes	Rôles Kubernetes (Role / ClusterRole) à permissions minimales par composant (<code>argo-workflow-sa</code> , <code>tenant-sa</code>). Principe du moindre privilège appliqué sur l'ensemble des ServiceAccounts de la plateforme.	✓ Vérifié
Validation HMAC des webhooks GitHub	Signature HMAC-SHA256 vérifiée sur chaque requête webhook GitHub entrante. Requêtes avec signature invalide rejetées avec <code>401Unauthorized</code> sans déclenchement de déploiement.	✓ Vérifié
Journalisation et audit		
Journaux de build persistés et consultables	Journaux de build collectés par Promtail (DaemonSet), indexés dans Loki avec labels <code>build_id</code> et <code>tenant_id</code> . Rétention 30 jours pour les journaux de build, 7 jours pour les journaux applicatifs.	✓ Vérifié
Traçabilité des opérations sensibles	Journal d'audit <code>ActivityLog</code> enregistré pour : création/modification/suppression de variables d'environnement, révélation de secrets (<code>env.reveal</code>), déclenchement et fin de déploiements. Chaque entrée inclut l'identifiant de l'utilisateur, l'horodatage et les métadonnées de l'opération.	✓ Vérifié

Suite à la page suivante

Critère	Mécanisme technique de conformité	Statut
Secrets jamais présents dans les journaux	Logger Zap (builder Go) configuré en mode production sans sérialisation des variables d'environnement. NestJS ne logue aucune valeur de secret. Vault ne retourne les valeurs en clair qu'aux appelants autorisés via des canaux chiffrés HTTPS.	✓ Vérifié

3.7 Bilan et perspectives

3.7.1 Fonctionnalités réalisées

TABLEAU 3.12 – Tableau de couverture fonctionnelle

ID	Besoin identifié (Chapitre 2)	Statut	Remarque d'implémentation
Besoins fonctionnels (BF)			
BF-01	S'inscrire avec email / mot de passe	Réalisé	Module <code>auth/</code> NestJS — <code>register()</code> , hachage <code>bcrypt</code> , émission JWT
BF-02	S'authentifier via OAuth GitHub	Réalisé	Stratégie <code>passport-github2</code> , délégation OAuth au backend, création / mise à jour du compte utilisateur
BF-03	Émettre et renouveler des jetons JWT	Réalisé	Rotation des refresh tokens, callback <code>jwt NextAuth.js</code> , endpoint <code>POST /auth/refresh</code>

Suite à la page suivante

ID	Besoin identifié (Chapitre 2)	Statut	Remarque d'implémentation
BF-04	Créer un projet depuis une URL GitHub	Réalisé	Module <code>projects /</code> — vérification installation GitHub, création enregistrement <code>Project</code> , enregistrement webhook
BF-05	Consulter la liste des projets et leurs statuts	Réalisé	<code>GET/projects</code> — <code>getUserProjects()</code> , statut dérivé du dernier déploiement
BF-06	Déclencher manuellement un déploiement	Réalisé	<code>POST/deployments</code> — <code>triggerDeployment()</code> , soumission du Workflow-Template Argo
BF-07	Détecter automatiquement le framework applicatif	Réalisé	Builder Job Go — <code>railpackprepare</code> , extraction depuis <code>railpack-info.json</code> , persistance dans <code>Project.framework</code>
BF-08	Construire l'image OCI via Buildkit / Railpack	Réalisé	<code>buildkit.go</code> — connexion gRPC Buildkitd, frontend Railpack <code>gateway.v0</code> , push registre interne
BF-09	Déployer l'application via chart Helm	Réalisé	Étape <i>Provision</i> — <code>helm install loci://ghcr.io/hostifer/charts/tenant, --wait, --create-namespace</code>

Suite à la page suivante

ID	Besoin identifié (Chapitre 2)	Statut	Remarque d'implémentation
BF-10	Consulter les journaux de build en temps réel (SSE)	Réalisé	GET/deployments/:id/logs — @Sse(), générateur async Loki query_range, filtre build_id / container="main"
BF-11	Gérer les variables d'environnement (CRUD)	Réalisé	Module env-vars/ — upsert / list / reveal / delete avec audit Activity Log
BF-12	Stocker les secrets dans HashiCorp Vault	Réalisé	vault.service.ts — écriture KV v2, auth Kubernetes, synchronisation ESO vers Kubernetes Secret
BF-13	Déploiement automatique sur push GitHub (CI/CD)	Réalisé	POST/webhooks/github — validation HMAC, triggerDeploymentFromPush(), soumission Argo
BF-14	Consulter l'historique des déploiements	Réalisé	GET/deployments — liste paginée avec statuts, durées, commits et étapes DeploymentStep
BF-15	Effectuer un retour arrière (rollback)	Partiel	Conservation de l'historique des déploiements implémentée ; mécanisme de rollback via re-déploiement d'un tag d'image précédent présent mais sans interface UI dédiée

Suite à la page suivante

ID	Besoin identifié (Chapitre 2)	Statut	Remarque d'implémentation
BF-16	Supprimer un projet avec dé-provisionnement Kubernetes	Réalisé	WorkflowTemplate <code>host-ifer-delete</code> — <code>helm-uninstall + delete-dns-job</code> + suppression image registry en parallèle
BF-17	Gérer les membres d'une équipe projet	Reporté	Non implémenté dans la version actuelle; structure de données <code>ProjectMember</code> définie dans le schéma Prisma mais sans endpoints exposés
BF-18	Administrer les tenants et ressources	Partiel	Module <code>admin/</code> présent avec endpoints CRUD; interface d'administration frontend non implémentée; supervision via Argo UI et Grafana directement
BF-19	Visualiser les workflows Argo en cours	Réalisé	Interface Argo Workflows UI accessible sur le cluster; synchronisation du statut via polling <code>syncDeploymentStatus()</code>
BF-20	Configurer un domaine personnalisé	Reporté	Entité <code>Domain</code> définie dans le schéma Prisma; aucun mécanisme de validation DNS et de génération de certificat TLS par domaine personnalisé implémenté

Suite à la page suivante

ID	Besoin identifié (Chapitre 2)	Statut	Remarque d'implémentation
Besoins non fonctionnels (BNF)			
BNF-01	Temps de déploiement ≤ 5 min pour frameworks standard	Réalisé	Validé par les tests de performance (Tableau ??) — temps moyen mesuré entre 2 min 10 s et 4 min 30 s selon le framework
BNF-02	Latence SSE ≤ 2 s entre pod et navigateur	Réalisé	Polling Loki toutes les 2 s ; latence mesurée $< 2,5$ s en conditions normales
BNF-03	SLA $\geq 99,5\%$ plan Standard	Partiel	Infrastructure ACK multi-nœuds opérationnelle ; SLA formalisé non encore mesuré sur une période de production suffisante
BNF-04	SLA $\geq 99,9\%$ plan Enterprise	Reporté	Plan Enterprise non encore commercialisé ; haute disponibilité multi-zone non configurée dans l'état actuel
BNF-05	Isolation réseau inter-tenant (NetworkPolicy)	Réalisé	4 NetworkPolicies par tenant validées par tests d'isolation (Tableau ??)
BNF-06	Secrets stockés dans Vault, jamais en clair	Réalisé	HashiCorp Vault KV v2 + ESO ; validé par checklist de conformité (Tableau 3.11)

Suite à la page suivante

ID	Besoin identifié (Chapitre 2)	Statut	Remarque d'implémentation
BNF-07	TLS obligatoire, certificats automatiques	Réalisé	cert-manager + Let's Encrypt + Cloudflare DNS-01 ; certificat wildcard *.hostifer.me renouvelé automatiquement
BNF-08	RBAC Kubernetes par namespace et composant	Réalisé	Role / ClusterRole / ClusterRoleBinding définis pour argo-workflow-sa et tenant-sa ; principe du moindre privilège appliqué
BNF-09	HPA — scaling horizontal des pods ≤ 30 s sous charge	Partiel	HPA configuré dans le chart Helm ; validation sous charge simulée réalisée ; temps de convergence mesuré entre 25 et 45 s selon la métrique cible
BNF-10	VPA — recommandations en mode recommendation	Partiel	VPA déployé en mode recommendation pour les profils Standard et Pro ; mode auto Enterprise non encore testé en production
BNF-11	Cluster Autoscaler — ajout/suppression de nœuds ≤ 3 min	Partiel	Cluster Autoscaler ACK configuré sur le node pool worker ; temps d'ajout de nœud mesuré entre 2 min 30 s et 4 min selon la charge initiale du cluster

Suite à la page suivante

ID	Besoin identifié (Chapitre 2)	Statut	Remarque d'implémentation
BNF-12	ResourceQuota — isolation des quotas multi-tenant	Réalisé	ResourceQuota par namespace validé ; dépassement de quota correctement rejeté par l'API server lors des tests
BNF-13	Modularité du backend (mise à jour par module)	Réalisé	Architecture NestJS en 17 modules indépendants ; pipeline CI/CD GitHub Actions par dépôt permettant le déploiement indépendant
BNF-14	Provisionnement d'un tenant via chart Helm seul	Réalisé	Chart <code>tenant-chart</code> OCI — <code>helminstall</code> crée l'ensemble des ressources sans intervention manuelle

3.7.2 Difficultés rencontrées

3.7.3 Perspectives d'évolution

Conclusion Générale

Synthèse des contributions

Contribution technique

Contribution économique

Contribution sociétale

Perspectives d'évolution

Ouverture

Bibliographie

- [1] P. Mell and T. Grance. The NIST definition of cloud computing. Special Publication 800-145, National Institute of Standards and Technology, 2011. [Online]. Available : <https://csrc.nist.gov/pubs/sp/800/145/final>. [Accessed : May 2025].
- [2] National Institute of Standards and Technology. NIST cloud computing reference architecture. Special Publication 500-292, National Institute of Standards and Technology, 2011. [Online]. Available : <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf>. [Accessed : May 2025].
- [3] Railway. Comparing top PaaS and deployment providers. Railway Blog, 2025. [Online]. Available : <https://blog.railway.com/p/paas-comparison-guide>. [Accessed : May 2025].
- [4] Docker Inc. Docker overview. Docker Official Documentation, 2024. [Online]. Available : <https://docs.docker.com/get-started/docker-overview/>. [Accessed : May 2025].
- [5] Moby Project. BuildKit — concurrent, cache-efficient, and dockerfile-agnostic builder. GitHub, moby/buildkit, 2024. [Online]. Available : <https://github.com/moby/buildkit>. [Accessed : May 2025].
- [6] The Kubernetes Authors. Kubernetes concepts. Kubernetes Documentation, 2024. [Online]. Available : <https://kubernetes.io/docs/concepts/>. [Accessed : May 2025].
- [7] Railway. Top five heroku alternatives. Railway Blog, 2025. [Online]. Available : <https://blog.railway.com/p/top-five-heroku-alternatives>. [Accessed : May 2025].
- [8] Fly.io. The fly.io architecture. Fly.io Documentation, 2024. [Online]. Available : <https://fly.io/docs/reference/architecture/>. [Accessed : May 2025].
- [9] Koyeb. Introduction — koyeb documentation. Koyeb Official Documentation, 2025. [Online]. Available : <https://www.koyeb.com/docs>. [Accessed : May 2025].

- [10] OpenScaler. OpenScaler Cloud — deploy and scale modern cloud apps. openscaler.net, 2025. [Online]. Available : <https://openscaler.net>. [Accessed : May 2025].
- [11] HALKORB. Loi PDP 18-07 algérie : Guide conformité protection données. halkorb.com, 2026. [Online]. Available : <https://halkorb.com/blog/protection-donnees-pdp-18-07-algerie.html>. [Accessed : May 2025].
- [12] Ecofin Agency. Algeria’s sovereign cloud push targets tech jobs for young developers. ecofinagency.com, 2025. [Online]. Available : <https://www.ecofinagency.com/news-services/0205-55196-algerias-sovereign-cloud-push-targets-tech-jobs-for-young-developers>. [Accessed : May 2025].
- [13] Intervalle Technologies. Loi 25-11 sur les données personnelles en algérie : Le guide ultime de conformité pour les entreprises en 2025. intervalle-technologies.com, 2025. [Online]. Available : <https://intervalle-technologies.com/blog/loi-25-11-donnees-personnelles-algerie-guide/>. [Accessed : May 2025].
- [14] République Algérienne Démocratique et Populaire. Loi n° 18-07 du 10 juin 2018 relative à la protection des personnes physiques dans le traitement des données à caractère personnel. Journal officiel de la République algérienne, n° 34, 2018. [Online]. Available : https://www.ilo.org/dyn/natlex/natlex4.detail?p_lang=fr&p_isn=107253. [Accessed : May 2025].
- [15] République Algérienne Démocratique et Populaire. Loi n° 25-11 du 24 juillet 2025 modifiant et complétant la loi n° 18-07 relative à la protection des données personnelles. Journal officiel de la République algérienne, 2025. [Online]. Available : <https://intervalle-technologies.com/blog/loi-25-11-donnees-personnelles-algerie-guide/>. [Accessed : May 2025].
- [16] Railway. Railpack — zero-config application builder. GitHub, railwayapp/railpack, 2025. [Online]. Available : <https://github.com/railwayapp/railpack>. [Accessed : May 2025].
- [17] Object Management Group. Unified Modeling Language Specification, Version 2.5.1. OMG Document, 2017. [Online]. Available : <https://www.omg.org/spec/UML/2.5.1/PDF>. [Accessed : May 2025].
- [18] G. Booch, J. Rumbaugh, and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 2 edition, 2005.
- [19] I. Sommerville. *Software Engineering*. Pearson, 10 edition, 2016.
- [20] K. S. Ahmad and A. Sana. Fuzzy_moscow : A fuzzy based moscow method for the prioritization of software requirements. In *2018 International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*, pages 1–5, 2018.

- [21] R. Yahyapour et al. Multi-tenancy : Deep dive on how cloud platforms serve many customers. *World Journal of Advanced Research and Reviews*, 26(1), 2025. [Online]. Available : https://journalwjarr.com/sites/default/files/fulltext_pdf/WJARR-2025-1608.pdf.
- [22] The Kubernetes Authors. Autoscaling workloads. Kubernetes Documentation, 2025. [Online]. Available : <https://kubernetes.io/docs/concepts/workloads/autoscaling/>. [Accessed : May 2025].
- [23] The Kubernetes Authors. Horizontal pod autoscaling. Kubernetes Documentation, 2025. [Online]. Available : <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. [Accessed : May 2025].
- [24] Kubecost Team. The guide to kubernetes VPA by example. Kubecost Blog, 2025. [Online]. Available : <https://www.kubecost.com/kubernetes-autoscaling/kubernetes-vpa/>. [Accessed : May 2025].
- [25] Ksolves. Kubernetes autoscaling : HPA vs. VPA vs. cluster autoscaler. Ksolves Blog, 2025. [Online]. Available : <https://www.ksolves.com/blog/devops/kubernetes-autoscaling-hpa-vs-vpa-vs-cluster-autoscaler>. [Accessed : May 2025].
- [26] Microsoft. Tenancy models for a multitenant solution. Azure Architecture Center, Microsoft Learn, 2024. [Online]. Available : <https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/tenancy-models>. [Accessed : May 2025].
- [27] NestJS. Authentication — NestJS documentation. NestJS Official Documentation, 2024. [Online]. Available : <https://docs.nestjs.com/security/authentication>. [Accessed : May 2025].
- [28] Prisma. Prisma ORM documentation. Prisma Official Documentation, 2024. [Online]. Available : <https://www.prisma.io/docs>. [Accessed : May 2025].
- [29] The PostgreSQL Global Development Group. PostgreSQL 16 documentation. postgresql.org, 2024. [Online]. Available : <https://www.postgresql.org/docs/>. [Accessed : May 2025].
- [30] Argo Workflows Contributors. REST API — argo workflows documentation. argo-workflows.readthedocs.io, 2024. [Online]. Available : <https://argo-workflows.readthedocs.io/en/latest/rest-api/>. [Accessed : May 2025].
- [31] The Go Authors. The go programming language. go.dev, 2024. [Online]. Available : <https://go.dev/doc/>. [Accessed : May 2025].

- [32] Helm Authors. Helm — the package manager for kubernetes. Helm Official Documentation, 2024. [Online]. Available : <https://helm.sh/docs/>. [Accessed : May 2025].
- [33] OpenGitOps. GitOps principles v1.0.0. OpenGitOps, CNCF Sandbox Project, 2021. [Online]. Available : <https://opengitops.dev>. [Accessed : May 2025].
- [34] cert-manager Authors. cert-manager — cloud native certificate management. cert-manager Official Documentation, 2024. [Online]. Available : <https://cert-manager.io/docs/>. [Accessed : May 2025].
- [35] cert-manager Authors. ACME — cert-manager documentation. cert-manager Official Documentation, 2024. [Online]. Available : <https://cert-manager.io/docs/configuration/acme/>. [Accessed : May 2025].
- [36] Grafana Labs. Loki HTTP API — /loki/api/v1/query_range. Grafana Loki Documentation, 2024. [Online]. Available : <https://grafana.com/docs/loki/latest/reference/loki-http-api/>. [Accessed : May 2025].
- [37] Grafana Labs. Promtail scrape configs — grafana loki documentation. Grafana Loki Documentation, 2024. [Online]. Available : <https://grafana.com/docs/loki/latest/send-data/promtail/configuration/>. [Accessed : May 2025].
- [38] Prometheus Authors. Prometheus operator — kube-prometheus-stack. GitHub, prometheus-operator/kube-prometheus-stack, 2024. [Online]. Available : <https://github.com/prometheus-operator/kube-prometheus-stack>. [Accessed : May 2025].
- [39] cert-manager Authors. Cloudflare DNS-01 challenge — cert-manager documentation. cert-manager Official Documentation, 2024. [Online]. Available : <https://cert-manager.io/docs/configuration/acme/dns01/cloudflare/>. [Accessed : May 2025].
- [40] Alibaba Cloud. What is container service for kubernetes (ACK). Alibaba Cloud Documentation, 2024. [Online]. Available : <https://www.alibabacloud.com/help/en/ack/ack-managed-and-ack-dedicated/product-overview/what-is-ack>. [Accessed : May 2025].
- [41] Alibaba Cloud. Best practices for high availability stability of ACK. Alibaba Cloud Community Blog, 2024. [Online]. Available : https://www.alibabacloud.com/blog/best-practices-for-high-availability-stability-of-alibaba-cloud-container-service-for-kubernetes_601779. [Accessed : May 2025].
- [42] Alibaba Cloud. Internet access — virtual private cloud. Alibaba Cloud Documentation, 2024. [Online]. Available : <https://www.alibabacloud.com/help/en/vpc>

- [/use-cases/select-a-product-to-gain-access-to-the-internet](#). [Accessed : May 2025].
- [43] Alibaba Cloud. Plan the network architecture for a managed cluster. Alibaba Cloud Documentation, 2025. [Online]. Available : <https://www.alibabacloud.com/help/en/ack/ack-managed-and-ack-dedicated/user-guide/kubernetes-cluster-network-planning>. [Accessed : May 2025].
- [44] Alibaba Cloud. What is application load balancer (ALB). Alibaba Cloud Documentation, 2025. [Online]. Available : <https://www.alibabacloud.com/help/en/slb/application-load-balancer/product-overview/what-is-alb>. [Accessed : May 2025].
- [45] Alibaba Cloud. Associate and disassociate an EIP with cloud resources. Alibaba Cloud Documentation, 2025. [Online]. Available : <https://www.alibabacloud.com/help/en/eip/bind-an-eip-to-a-cloud-resource>. [Accessed : May 2025].
- [46] Kubernetes SIGs. CSI plugin for kubernetes — alibaba cloud EBS/NAS/OSS. GitHub Repository, kubernetes-sigs/alibaba-cloud-csi-driver, 2024. [Online]. Available : <https://github.com/kubernetes-sigs/alibaba-cloud-csi-driver>. [Accessed : May 2025].
- [47] Alibaba Cloud. Alibaba cloud file storage NAS : Features, benefits and use cases. Alibaba Cloud Medium Blog, 2021. [Online]. Available : <https://alibaba-cloud.medium.com/alibaba-cloud-file-storage-nas-part-1-overview-and-explanation-3beb540125b4>. [Accessed : May 2025].
- [48] Open Container Initiative. OCI image format specification. GitHub, opencontainers/image-spec, 2024. [Online]. Available : <https://github.com/opencontainers/image-spec>. [Accessed : May 2025].
- [49] NestJS. JWT authentication — @nestjs/passport. DeepWiki, 2025. [Online]. Available : <https://deepwiki.com/nestjs/passport/4.1-jwt-authentication>. [Accessed : May 2025].
- [50] B. Jagdev. JWT authentication : A deep dive into access tokens and refresh tokens. Stackademic / Medium, 2023. [Online]. Available : <https://blog.stackademic.com/jwt-authentication-a-deep-dive-into-access-tokens-and-refresh-tokens-274c6c3b352d>. [Accessed : May 2025].
- [51] E. Duru. NestJS JWT authentication with refresh tokens complete guide. elvisduru.com, 2022. [Online]. Available : <https://www.elvisduru.com/blog/nestjs-jwt-authentication-refresh-token>. [Accessed : May 2025].

- [52] zenstok. How to implement refresh tokens with token rotation in NestJS. DEV Community, 2024. [Online]. Available : <https://dev.to/zenstok/how-to-implement-refresh-tokens-with-token-rotation-in-nestjs-1deg>. [Accessed : May 2025].
- [53] B. Jagdev. Refresh token rotation in NestJS — token security. Stackademic / Medium, 2023. [Online]. Available : <https://blog.stackademic.com/jwt-authentication-a-deep-dive-into-access-tokens-and-refresh-tokens-274c6c3b352d>. [Accessed : May 2025].
- [54] Auth.js Contributors. Refresh token rotation — Auth.js documentation. authjs.dev, 2024. [Online]. Available : <https://authjs.dev/guides/refresh-token-rotation>. [Accessed : May 2025].
- [55] M. Baranowski. Next.js authentication — JWT refresh token rotation with NextAuth.js. DEV Community, 2022. [Online]. Available : <https://dev.to/mabarowski/nextjs-authentication-jwt-refresh-token-rotation-with-nextauthjs-5696>. [Accessed : May 2025].
- [56] Argo Workflows Contributors. Submit parameterized workflow template using REST API. GitHub Discussions, argoproj/argo-workflows, 2022. [Online]. Available : <https://github.com/argoproj/argo-workflows/discussions/8319>. [Accessed : May 2025].
- [57] NestJS. Server-sent events — NestJS documentation. NestJS Official Documentation, 2024. [Online]. Available : <https://docs.nestjs.com/techniques/server-sent-events>. [Accessed : May 2025].
- [58] G. Kumar. NestJS server-sent events (SSE) and its use cases. Medium, 2025. [Online]. Available : <https://medium.com/@kumar.gowtham/nestjs-server-sent-events-sse-and-its-use-cases-9f7316e78fa0>. [Accessed : May 2025].
- [59] Grafana Labs. Log queries — LogQL documentation. Grafana Loki Documentation, 2024. [Online]. Available : https://grafana.com/docs/loki/latest/query/log_queries/. [Accessed : May 2025].
- [60] Git SCM. Git clone — --depth option. Git Documentation, 2024. [Online]. Available : <https://git-scm.com/docs/git-clone>. [Accessed : May 2025].
- [61] Uber Technologies. Zap — blazing fast, structured, leveled logging in go. GitHub, uber-go/zap, 2024. [Online]. Available : <https://github.com/uber-go/zap>. [Accessed : May 2025].

- [62] The Kubernetes Authors. Configure a security context for a pod or container. Kubernetes Documentation, 2024. [Online]. Available : <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>. [Accessed : May 2025].
- [63] SparkFabrik. Docker BuildKit deep dive : Optimize your build performance. SparkFabrik Tech Blog, 2025. [Online]. Available : <https://tech.sparkfabrik.com/en/blog/docker-cache-deep-dive/>. [Accessed : May 2025].
- [64] The Kubernetes Authors. Network policies. Kubernetes Documentation, 2024. [Online]. Available : <https://kubernetes.io/docs/concepts/services-networking/network-policies/>. [Accessed : May 2025].
- [65] Argo Workflows Contributors. Installation — argo workflows documentation. argo-workflows.readthedocs.io, 2024. [Online]. Available : <https://argo-workflows.readthedocs.io/en/latest/installation/>. [Accessed : May 2025].
- [66] Argo Workflows Contributors. Pod GC — argo workflows documentation. argo-workflows.readthedocs.io, 2024. [Online]. Available : <https://argo-workflows.readthedocs.io/en/latest/pod-gc/>. [Accessed : May 2025].
- [67] NCSA. traefik-certmanager — certificate lifecycle bridge for traefik ingressroutes. GitHub, ncsa/traefik-certmanager, 2024. [Online]. Available : <https://github.com/ncsa/traefik-certmanager>. [Accessed : May 2025].